14 haftalık yol haritasında 8. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

HAFTA 08

Dinamik Diziler ve Bağlı Listeler

Bellekte Esnek Yapılar

MODÜL III · DİNAMİK VERİ VE HIZLI ERİŞİM · H8–H10

> HAFTANIN BÜYÜK SORUSU
>
> *Bir dizinin ortasına eleman eklemek için bağlı liste her zaman daha mı hızlıdır?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları.....4
1.1 Öğrenme çıktıları.....4
2. Geçen Haftadan ve AP-1'den Bağlantı.....4
3. Kavramsal Anlatım: Bitişik ve Bağlı Bellek.....5
3.1 Bitişik ve bağlı yerleşim.....5
3.2 Soyut veri tipi: arayüz ve gerçekleştirim.....7
3.3 Bağlı listenin dili: baş, NULL, kuyruk, kukla düğüm ve Dugum **.....8
3.4 Kaputun Altında: önbellek, düğüm yükü ve realloc.....8
4. Algoritmalar.....9
4.1 Dinamik dizi: sona ekleme ve amortize analiz.....9
4.2 Tek bağlı liste: ekleme, arama ve silme.....12
4.3 Listeyi ters çevirme.....16
4.4 İki sıralı listeyi birleştirme (kukla düğüm).....19
4.5 Floyd döngü tespiti (kaplumbağa ve tavşan).....22
4.6 Listede birleştirmeli sıralama.....25
5. Çözümlü Örnekler ve Bellek Sahipliği.....27
5.1 Sahiplik sözleşmesi ve listenin yok edilmesi (Örnek 6).....27
5.2 Araçlarla doğrulama: ASan, UBSan, LSan ve Valgrind.....28
5.3 Ortadaki düğüm ve sondan k'ıncı (Örnek 9).....29
5.4 Çift bağlı dairesel liste ve O(1) silme (Örnek 11).....30
6. Haftanın Deneyi: Dizi mi Liste mi, Gerçekten?.....30
6.1 Ölçüm düzeni.....30
6.2 Sonuç: C ve C++'ta ölçülen her n'de dizi daha hızlı.....31
6.3 Neden? Konum bulma ve önbellek.....32
6.4 Büyüme stratejisi: kopya sayısı ve süre.....34
6.5 Python: ters yönlü bulgu.....34
6.6 Yorum, sınırlılıklar ve Büyük Soru'nun yanıtı.....35
7. Hata Avcısı.....35
8. Dil Penceresi: C · C++ · Python.....36
9. Mühendis Gözüyle: Önce Ölç, Sahipliği Yaz.....37
10. Mini-OBS 2 v2.7: Bekleme Listesi ve Bağlı Listede Birleştirme.....38
10.1 Gereksinimler ve ipuçları.....38
10.2 Kabul ölçütleri ve test senaryoları.....39
11. Alıştırmalar ve Atölyeler.....40

11.1 İzleme ve çıktı tahmini (A) 40
11.2 Karmaşıklık (B) 41
11.3 Değişmez yazma (C) 42
11.4 Hata bulma (D) 42
11.5 Parsons (E) 43
11.6 Kod yazma (F) 43
11.7 Diller arası okuma (G) 44
11.8 Atölye 1 · Kutu–ok izleme: altı liste işlemi (sınıf, ≈ 15 dk) 44
11.9 Atölye 2 · Liste kütüphanesi ve birim testleri (eşli, ≈ 30 dk) 45
11.10 Atölye 3 · Sızıntı avı (bireysel ya da eşli, ≈ 20 dk) 45
12. Kendini Değerlendirme 45
13. İleri Bakış ve Merak Kutusu 46
14. Özet, Kavram Haritası ve Terim Sözlüğü 47
15. Kaynaklar 48
Ek A. Dil Karşılıkları 49
A.1 Örnek 1 · Dinamik dizi ADT'si 49
A.2 Örnek 2 · Büyüme stratejisi 50
A.3 Örnek 3 · Tek bağlı liste: başa ekleme 51
A.4 Örnek 4 · Sona ekleme ve arama 51
A.5 Örnek 5 · Değere göre silme 52
A.6 Örnek 6 · Listenin yok edilmesi ve sahiplik (Δ) 53
A.7 Örnek 7 · Ters çevirme 54
A.8 Örnek 8 · Sıralı listeleri birleştirme 55
A.9 Örnek 9 · Ortadaki ve sondan k'ıncı düğüm 56
A.10 Örnek 10 · Floyd döngü tespiti 57
A.11 Örnek 11 · Çift bağlı dairesel liste 58
A.12 Örnek 12 · Listede birleştirmeli sıralama 58
A.13 Örnek 13 · Dizi mi liste mi? 59

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Bir dizinin ortasına eleman eklemek için bağlı liste her zaman daha mı hızlıdır?

AP-1'de bir dizinin ortasına eleman eklemek için sağındaki bütün elemanları birer konum kaydırdık: n elemanlı dizide Θ(n) iş. Bağlı listede ise yeni düğüm yalnızca iki bağlantı değiştirilerek araya girer: O(1). Ders kitaplarının karmaşıklık tabloları bunu söyler ve çoğumuzun ilk yanıtı "liste, elbette" olur. Bu hafta yanıtı ölçerek sınıyoruz. İş yükü şudur: n rastgele tamsayı, doğrusal aramayla bulunan yerine eklenerek sıralı bir yapıda tutulur, sonra rastgele konumlardan tek tek silinir. Bu makinede (C, -O2) bağlı liste $n = 10^3$'te dinamik dizinin 6,2 katı, $n = 5{\cdot}10^4$'te 72,9 katı sürdü (tek ölçüm); C++'ta `std::list` `std::vector`'ün 2,9 ile 35,7 katı arasında kaldı (Bölüm 6).

Yanıt **hayırdır** ve bu haftanın asıl konusu nedenidir. Araya ekleme iki işten oluşur: yeri **bulmak** ve **eklemek**. Listede yeri bulmak da Θ(n)'dir ve her adımı bellekte dağınık duran bir düğüme atlamaktır; ölçümde listenin süresinin %99,1–99,7'si yeri bulmaya gitti. Python'da ise sonuç tersine döndü: elle yazılmış liste, elle yazılmış dinamik diziden hızlıydı (süre oranı 0,51, 0,40, 0,61). Hangi yapının hızlı olduğu dile, gerçekleştirime ve belleğin nasıl kullanıldığına bağlıdır; bu hafta ikisini de kuruyor, kuramını yazıyor ve ölçüyoruz.

> **GERÇEK DÜNYADAN · "Liste, elbette!" Stroustrup'un sorusu**
>
> C++'ın tasarımcısı Bjarne Stroustrup, John Bentley'nin önerdiği bir alıştırmayı birçok konuşmasında kullandığını yazar: rastgele tamsayıları sıralı bir diziye ekleyin, sonra öğeleri rastgele konumlardan tek tek silin; vektör mü daha hızlıdır, bağlı liste mi? Stroustrup'a göre çoğu kişinin ilk yanıtı "liste, elbette!" olur ve bu yanıt tamamen ve çarpıcı biçimde yanlıştır. Gerekçesi önbellektir: vektör öğeleri bitişik tutar; listenin bağlantıları önceden ayrılsa bile dolaşma performansı yine öldürür (Stroustrup, 2014).
>
> Yazı ölçüm tablosu ya da grafik içermez (andığı tek oran, sıfır–iki öğelik bir listeyi vektörle değiştirmenin bir algoritmayı iki kat hızlandırdığı bir vakadır); bu notlardaki bütün süreler kendi ölçümümüzdür. Stroustrup sonucun kapsamını da belirtir: öğeler çok daha büyüdüğünde listeler eninde sonunda öne geçer, yani bulgu int gibi küçük öğeler içindir. Çıkardığı üç ders bu haftanın mühendislik ilkeleridir: veriyi gereksiz yere saklamayın, veriyi sıkışık tutun, belleğe öngörülebilir biçimde erişin (Stroustrup, 2014).

1.1 Öğrenme çıktıları

- **ÖÇ 8.1** Kapasitesini ikiye katlayarak büyüyen dinamik diziyi gerçekler ve sona eklemenin amortize O(1) olduğunu toplam yöntemiyle gösterir. (Bölüm 4.1, Örnek 1–2)
- **ÖÇ 8.2** Tek ve çift bağlı listeyi düğüm yapısıyla gerçekler; başa, sona ve araya ekleme ile silme işlemlerini sınır durumlarıyla birlikte doğru kodlar. (Bölüm 3.3, 4.2 ve 5.4, Örnek 3–5 ve 11)
- **ÖÇ 8.3** Liste algoritmalarını (ters çevirme, iki sıralı listeyi birleştirme, ortadaki düğüm, döngü tespiti) iki işaretçi tekniğiyle çözer. (Bölüm 4.3–4.6 ve 5.3, Örnek 7–10 ve 12)
- **ÖÇ 8.4** Dizi ve bağlı listeyi işlem karmaşıklığı, bellek kullanımı ve önbellek davranışı açısından deneysel olarak karşılaştırır. (Bölüm 3.4 ve 6, Örnek 13)
- **ÖÇ 8.5** Bellek sahipliğini ve yaşam döngüsünü yönetir; sızıntı ve sarkan işaretçi hatalarını AddressSanitizer ve Valgrind ile tespit eder. (Bölüm 5.1–5.2 ve 7, Örnek 6)

2. Geçen Haftadan ve AP-1'den Bağlantı

H7'de öbeği (heap) bir dizinin içinde, hiç işaretçi kullanmadan bir ağaç olarak tuttuk: düğüm i'nin çocukları 2i + 1 ve 2i + 2'deydi, komşuluk bilgisi indis aritmetiğinden geliyordu. Bu hafta komşuluğu **açıkça saklayan** yapılara geçiyoruz: her düğüm bir sonrakinin adresini taşır. Bunun için AP-1'den üç araç

gerekir: yapılar (`struct`, H14), `malloc`/`realloc`/`free` ile yönetilen öbek bellek bölgesi (H12) ve bir fonksiyonun çağıranın değişkenini işaretçiyle değiştirmesi (H8, H12). AP-1 H12'nin Örnek 12'si bu haftanın Örnek 1'inin çekirdeğiydi: 1 … 20'nin kareleri, ilk kapasite 1, dolunca ikiye katlama; çıktısı `20 eleman, kapasite 32, 1 taşınma; son eleman 400` idi. Aynı program Valgrind altında 5 taşınma saydı: `realloc`'un bloğu yerinde mi büyüteceği yoksa taşıyacağı mı ayırıcıya bağlıdır (Bölüm 3.4).

> NOT · **Terimler: öbek, yığın, düğüm, sahiplik**
>
> Bu hafta "öbek" her geçtiğinde `malloc`'un yönettiği **dinamik bellek bölgesi** kastedilir; H7'nin öbek veri yapısıyla (ikili öbek) yalnızca adı ortaktır. "Yığın" (stack) çağrı yığınıdır (özyinelemeli liste algoritmalarının derinliği orada birikir) ya da H9'un son giren ilk çıkar yapısıdır.
>
> Ayrıca: **sarkan işaretçi** (dangling pointer) serbest bırakılmış belleği gösteren işaretçidir; **sızıntı** (memory leak) hiçbir işaretçinin göstermediği, bırakılmamış bloktur; **sahiplik** (ownership) belleği kimin bırakacağıdır. Terim sözlüğü Bölüm 14'tedir.

> TAHMİN ET · **Geri çağırma: üç soru (yanıtlar aşağıda)**
>
> - **S1 (AP-1 H12).** `dizi = realloc(dizi, 2 * kap * sizeof *dizi);` satırının tehlikesi nedir? Doğrusu nasıl yazılır?
> - **S2 (AP-1 H8).** `void takas(int a, int b)` çağrıldıktan sonra `main`'deki a ve b neden değişmez? Bir fonksiyonun çağıranın **işaretçisini** değiştirebilmesi için parametresi hangi türde olmalıdır?
> - **S3 (H7).** Öbekte düğüm i'nin ebeveyni ⌊(i − 1)/2⌋ indisindedir, O(1). Tek bağlı bir listede bir düğümün **öncülünü** bulmak ne kadar sürer?

Yanıtlar. S1: `realloc` başarısız olursa NULL döndürür ve eski bloğa dokunmaz; tek işaretçi NULL ile ezildiği için eski blok kaybolur (sızıntı). Sonuç önce geçici bir işaretçiye alınır, NULL değilse atanır (Hata Avcısı 5). S2: C argümanları **değerle** aktarır, fonksiyon kopyaları değiştirir; adresleri (`int *a, int *b`) vermek gerekir. Aynı nedenle listenin baş işaretçisini değiştiren bir fonksiyon `Dugum **` alır ya da yeni başı döndürür (Bölüm 4.2, Hata Avcısı 4). S3: tek bağlı listede öncülün adresi saklanmaz; baştan yürümek gerekir, Θ(konum). Çift bağlı listede `onceki` bağlantısıyla O(1)'dir (Örnek 11'de 1000 rastgele silmede tek bağlı liste 255 893 düğüm ziyaret etti, çift bağlı 0; Bölüm 5.4).

3. Kavramsal Anlatım: Bitişik ve Bağlı Bellek

3.1 Bitişik ve bağlı yerleşim

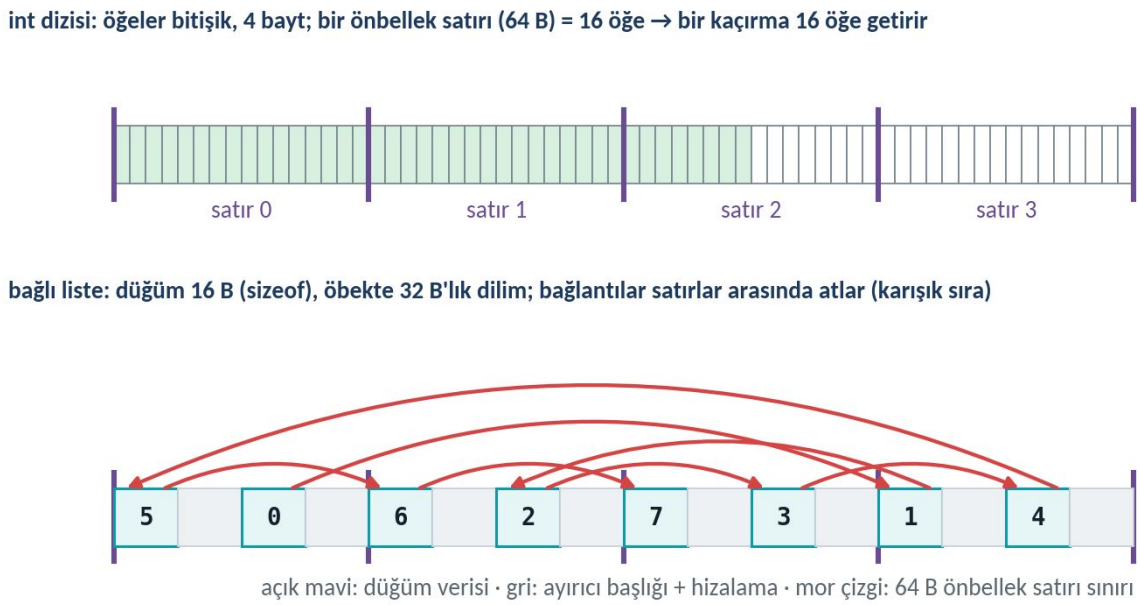
Bir **dizi** öğelerini bellekte **bitişik** tutar: a[i]'nin adresi taban adres + i · `sizeof` öğedir, bu yüzden indisle erişim O(1)'dir. Bedeli, boyutun blok ayrılırken belirlenmesidir; büyümek yeni ve daha büyük bir blok ister. **Bağlı liste** ise her öğeyi ayrı ayrılmış bir **düğümde** tutar; düğüm, verinin yanında listedeki sonraki düğümün adresini (**bağlantı**) taşır. Düğümler öbekte herhangi bir yerde olabilir. Ekleme ve silme yalnızca bağlantıları değiştirir, öğe taşınmaz; ama i. öğeye ulaşmak için baştan i bağlantı izlemek gerekir: Θ(i).

c/H08_03_tek_bagli_liste.c · satır 16–19

```c
typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;   /* sonraki düğümün adresi; son düğümde NULL */
} Dugum;
```

Düğümün bellek bedeli küçük değildir. Bu makinede (x86-64, LP64) `sizeof(int)` = 4, `sizeof(Dugum)` = 16 bayttır: 4 baytlık verinin yanında 8 baytlık işaretçi ve hizalama boşluğu vardır. glibc'nin `malloc(16)` çağrısı 24 kullanılabilir bayt verir ve art arda ayrılan 100 000 düğümün ardışık adres farkının ortancası 32 bayttır (99997/99999 çiftte tam 32). Yani int başına dizi 4 bayt, tek bağlı liste 32 bayt kullanır: **8 kat** (yalnızca `sizeof`'a bakılırsa 4 kat). Şekil 8.1 iki yerleşimi önbellek satırlarıyla birlikte gösterir; Şekil 8.2 iki yapıyı işlem işlem karşılaştırır.

Şekil 8.1
Bitişik dizi ve öbekte dağınık düğümler: 64 baytlık bir önbellek satırına 16 int ya da 2 düğüm bloğu sığar (bu makinede düğüm 16 bayt, öbek bloğu 32 bayt)



bu makinede (C): n = $3 \cdot 10^5$ düğümde dolaşma düğüm başına dizi 0,48 ns, bitişik liste 2,55 ns, karışık liste 146,0 ns

Şekil 8.2
Dinamik dizi ve tek bağlı liste: erişim, ekleme, silme, bellek ve önbellek; son satır bu makinedeki Stroustrup ölçümü (Bölüm 6)

İşlem	dinamik dizi	tek bağlı liste	çift bağlı liste
i. öğeye erişim	O(1)	Θ(i)	Θ(i)
başa ekleme / silme	Θ(n) kaydırma	O(1)	O(1)
sona ekleme	O(1) amortize	O(1) (kuyruk işaretçisiyle)	O(1)
konumu bilinen yere ekleme	Θ(n) kaydırma	O(1) (öncekinden sonra)	O(1)
adresi bilinen düğümü silme	Θ(n) kaydırma	Θ(konum): öncekini bul	O(1)
değer arama	Θ(n) (sıralıysa O(log n))	Θ(n)	Θ(n)
öğe başına bellek (int)	4 B + büyüme payı	16 B (sizeof) / 32 B blok	24 B (sizeof) / 32 B blok
önbellek	bitişik, ön-getirme dostu	dağınık; her adım bağımlı yükleme	dağınık
Stroustrup iş yükü, C, $5{\cdot}10^4$	1×	72,9× yavaş (bu makinede, tek ölçüm)	—

3.2 Soyut veri tipi: arayüz ve gerçekleştirim

Soyut veri tipi (ADT), bir veri türünü yalnızca **işlemleri ve bu işlemlerin sözleşmeleriyle** tanımlar: "sona ekle", "i. öğeyi al", "boyutu ver". Verinin bellekte nasıl tutulduğu bir **gerçekleştirim** ayrıntısıdır. C'de bu ayrım dosya düzeyinde yapılır: arayüz bir başlık dosyasında (`.h`), gerçekleştirim bir kaynak dosyasında (`.c`) durur. `typedef struct Dizi Dizi;` bildirimi tipi **opak** yapar: istemci `Dizi *` tutabilir ama alanlarına erişemez, çünkü yapının tanımı yalnızca `.c` dosyasındadır. Sahiplik sözleşmesi arayüzün parçasıdır ve yorumda yazılır:

C **kavram/H08_K01_dizi_adt/dizi.h · satır 14–33**

```c
typedef struct Dizi Dizi;   /* opak tip: alanlar yalnızca .c dosyasında */

/* Sahiplik: dönen dizi çağıranındır; dizi_yok_et ile bırakılır.
 * Bellek yetersizse NULL döner. */
Dizi *dizi_olustur(void);

/* x'i sona ekler. 0: başarı, -1: bellek yetersiz (dizi değişmez). */
int dizi_ekle(Dizi *d, int x);

/* i < boyut ise *x'e yazar ve true döner; değilse false döner. */
bool dizi_al(const Dizi *d, size_t i, int *x);

size_t dizi_boyut(const Dizi *d);

/* Öğretim amaçlı: gerçekleştirimin adı ve öğe/düğüm erişim sayacı */
const char *dizi_gerceklestirim(void);
long long dizi_adim(void);

/* Bütün belleği bırakır; d bundan sonra kullanılmaz. NULL güvenlidir. */
```

```
33  void dizi_yok_et(Dizi *d);
```

Kavram 1 (`kavram/H08_K01_dizi_adt/`) aynı arayüzün iki gerçekleştirimini verir: `dizi_dinamik.c` (× 2 büyüyen dinamik dizi) ve `dizi_bagli.c` (kuyruk işaretçili tek bağlı liste). Tek bir `main.c` yalnızca `dizi.h`'yi görür ve iki gerçekleştirimle de derlenir. Sonuçlar aynıdır (indisle okunan öğelerin toplamı iki gerçekleştirimde de 333 833 500), adım sayıları değil: 1000 öğeyi indisle okumak dinamik dizide 1 000 öğe erişimi, listede `dizi_al` her çağrıda baştan yürüdüğü için 500 500 düğüm ziyaretidir; 1000 sona ekleme ise dinamik dizide 1 023 öğe kopyası, listede 0 kopyadır. Arayüz aynı kaldıkça istemci kodu değişmeden gerçekleştirim değiştirilebilir; ama maliyet sözleşmenin parçası değilse istemci büyük bir sürprizle karşılaşabilir.

3.3 Bağlı listenin dili: baş, NULL, kuyruk, kukla düğüm ve Dugum **

Bağlı yerleşim ve aşağıdaki teknikler (kuyruk işaretçisi, kukla ya da başlık düğümü, çift bağlantı) klasik kaynaklarda ayrıntılı işlenir (Knuth, 1997; Sedgewick & Wayne, 2011). Bu derste kullandığımız sözcük dağarcığı şudur:

- **Baş işaretçisi ve boş liste.** Liste, ilk düğümü gösteren `Dugum *bas` ile temsil edilir; boş listede `bas == NULL`. Son düğümün `sonraki` alanı NULL'dır. Dolaşma her zaman aynı kalıptır: `for (p = bas; p != NULL; p = p->sonraki)`; uzunluğu bulmak Θ(n)'dir.
- **Kutu–ok gösterimi (AP-1).** Düğüm iki bölmeli bir kutudur (veri | bağlantı); ok bağlantıdır; NULL çarpıyla gösterilir. Bu derste liste metin olarak `[3 -> 1 -> 4 -> NULL]` yazılır, boş liste `[NULL]` (üç dilde aynı). Her liste işlemini önce kâğıtta kutu–okla çizin; değişen bağlantıyı kırmızıyla işaretleyin.
- **Dört sınır durumu.** Her işlem boş listede, tek düğümlü listede, baş düğümde ve son düğümde ayrıca düşünülmelidir. Hataların çoğu bu dört durumdan birinin unutulmasıdır (Hata Avcısı 1).
- **Kuyruk işaretçisi.** Liste başlığı `Liste {bas, son}` son düğümü de tutarsa sona ekleme yürümeden O(1) olur. Bedel bir **değişmezdir**: `son` her zaman son düğümü (boş listede NULL'ı) gösterir; boş listeye ilk ekleme ve son düğümün silinmesi bu yüzden `son`'u da günceller.
- **Kukla düğüm.** Listenin önüne veri taşımayan bir düğüm koymak, "önceki düğüm yok" durumunu ortadan kaldırır: her gerçek düğümün bir öncülü olur. Birleştirme (Bölüm 4.4) ve çift bağlı dairesel liste (Bölüm 5.4) bunu kullanır.
- **İşaretçiye işaretçi.** `Dugum **p` bir düğümü değil bir **bağlantıyı** gösterir: ya `bas` değişkenini ya da bir düğümün `sonraki` alanını. Baş da bir bağlantı olduğu için "baş mı siliniyor?" sorusu kalkar (Bölüm 4.2). AP-1 H12'deki işaretçiye işaretçi örneği bunun ilk adımıydı.

3.4 Kaputun Altında: önbellek, düğüm yükü ve realloc

> KAPUTUN ALTINDA · **Dağınık düğümler, önbellek kaçırmaları ve realloc'un iki yolu**
>
> **Önbellek.** İşlemci belleği 64 baytlık satırlar hâlinde getirir ve ardışık erişimi önceden sezip sonraki satırları erken yükler (ön-getirme). Dizide toplam alırken her satır 16 öğe getirir; listede ise bir sonraki düğümün adresi, o düğüm yüklenmeden bilinemez. Bu makinede (L2 2 MiB, L3 33 MiB) aynı düğümler bellekteki sırayla bağlıyken (bitişik) ve karıştırılmış sırayla bağlıyken (karışık) dolaşıldı: n = 10^3'te karışık/bitişik süre oranı 0,96, yani fark yok; n = 300 000'de 57,3× (düğüm başına 146 ns, dizide öğe başına 0,48 ns). Bitişik liste bile diziden 2,9–10,2 kat yavaştır: her adım bir öncekinin yüklenmesini bekler (Bölüm 6.3, Şekil 8.13).
>
> **realloc: yerinde büyütme ya da taşıma.** C standardına göre `realloc` eski nesneyi serbest bırakır ve istenen boyutta yeni bir nesne döndürür; dönen işaretçi eskisiyle aynı olabilir (ISO/IEC, 2018). Pratikte ayırıcı bloğun arkasındaki alan boşsa bloğu **yerinde** büyütür; değilse yeni bir blok ayırır, içeriği kopyalar ve eskisini bırakır.

Taşınmada eski bloğu gösteren **her** işaretçi geçersiz olur. Kavram 5 bloğu üç kez dört katına çıkarır ve yalnızca `realloc`'tan önce tamsayıya (`uintptr_t`) çevrilmiş adresi karşılaştırır; bu çalıştırmada turlar sırasıyla taşındı, yerinde kaldı, yerinde kaldı (sonuç ayırıcıya ve çalıştırmaya bağlıdır). C++'ta `std::vector` kapasitesi değişince yineleyiciler, başvurular ve işaretçiler aynı nedenle geçersizleşir (**yineleyici geçersizleşmesi**); güvenli seçenek indis tutmaktır (`kavram/H08_K05_realloc_gecersizlesme.c`). Kopya sayacı taşınmanın en kötü durumunu sayar; bu makinede glibc'nin `realloc`'u +1 büyütmeyle 10^6 eklemenin ölçülen (ısınmadan sonraki) tekrarlarında bloğu 0 kez taşıdı (ekleme başına 16,4 ns; taşıma sayısı ayırıcının o anki durumuna bağlıdır): model ile ölçüm ayrılabilir (Bölüm 6.4).

4. Algoritmalar

Altı algoritmayı aynı on bir adımlı standart anlatımla işliyoruz: problem, fikir, elle izleme, sözde kod, doğruluk, analiz (ve kimlik kartı), C gerçekleştirimi, üç dil karşılaştırması, test, deney ve varyantlar. İlki dinamik diziye (4.1), ötekiler bağlı listeye aittir: temel işlemler (4.2), ters çevirme (4.3), birleştirme (4.4), döngü tespiti (4.5) ve listede birleştirmeli sıralama (4.6). Bütün listeler kutu–ok gösterimiyle çizilir; değişen bağlantı kırmızıdır.

4.1 Dinamik dizi: sona ekleme ve amortize analiz

1 · Problem. Kaç öğe geleceği bilinmeden öğeleri sona eklemek ve i. öğeyi O(1)'de okumak. **2 · Fikir.** Öğeler bitişik bir blokta tutulur; **boyut** (kullanılan yuva) ile **kapasite** (ayrılmış yuva) ayrı izlenir. Kapasite dolunca **iki katı** büyüklükte yeni blok ayrılır, öğeler kopyalanır, eski blok bırakılır. Büyütme pahalıdır (boyut kadar kopya) ama seyrektir: n eklemede yalnızca $\lceil \log_2 n \rceil$ kez olur.

3 · Elle izleme. Örnek 1, 1 … 20'nin karelerini ilk kapasitesi 1 olan diziye ekler ve her eklemeden sonra boyutu, kapasiteyi, o adımdaki kopyayı ve "iş/ekleme"yi (yazma + kopya, eklemeye bölünmüş) yazar. Kapasite 1, 2, 4, 8, 16 iken dolar: 2. ekleme 1 kopya, 3. ekleme 2 kopya, 5. ekleme 4 kopya, 9. ekleme 8 kopya, 17. ekleme 16 kopya; aradaki eklemeler kopyasızdır. İş/ekleme 17. eklemede 2,82 olur ve 3'ün altında kalır (Şekil 8.3).

> **TAHMİN ET · Sizce kaç kopya?**
>
> 32 öğe eklendikten sonra kapasite 32'dir. 33. eklemede kaç öğe kopyalanır, kapasite kaç olur ve o ana kadarki iş/ekleme kaçtır? (Yanıt: 32 kopya, kapasite 64, toplam iş 96 = 33 yazma + 63 kopya; iş/ekleme 2,91. İlk 64 eklemede en büyük değer budur; 3'e yaklaşır ama ulaşmaz.)

SÖZDE KOD · 4 · Sözde kod: dinamik diziye sona ekleme (ikiye katlama)

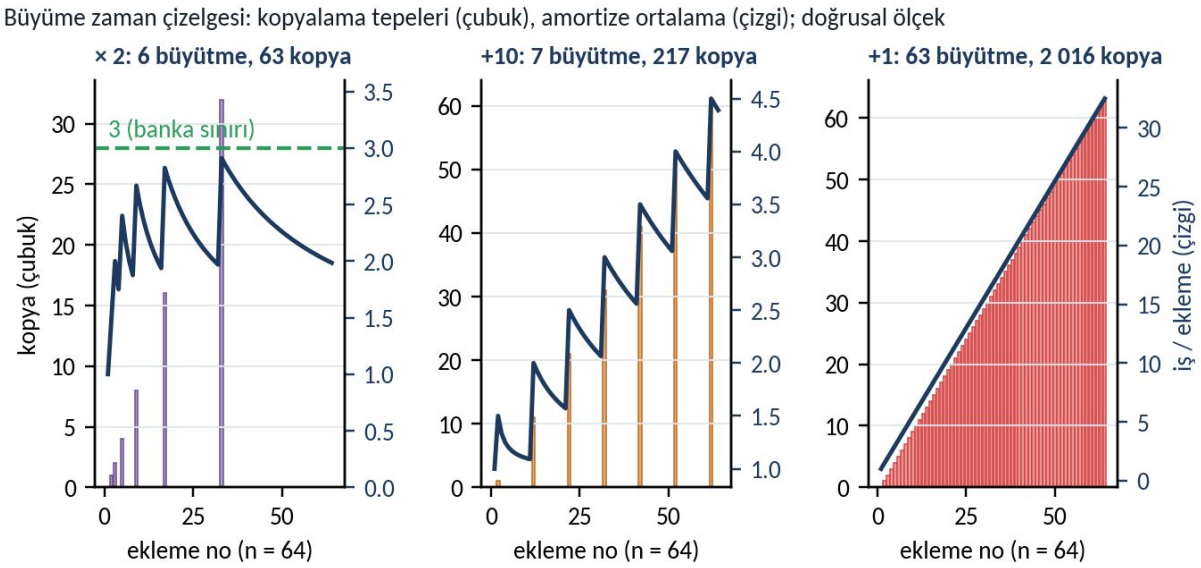
```
algoritma SONA-EKLE(d, x)              ▷ d: {veri, boyut, kapasite}
    eğer d.boyut = d.kapasite ise
        yeni ← YENİ-BLOK(2 · d.kapasite)
        her i için (0 ≤ i < d.boyut) yeni[i] ← d.veri[i]   ▷ boyut kadar kopya
        BLOĞU-BIRAK(d.veri);  d.veri ← yeni;  d.kapasite ← 2 · d.kapasite
    d.veri[d.boyut] ← x;  d.boyut ← d.boyut + 1
```

5 · Doğruluk. Değişmez (Tablo 8.1): boyut ≤ kapasite ve veri[0..boyut−1] eklenen öğelerdir; ikiye katlamada n eklemenin toplam kopyası 1 + 2 + 4 + … < 2n (her eklemeye 3 jetonluk banka sezgisi). Büyütme öğelerin sırasını ve değerini korur; ekleme yalnızca veri[boyut]'a yazar, bu yuva boyut < kapasite olduğu için ayrılmıştır. **6 · Analiz: toplam yöntemi.** Tek bir ekleme en kötü durumda O(n)'dir (büyüten ekleme). Ama n eklemenin **toplamı** küçüktür: büyütmeler boyut 1, 2, 4, …, 2^{m-1} iken olur ve toplam kopya $1 + 2 + \ldots + 2^{m-1} = 2^m - 1 < 2n$'dir. n yazma ile birlikte toplam iş < 3n, ekleme başına

amortize O(1) (Cormen vd., 2022). Örnek 1 bunu sayar: n = 20'de `toplam kopya = 31, ekleme sayısı = 20, kopya/ekleme = 1.55` ($31 = 2^5 - 1$), n = 10^6'da `toplam kopya = 1048575, kopya/ekleme = 1.049 (< 2)` ($2^{20} - 1$).

Banka (muhasebe) sezgisi. Her eklemeye 3 jeton ödediğimizi düşünelim: biri öğenin kendi yazımına harcanır, ikisi bankaya konur. Kapasite k'dan 2k'ya çıkarken son k/2 öğenin her biri 2 jeton biriktirmiştir; bu k jeton tam olarak k öğenin kopyasını öder. Bakiye hiçbir zaman negatife düşmez; bu yüzden toplam iş en çok 3n'dir. Örnek 1'in iş/ekleme sütunu bu bakiyenin aynasıdır: 64 eklemede (izlemeler.json) en büyük değer 2,91 (ekleme 33).

Şekil 8.3
Büyüme zaman çizelgesi, 64 ekleme: çubuklar o adımdaki kopya (kopyalama tepeleri), çizgi iş/ekleme (amortize ortalama); üstte × 2, altta +1 ve +10 (izlemeler.json)



Sabit miktarda büyütme neden O(n²)? Kapasite her dolduğunda yalnızca k yuva eklenirse (ilk kapasite 1) büyütmeler boyut 1, 1 + k, 1 + 2k, … iken olur; m = ⌈(n − 1)/k⌉ büyütmenin toplam kopyası m + k·m(m − 1)/2 ≈ $n^2/(2k)$'dir: toplam Θ(n^2), ekleme başına amortize Θ(n). Çarpanla büyütmede ise kopyalar geometrik bir dizi oluşturur: × 1,5'te toplam kopya son kapasitenin iki katını geçmez, son kapasite de 1,5n'den küçüktür; toplam < 3n. Örnek 2 dört stratejiyi n = 10^3, 10^4, 10^5 için sayar; +k ve × 2 satırlarını kapalı biçimle ("="), × 1,5 satırlarını 3n sınırıyla karşılaştırır; n = 10^5'te toplam kopya +1 için 4 999 950 000, +10 için 499 960 000, × 1,5 için 276 492, × 2 için 131 071'dir: öğe başına 49999,5, 4999,6, 2,765 ve 1,311.

Tablo 8.1
Dinamik diziye sona ekleme kimlik kartı

ALGORİTMA KİMLİK KARTI **Dinamik dizi: sona ekleme** *(dynamic array append (push_back))*	
Strateji	Büyüyen dizi: kapasite dolunca ikiye katlanır (yeni blok + öğelerin kopyası); amortize analiz
Girdi önkoşulu	Yok; ilk kapasite ≥ 1 (sayaç: büyütmede boyut kadar kopya)
Zaman (en iyi · ortalama · en kötü)	O(1) · O(1) amortize · O(n)
Ek bellek	O(n) toplam
Özellikler	Kapasite × 2; indisle erişim O(1). Toplam yöntemi: n = 10^6 eklemede toplam kopya 1 048 575; +1 büyütmede n = 10^5 için 4 999 950 000. Bedel: en çok n boş yuva

Değişmez / doğruluk	boyut ≤ kapasite ve veri[0..boyut−1] eklenen öğelerdir; ikiye katlamada n eklemenin toplam kopyası 1 + 2 + 4 + … < 2n (her eklemeye 3 jetonluk banka sezgisi)
Kütüphane karşılığı	C: yok (elle; `realloc`) · C++: `std::vector::push_back` (büyüme çarpanı gerçekleştirime bağlı) · Python: `list.append` (CPython fazladan ayırır)
Ne zaman kullanılır	Öğe sayısı önceden bilinmeyen, sona eklenip indisle okunan veriler: okunan kayıtlar, ölçüm dizileri, dizi tabanlı yığın
Ne zaman kullanılmaz	Tek bir eklemenin O(n) gecikmesi kabul edilemiyorsa (önceden ayırın: reserve); öğelere dışarıdan işaretçi tutuluyorsa (büyütme onları geçersiz kılar)

7 · C gerçekleştirimi. (Örnek 1; `realloc` sonucu önce geçici işaretçiye alınır, başarısızlıkta dizi değişmeden geçerli kalır):

C **c/H08_01_dinamik_dizi.c · satır 45–63**

```c
int dd_ekle(DinamikDizi *d, int x)
{
    if (d->boyut == d->kapasite) {
        size_t yeni_kapasite = 2 * d->kapasite;
        uintptr_t eski_adres = (uintptr_t)d->veri;  /* yalnız stderr için */
        int *gecici = realloc(d->veri, yeni_kapasite * sizeof *d->veri);
        if (gecici == NULL) {
            return -1;              /* d->veri hâlâ eski, geçerli blok */
        }
        tasima_sayisi += (uintptr_t)gecici != eski_adres;
        kopya_sayisi += (long long)d->boyut;  /* taşınırsa boyut kadar */
        buyutme_sayisi++;
        d->veri = gecici;
        d->kapasite = yeni_kapasite;
    }
    d->veri[d->boyut] = x;
    d->boyut++;
    return 0;
}
```

8 · Üç dil karşılaştırması. C++'ta `new[]` ile ayrılmış blok `realloc` edilemez; yeni blok, kopya döngüsü ve `delete[]` gerekir (Ek A.1). Python'da yuvalar `[None] * kapasite` ile ayrılır (yerleşik `list`'in kendi büyümesine güvenilmez; Bölüm 8):

Python **H08_01_dinamik_dizi.py · satır 32–40**

```python
    if d.boyut == d.kapasite:
        yeni_kapasite = 2 * d.kapasite
        yeni = [None] * yeni_kapasite
        for i in range(d.boyut):
            yeni[i] = d.veri[i]
        kopya_sayisi += d.boyut
        buyutme_sayisi += 1
        d.veri = yeni
        d.kapasite = yeni_kapasite
```

9 · Test. Örnek 1, 10^6 eklemeden sonra öğelerin toplamını kapalı biçimle karşılaştırır (`öğelerin toplamı = 499999500000 (beklenen 499999500000)`) ve sınır dışı `dd_al`'ı sınar (`dd_al(20): sınır dışı (boyut = 20)`). Örnek 2 gerçek bir dinamik dizinin kopya sayacını benzetimle

karşılaştırır: `× 2 : gerçek = 1023, benzetim = 1023, aynı mı? evet; toplam = 499500`. **10 · Deney.** Bu makinede elle kopyalayan C dizisinde × 2 ile ekleme başına 7,0 ns (n = 10^6); +1 ile n = 10^5 eklemenin tamamı 7,36 s sürdü (tek ölçüm). Ayrıntı ve `realloc`'un sapması Bölüm 6.4'tedir.

11 · Varyantlar. Büyüme çarpanı. × 1,5 daha sık büyütür; karşılığında en kötü durumda daha az boş yuva bırakır: büyütmeden hemen sonra kapasitenin en çok üçte biri boştur, × 2'de yarısı. Boş yuva oranı n'ye göre dalgalanır; n = 10^5'te × 1,5'te %27,7, × 2'de %23,7 (Örnek 2, 4. bölüm), yani tek bir n'de × 2 daha az yuva boş bırakabilir. İkisi de amortize O(1)'dir. × 1,5'in yuvarlaması gerçekleştirime bağlıdır: Örnek 2 yeni = k + ⌈k/2⌉ alır (1, 2, 3, 5, 8, 12, 18, 27, …); Mini-OBS 2 v2.7'nin `dinamik.h`'si k + ⌊k/2⌋ alır ve sonuç k'ya eşitse k + 1 yapar (1, 2, 3, 4, 6, 9, 13, 19, …); kopya sayıları bu yüzden farklıdır. **Küçültme:** boyut kapasitenin yarısına inince yarıya küçültmek, sınırda gidip gelen ekle–sil dizisinde her işlemi O(n) yapar; dörtte bire inince yarıya küçültmek bunu önler (Alıştırma 8.4). **Önceden ayırma:** son boyut biliniyorsa kapasite baştan ayrılır (C++ `reserve`). C++ `std::vector` ve Python `list` aynı yapının kütüphane sürümleridir (Bölüm 8).

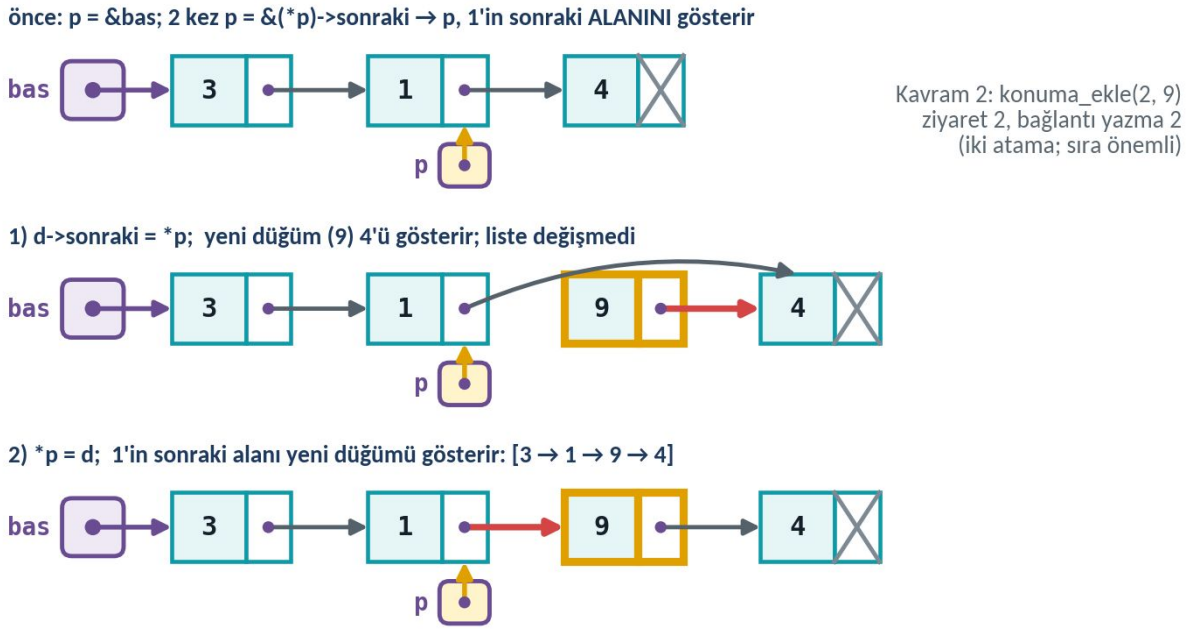
4.2 Tek bağlı liste: ekleme, arama ve silme

1 · Problem. Tek bağlı listede başa, sona ve araya ekleme; değere göre arama ve silme; boş liste, tek düğüm, baş ve son düğüm durumlarında doğru sonuç. **2 · Fikir. Başa ekleme:** yeni düğümün `sonraki`'si eski başı, `bas` yeni düğümü gösterir; yürüme yoktur. **Sona ekleme:** kuyruk işaretçisi `son` varsa son düğümün `sonraki`'si yeni düğüme bağlanır; yoksa sona kadar yürümek gerekir. **Araya ekleme ve silme:** hedefin **öncülüne** kadar yürünür, öncülün bağlantısı değiştirilir; silinen düğüm önce bağlantıdan çıkarılır, **sonra** serbest bırakılır.

3 · Elle izleme. Örnek 3 boş listeye basa_ekle(4), basa_ekle(1), basa_ekle(3) uygular: `basa_ekle(3): [3 -> 1 -> 4 -> NULL]`. Başa ekleme sırayı tersine çevirir: Örnek 3'ün 3. bölümünde on rastgele değer ters sırada dizilir. Örnek 4 3, 1, 4, 1, 5 değerlerini kuyruk işaretçisiyle, yürümeden sona ekler. Araya eklemede (Şekil 8.4) iki bağlantı değişir: önce yeni düğümün `sonraki`'si, sonra öncülünki; sıra ters olursa listenin kalanı kaybolur. Kavram 2 (`kavram/H08_K02_liste_islemleri.c`) aynı `konuma_ekle` fonksiyonuyla başa, ortaya ve sona ekler; her işlemden sonra listeyi, `bas`/`son`'u, yürünen düğüm sayısını (ziyaret) ve yazılan `sonraki` alanı sayısını (bağlantı) basar: `konuma_ekle(2, 9)  [3 -> 1 -> 9 -> 4 -> NULL]  bas = 3, son = 4  (ziyaret 2, bağlantı 2)`.

> TAHMİN ET · **Bu adımdan sonra baş nereyi gösterir?**
>
> [3 -> 1 -> 9 -> 4 -> NULL] listesine `konuma_ekle(0, 7)` uygulanıyor. `Dugum **p` hangi bağlantıyı gösterir, işlemden sonra baş nereyi gösterir ve kaç düğüm alanı yazılır? (Yanıt: döngü hiç dönmez; p, `bas` değişkeninin kendisini gösterir. `d->sonraki = *p` ve `*p = d` ile baş 7 olur: `[7 -> 3 -> 1 -> 9 -> 4 -> NULL]`; yazılan düğüm alanı 1 (yeni düğümünki; `bas` bir düğüm alanı değildir), ziyaret 0.)

Şekil 8.4
Araya ekleme, Kavram 2 konuma_ekle(2, 9): öncüle kadar yürü, önce yeni düğümün sonraki'si, sonra öncülün sonraki'si (sıra önemlidir)



SÖZDE KOD · 4 · Sözde kod: başa ekleme ve işaretçiye işaretçiyle silme

```
algoritma BAŞA-EKLE(bas, x)
    d ← YENİ-DÜĞÜM(x);  d.sonraki ← bas;  bas ← d
algoritma SİL(bas, x)                  ▷ p: düğümü değil, BAĞLANTIYI gösterir
    p ← &bas
    iken *p ≠ NULL ve (*p).veri ≠ x
        p ← &(*p).sonraki
    eğer *p = NULL ise döndür yanlış      ▷ x yok
    s ← *p;  *p ← s.sonraki;  BIRAK(s)  ▷ baş da aynı satır
    döndür doğru
```

5 · Doğruluk. Aramanın değişmezi (Tablo 8.2): bakılan ilk i düğümün hiçbirinin verisi x değildir. Silmenin değişmezi: **p, incelenmemiş ilk düğümü gösteren bağlantının adresidir ve p'den önceki düğümlerin hiçbirinin verisi x değildir.** Başta p = &bas olduğu için değişmez doğrudur; döngü sona erdiğinde ya *p = NULL (x yok) ya da *p x'i taşıyan ilk düğümdür ve `*p = s->sonraki` tek atamayla onu zincirden çıkarır, baş düğüm dahil. Çıkarma serbest bırakmadan önce yapılmalıdır: tersi, serbest bırakılmış düğümün `sonraki`'sini okumaktır (Hata Avcısı 3); serbest bırakmamak sızıntıdır (Hata Avcısı 2).

6 · Analiz. Başa ekleme O(1): bir düğüm alanı yazılır (Örnek 3'ün maliyet tablosunda n eklemede n bağlantı, 0 ziyaret). Kuyruk işaretçisiyle sona ekleme O(1); kuyruksuz n sona ekleme 499 500 = n(n − 1)/2 ziyaret ister (n = 1000; Örnek 4). Arama Θ(n): 0 … 999 karıştırılmış listede `başarılı arama: ortalama ziyaret = 500.50, (n + 1) / 2 = 500.50`;`başarısız arama: ziyaret = 1000 (= n)`. Silme ve araya ekleme Θ(konum) yürüme + O(1) bağlama: “O(1) ekleme” yalnızca yeri bilinen bir düğüm için doğrudur.

Tablo 8.2
Bağlı listede başa ekleme ve arama kimlik kartı

ALGORİTMA KİMLİK KARTI **Bağlı liste: başa ekleme ve arama** *(singly linked list: push front / linear search)*	
Strateji	Düğüm ve bağlantı: başa ekleme bağlantı yeniden kurma; arama baştan doğrusal dolaşma
Girdi önkoşulu	Yok (boş listede baş = NULL)
Zaman (en iyi · ortalama · en kötü)	O(1) / O(1) · O(1) / Θ(n) · O(1) / Θ(n)
Ek bellek	Düğüm başına O(1)
Özellikler	Rastgele erişim yok. Başa ekleme 1 bağlantı yazması; başarılı aramada ortalama 500,50 ziyaret, (n + 1)/2; kuyruk işaretçisi olmadan 1000 sona ekleme 499 500 ziyaret. int için düğüm 16 bayt, malloc ile 24 kullanılabilir bayt
Değişmez / doğruluk	Arama: bakılan ilk i düğümün hiçbirinin verisi x değildir
Kütüphane karşılığı	C: yok (elle) · C++: `std::forward_list::push_front`, `std::find` · Python: yerleşik yok; `collections.deque.appendleft`, `x in d`
Ne zaman kullanılır	Sık başa ekleme/çıkarma, düğüm adresinin sabit kalması gereken yapılar: yığın ve kuyruk, karma tablosunda zincir, bekleme listesi
Ne zaman kullanılmaz	İndisle erişim ya da sık arama gerekiyorsa; küçük öğelerde bellek ve önbellek maliyeti yüksektir

7 · C gerçekleştirimi. Başa ekleme yeni başı döndürür; çağıran `bas = basa_ekle(bas, x)` yazar (Örnek 3; Örnek 4'ün `sona_ekle`'si liste başlığını alır, C++ sürümü Ek A.4'te):

C `c/H08_03_tek_bagli_liste.c · satır 45–51`

```c
45 static Dugum *basa_ekle(Dugum *bas, int x)
46 {
47     Dugum *d = dugum_olustur(x);
48     d->sonraki = bas;
49     baglanti_sayisi++;
50     return d;
51 }
```

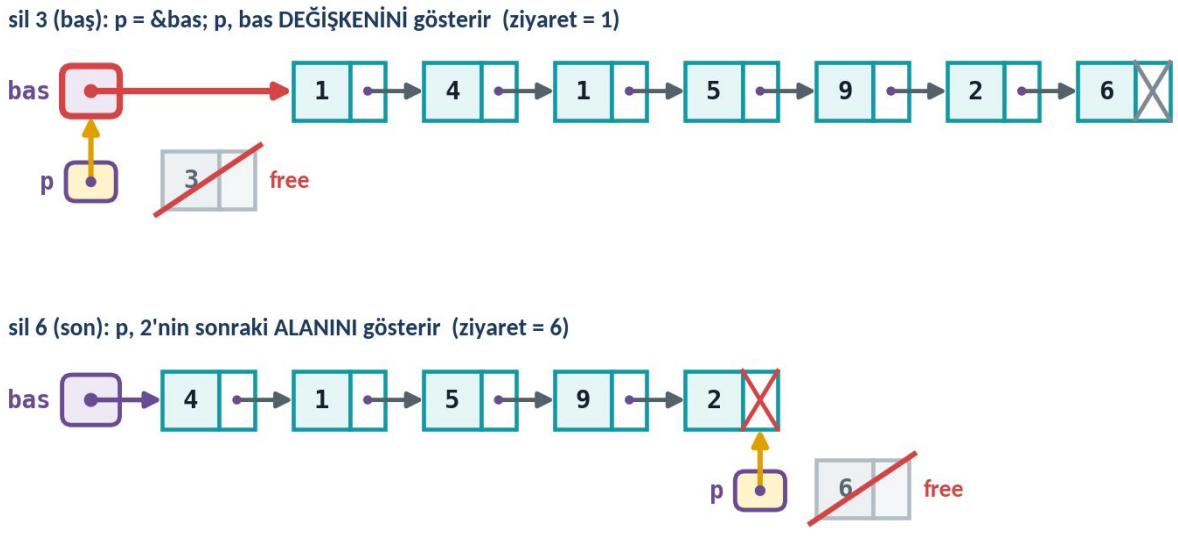
Örnek 5 değere göre silmenin iki sürümünü yan yana yazar. Sürüm 1 `onceki-gecerli` ikilisiyle yürür ve baş siliniyorsa ayrı bir dal gerektirir; sürüm 2'de p bir bağlantının adresidir ve dal yoktur (Şekil 8.5):

C `c/H08_05_degere_gore_silme.c · satır 89–124`

```c
89  static Dugum *sil_v1(Dugum *bas, int x)
90  {
91      Dugum *onceki = NULL;
92      Dugum *gecerli = bas;
93      while (gecerli != NULL && !aranan_mi(gecerli, x)) {
94          onceki = gecerli;
95          gecerli = gecerli->sonraki;
96      }
97      if (gecerli == NULL) {
98          return bas;                          /* x yok: liste aynı */
99      }
```

```
100     if (onceki == NULL) {
101         bas = gecerli->sonraki;              /* ÖZEL DURUM: baş siliniyor */
102     } else {
103         onceki->sonraki = gecerli->sonraki;  /* ortadaki ya da son düğüm */
104     }
105     dugum_birak(gecerli);
106     return bas;
107 }
108
109 /* Sürüm 2 (Dugum **): p, "sıradaki düğümü gösteren bağlantının" adresidir.
110  * Baş işaretçisi de bir bağlantıdır: özel durum kalmaz. */
111 static bool sil_v2(Dugum **bas, int x)
112 {
113     Dugum **p = bas;
114     while (*p != NULL && !aranan_mi(*p, x)) {
115         p = &(*p)->sonraki;
116     }
117     if (*p == NULL) {
118         return false;
119     }
120     Dugum *silinen = *p;
121     *p = silinen->sonraki;  /* baş da aynı satır */
122     dugum_birak(silinen);
123     return true;
124 }
```

Şekil 8.5
*Dugum ** ile silme, sürüm 2: p'nin gösterdiği bağlantı (bas ya da bir düğümün sonraki alanı) aynı satırla değişir; solda baş, sağda son düğüm siliniyor*



iki durumda da TEK satır: *p = silinen->sonraki; (baş için ayrı dal yok)

> GERÇEK DÜNYADAN · **Özel durumu ortadan kaldırmak: Torvalds'ın iki sürümü**
>
> Linus Torvalds 2016'daki TED söyleşisinde tek bağlı listeden bir öğeyi çıkaran iki kod parçası gösterdi. İlkinin

> sonunda bir `if` vardı: çıkarılan ilk öğeyse listenin başını gösteren işaretçi değiştirilmeliydi. İkinci sürümde bu `if` yoktu. Torvalds'ın vardığı ders, bir problemi farklı görüp özel durumun ortadan kalkacağı, olağan duruma dönüşeceği biçimde yeniden yazmanın bazen mümkün olduğuydu (Torvalds, 2016). İkinci sürümün ardındaki fikir sürüm 2'nin `Dugum **p`'sidir: baş işaretçisi de bir bağlantıdır.
>
> Kavram 3 (`kavram/H08_K03_ozel_durumsuz_silme.c`) aynı iki yaklaşımı adresi bilinen bir düğümü çıkarmak için yan yana yazar: `cikar_ozel` `onceki` ile yürür ve `if (onceki == NULL)` dalını taşır, `cikar_pp` dört satırdır. Baş, orta, son ve tek düğüm durumlarında sonuçlar aynıdır (3 durumda "aynı mı? evet"; tek düğümde ikisi de boş liste).

8 · Üç dil karşılaştırması. C++'ta baş başvuruyla (`Dugum*&`) gelir ama yürüyüşte yine `Dugum**` gerekir, çünkü başvuru yeniden bağlanamaz (Ek A.5). Python'da işaretçiye işaretçi yoktur; aynı "dalsız" etkiyi başın önüne konan geçici bir **kukla düğüm** verir:

Python **H08_05_degere_gore_silme.py · satır 95–105**

```
95      kukla = Dugum(0)
96      kukla.sonraki = bas
97      onceki = kukla
98      while onceki.sonraki is not None and not aranan_mi(onceki.sonraki, x):
99          onceki = onceki.sonraki
100     if onceki.sonraki is None:
101         return kukla.sonraki, False
102     silinen = onceki.sonraki
103     onceki.sonraki = silinen.sonraki  # baş da aynı
104     dugum_birak(silinen)
105     return kukla.sonraki, True
```

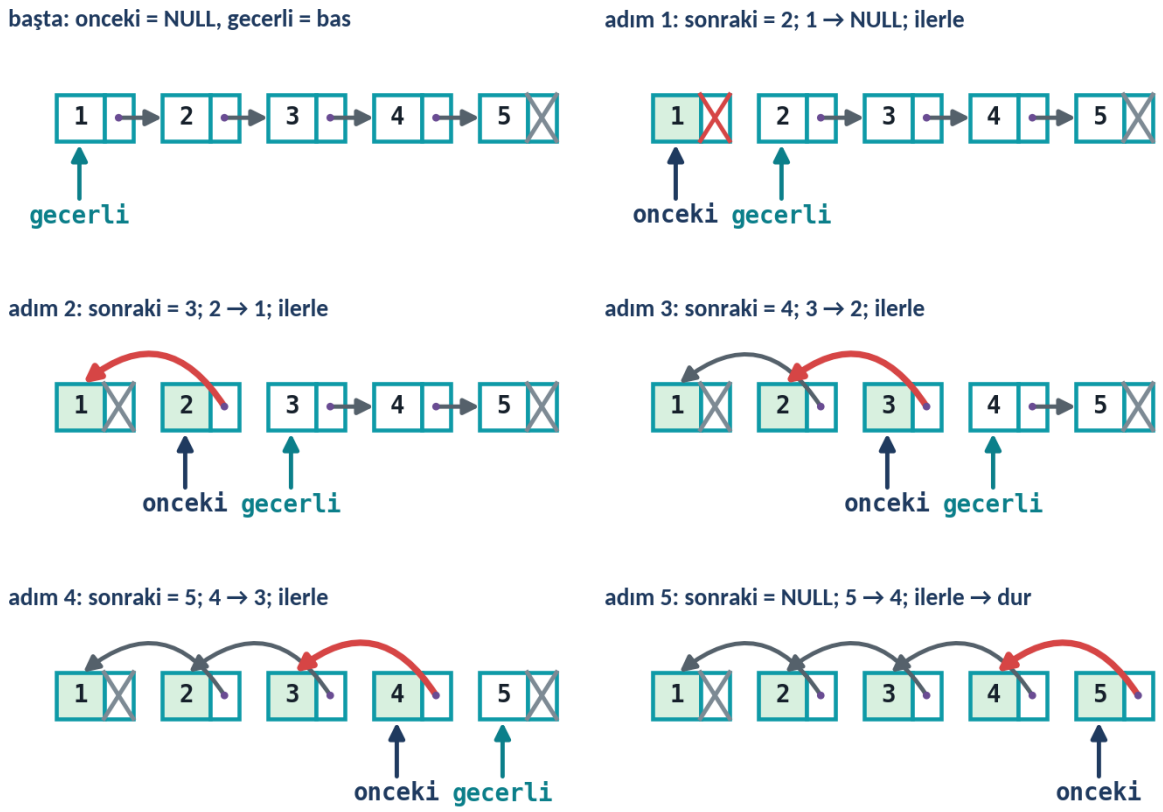
9 · Test. Örnek 5 iki sürümü aynı dört silmede (baş, ilk eşleşme, son, olmayan) ve sınır durumlarında çalıştırır; ziyaret sayıları da aynıdır. Rastgele sınama: `sonuç listeleri aynı: 1000/1000, ziyaret sayıları aynı: 1000/1000`. Kâhin burada "öteki sürüm"dür; iki sürüm aynı hatayı yapabileceği için Örnek 5 ayrıca sonucu elle hesaplanmış listelerle karşılaştırır (1–3. bölümler). **10 · Deney.** Silmenin maliyeti yürümedir; listede yürümenin bedeli Bölüm 6.3'te ölçülür. **11 · Varyantlar.** Bütün eşleşenleri silme (`tumunu_sil`, Alıştırma 8.5), sıralı listeye ekleme (Örnek 13), konumla ekleme ve silme (Kavram 2), baştan çıkarma (H9'da kuyruk). Kuyruk işaretçili listede son düğüm silinince `son` öncüle çekilmelidir: Kavram 2'de `sil(8) [3 -> 1 -> 9 -> 4 -> NULL] bas = 3, son = 4 (ziyaret 5, bağlantı 1)`.

4.3 Listeyi ters çevirme

1 · Problem. Tek bağlı listeyi yeni düğüm ayırmadan, bağlantıları yeniden kurarak ters çevirmek. **2 · Fikir.** Listeyi soldan sağa dolaşırken her düğümün okunu bir öncekine çevir. Ok çevrilince düğümün eski ardılına giden tek yol kaybolacağı için **önce** ardıl kaydedilmelidir. Üç işaretçi yeter: `onceki` (ters çevrilmiş kısmın başı), `gecerli` (işlenecek düğüm), `sonraki` (kaydedilmiş ardıl).

3 · Elle izleme. Örnek 7, [1 2 3 4 5] listesini adım adım çevirir (Şekil 8.6); her adımda bir bağlantı yazılır ve ters önek bir düğüm büyür: `adım 1: sonraki = 2, bağ 1 -> NULL, onceki = 1, gecerli = 2`, … , `adım 5: sonraki = NULL, bağ 5 -> 4, onceki = 5, gecerli = NULL`.

Şekil 8.6
Ters çevirmede onceki/gecerli/sonraki: her karede önce sonraki kaydedilir, gecerli'nin oku çevrilir (kırmızı), sonra iki işaretçi ilerler; yeşil ters önek, gri kalan sonek



yeşil: ters çevrilmiş önek (başı onceki) · beyaz: kalan sonek (başı gecerli) · kırmızı: bu adımda çevrilen bağlantı

> TAHMİN ET · **Sizce sırada ne var?**
>
> 3. adımdan sonra onceki = 3, gecerli = 4'tür. 4. adımda hangi bağlantı yazılır ve "kalan" ne olur? (Yanıt: önce sonraki = 5 kaydedilir, sonra 4'ün oku 3'e çevrilir; ters önek [4 -> 3 -> 2 -> 1 -> NULL], kalan [5 -> NULL].)

SÖZDE KOD · 4 · Sözde kod: yinelemeli ters çevirme

```
algoritma TERS-ÇEVİR(bas)
    onceki ← NULL;  gecerli ← bas
    iken gecerli ≠ NULL
        sonraki ← gecerli.sonraki          ▷ 1) kaydet
        gecerli.sonraki ← onceki           ▷ 2) çevir
        onceki ← gecerli;  gecerli ← sonraki   ▷ 3) ilerle
    döndür onceki                          ▷ yeni baş
```

5 · Doğruluk. Değişmez (Tablo 8.3): onceki ters çevrilmiş önekin, gecerli henüz işlenmemiş sonekin başıdır; iki parça birlikte tüm düğümleri içerir. Başta ters önek boştur (onceki = NULL), kalan bütün listedir. Her tur kalanın ilk düğümünü ters önekin başına taşır; değişmez korunur. Döngü gecerli = NULL olunca biter: kalan boştur, onceki bütün listenin ters çevrilmiş hâlinin başıdır. **6 · Analiz.** Θ(n) zaman, O(1) ek bellek, tam n bağlantı yazma (`bağlantı yazma = 5 (n)`). Özyinelemeli sürüm (önce

kuyruğu çevir, sonra başı onun sonuna tak) `bağlantı yazma = 8 (2(n - 1))`, `en büyük derinlik = 5 (n)` verir: her düzey bir yığın çerçevesidir, O(n) yığın.

Tablo 8.3
Listeyi ters çevirme kimlik kartı

ALGORİTMA KİMLİK KARTI **Listeyi ters çevirme** *(linked list reversal)*	
Strateji	Yerinde bağlantı yeniden kurma (yinelemeli; özyinelemeli varyant)
Girdi önkoşulu	Yok
Zaman (en iyi · ortalama · en kötü)	Θ(n) · Θ(n) · Θ(n)
Ek bellek	O(1)
Özellikler	onceki–gecerli–sonraki üçlüsü: önce sonraki saklanır, sonra bağlantı çevrilir; her düğüme bir bağlantı yazılır. Özyinelemeli sürüm 2(n − 1) yazma ve n derinlik ister
Değişmez / doğruluk	onceki ters çevrilmiş önekin, gecerli henüz işlenmemiş sonekin başıdır; iki parça birlikte tüm düğümleri içerir
Kütüphane karşılığı	C: yok (elle) · C++: `std::forward_list::reverse`, `std::list::reverse` · Python: `list.reverse()` (dinamik dizide takas)
Ne zaman kullanılır	Başa ekleyerek kurulan listeyi giriş sırasına çevirmek; yığın sırasını kuyruk sırasına dönüştürmek; palindrom denetimi
Ne zaman kullanılmaz	Özyinelemeli sürüm büyük n'de: Python'da varsayılan özyineleme sınırı (≈ 1000), C'de bu makinede n = 10^6 yığın taşırdı

7 · C gerçekleştirimi. (Örnek 7; üç satır, üç adım):

`c/H08_07_ters_cevirme.c · satır 91–103`

```c
static Dugum *ters_cevir(Dugum *bas)
{
    Dugum *onceki = NULL;
    Dugum *gecerli = bas;
    while (gecerli != NULL) {
        Dugum *sonraki = gecerli->sonraki; /* kaydet */
        gecerli->sonraki = onceki;         /* çevir */
        baglanti_sayisi++;
        onceki = gecerli;                  /* ilerle */
        gecerli = sonraki;
    }
    return onceki;
}
```

8 · Üç dil karşılaştırması. Yinelemeli sürüm üç dilde satır satır aynıdır (C++: Ek A.7); Python'da da önce `sonraki` saklanmalıdır:

Python `H08_07_ters_cevirme.py · satır 77–85`

```python
    onceki = None
    gecerli = bas
    while gecerli is not None:
        sonraki = gecerli.sonraki   # kaydet
        gecerli.sonraki = onceki    # çevir
        baglanti_sayisi += 1
        onceki = gecerli            # ilerle
        gecerli = sonraki
    return onceki
```

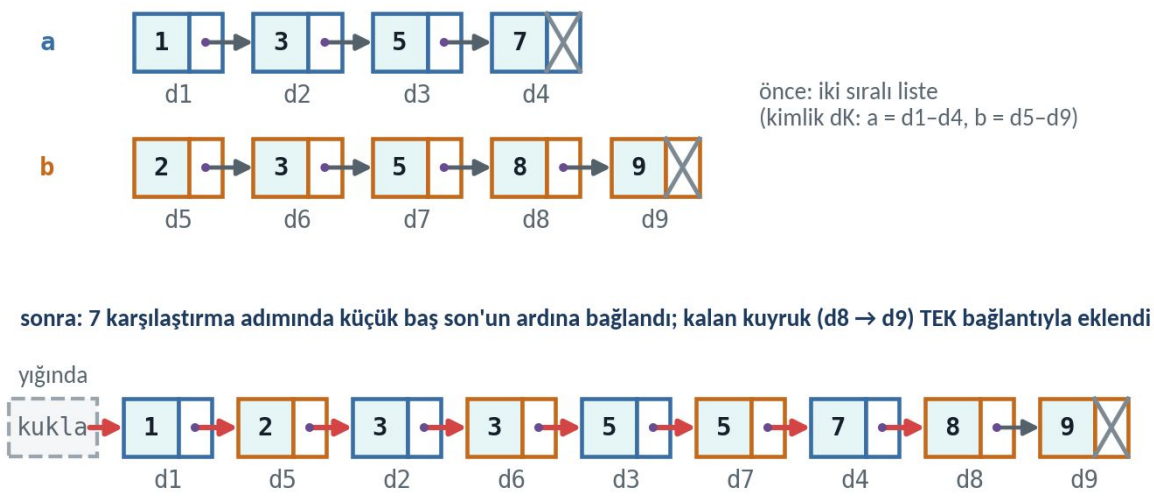
9 · Test. Kâhin listeyi diziye çevirip ters diziyle karşılaştırır; iki kez ters çevirmek asıl listeyi vermelidir: `yinelemeli, n = 1000: tersiyle aynı mı? evet; iki kez ters = asıl mı? evet`. Boş ve tek düğümlü liste ayrıca sınanır (`n = 0: yinelemeli [NULL], özyinelemeli [NULL]`). **10 · Deney.** Özyinelemeli sürümün derinliği n'dir: Python varsayılan özyineleme sınırında (1000 çerçeve) `RecursionError` verir; C ve C++ bu makinede (-O2, 8 MB yığın) n = 5·10⁵'i taşıdı, n = 10⁶'da yığın taştı (`./H08_07_ters_cevirme 1000000`; Örnek 7'nin dil notu). **11 · Varyantlar.** Özyinelemeli sürüm (`ters_cevir_ozyinelemeli`, C 136–149. satırlar; Python'u Ek A.7'de): önce kuyruğu çevirir, sonra `bas->sonraki->sonraki = bas` ile başı onun sonuna takar; k'şarlı gruplar hâlinde ters çevirme; palindrom denetimi (ortadan bölüp ikinci yarıyı çevirmek, Alıştırma 8.12). `sonraki` kaydedilmezse liste ilk düğümden sonra kaybolur ve sızar (Hata Avcısı 6).

4.4 İki sıralı listeyi birleştirme (kukla düğüm)

1 · Problem. Artan sıralı iki listeyi tek artan listede birleştirmek; yeni düğüm ayırmadan ve eşit anahtarlarda girdi sırasını koruyarak (kararlı). **2 · Fikir.** H6'daki dizide birleştirmenin aynısı, ama kopya yerine **yeniden bağlama**: iki listenin başlarından küçüğü sonucun sonuna bağlanır. Sonucun ilk düğümü için özel durum yazmamak üzere sonuç, veri taşımayan bir **kukla düğümle** başlar; `son` işaretçisi her zaman sonucun son düğümünü gösterir. Listelerden biri bitince öteki tek bağlantıyla eklenir.

3 · Elle izleme. Örnek 8'de `a = [1 -> 3 -> 5 -> 7 -> NULL]`, `b = [2 -> 3 -> 5 -> 8 -> 9 -> NULL]`. Her adımda iki başın küçüğü sonuca bağlanır (Şekil 8.7): `adım 1: a.baş = 1, b.baş = 2 -> 1 (a)`, `adım 2: a.baş = 3, b.baş = 2 -> 2 (b)`, … ; 7 seçimden sonra a biter ve `kalan kuyruk (b) tek bağlantıyla eklendi: [8 -> 9 -> NULL]`.

Şekil 8.7
Kukla düğümle birleştirme: gri kesikli kukla, son işaretçisinin ilerleyişi; a'dan gelen düğümler mavi, b'den gelenler turuncu; kalan kuyruk tek bağlantıyla



SÖZDE KOD · 4 · Sözde kod: iki sıralı listeyi birleştirme

```
algoritma BİRLEŞTİR(a, b)                    ▷ a ve b artan sıralı
    kukla.sonraki ← NULL;  son ← &kukla
    iken a ≠ NULL ve b ≠ NULL
        eğer a.veri ≤ b.veri ise son.sonraki ← a;  a ← a.sonraki   ▷ ≤: kararlı
        değilse son.sonraki ← b;  b ← b.sonraki
        son ← son.sonraki
    son.sonraki ← (a ≠ NULL ise a, değilse b)   ▷ kalan kuyruk
    döndür kukla.sonraki
```

5 · Doğruluk. Değişmez (Tablo 8.4): kukla.sonraki'den son'a uzanan zincir, şimdiye dek alınan öğelerin kararlı sıralı birleşimidir; a ve b'de kalanların her biri bu öğelerden büyük ya da eşittir. Seçilen düğüm iki başın küçüğüdür, dolayısıyla kalanların hepsinden küçük ya da eşittir; sonuca eklenince değişmez korunur. **Kararlılık** ≤'den gelir: eşitlikte a'nın düğümü önce alınır (`c = [1#1 -> 2#5 -> 3#2 -> 3#6 -> 5#3 -> 5#7 -> 7#4 -> 8#8 -> 9#9 -> NULL]`). **6 · Analiz.** Her karşılaştırma bir düğümü sonuca bağlar; bir liste bitince durulur: en çok n + m − 1 karşılaştırma, Θ(n + m) zaman, O(1) ek bellek. n = m = 10^5'te `karşılaştırma = 199746 (<= n + m - 1 = 199999), bağlantı yazma = 199747`.

Tablo 8.4
İki sıralı listeyi birleştirme kimlik kartı

ALGORİTMA KİMLİK KARTI **İki sıralı listeyi birleştirme** *(merging two sorted linked lists)*	
Strateji	İki işaretçi + kukla düğüm: küçük baş sonuca bağlanır
Girdi önkoşulu	İki liste de artan sıralı
Zaman (en iyi · ortalama · en kötü)	Θ(n + m) · Θ(n + m) · Θ(n + m)
Ek bellek	O(1)

Özellikler	Düğümleri yeniden bağlama (yeni düğüm 0); kararlı (eşitlikte a'dan gelen önce). n = m = 10^5'te 199 746 karşılaştırma (≤ n + m − 1); 1000 rastgele sınamada kâhinle aynı 1000/1000
Değişmez / doğruluk	kukla.sonraki'den son'a uzanan zincir, şimdiye dek alınan öğelerin kararlı sıralı birleşimidir; a ve b'de kalanların her biri bu öğelerden büyük ya da eşittir
Kütüphane karşılığı	C: yok (elle) · C++: `std::list::merge`, `std::forward_list::merge` (düğümleri taşır, kararlı) · Python: `heapq.merge` (yineleyici), `sorted(a + b)`
Ne zaman kullanılır	Listede birleştirmeli sıralamanın birleştirme adımı; iki sıralı kaydın birleştirilmesi
Ne zaman kullanılmaz	Listeler sıralı değilse; girdi listeleri sonradan da ayrı ayrı gerekiyorsa (düğümler sonuca geçer: önce kopyalayın)

7 · C gerçekleştirimi. (Örnek 8; kukla düğüm yığında yerel bir değişkendir: ayrılmaz, serbest bırakılmaz. Birleştirmeden sonra a ve b'nin düğümleri sonucun olur):

C `c/H08_08_sirali_birlestirme.c · satır 88–107`

```c
static Dugum *birlestir(Dugum *a, Dugum *b)
{
    Dugum kukla = {0, 0, NULL};   /* yığında: ayrılmaz, bırakılmaz */
    Dugum *son = &kukla;          /* sonucun son düğümü */
    while (a != NULL && b != NULL) {
        karsilastirma_sayisi++;
        if (a->veri <= b->veri) {
            son->sonraki = a;
            a = a->sonraki;
        } else {
            son->sonraki = b;
            b = b->sonraki;
        }
        baglanti_sayisi++;
        son = son->sonraki;
    }
    son->sonraki = (a != NULL) ? a : b;   /* kalan kuyruk tek bağlantıyla */
    baglanti_sayisi++;
    return kukla.sonraki;
}
```

8 · Üç dil karşılaştırması. C++ sürümü aynıdır; Python'da kukla geçici bir nesnedir:

Python `H08_08_sirali_birlestirme.py · satır 76–85`

```python
    while a is not None and b is not None:
        karsilastirma_sayisi += 1
        if a.veri <= b.veri:
            son.sonraki = a
            a = a.sonraki
        else:
            son.sonraki = b
            b = b.sonraki
        baglanti_sayisi += 1
        son = son.sonraki
```

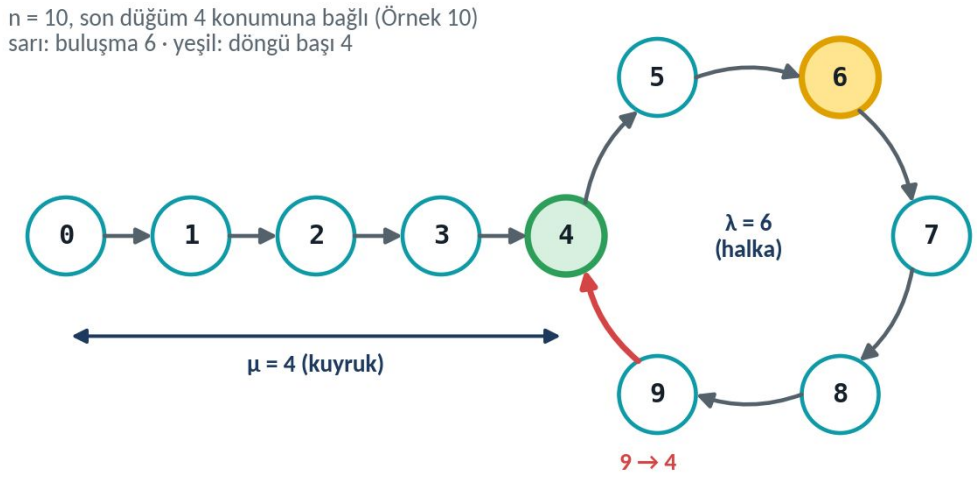
9 · Test. Kâhin iki listenin öğelerini tek diziye koyup kütüphane sıralamasıyla sıralar; kararlılık düğüm kimlikleriyle ayrıca denetlenir: `kâhinle aynı: 1000/1000, kararlı: 1000/1000, birleştirmede yeni düğüm: 0`. Boş listeler sınır durumu olarak sınanır (`[NULL] + [1 -> 2 -> NULL] = [1 -> 2 -> NULL]`). **10 · Deney.** Karşılaştırma sayısı üst sınıra yakındır (199 746 / 199 999), çünkü artışları 0 … 9 olan iki liste sonuna dek iç içe geçer. **11 · Varyantlar.** Listede birleştirmeli sıralamanın birleştirme adımı (Bölüm 4.6); C++ `std::list::merge` (düğümleri taşır, kararlı); Mini-OBS 2 v2.7'nin iki şube listesini birleştirmesi (Bölüm 10); k sıralı listeyi öbekle birleştirme (H7).

4.5 Floyd döngü tespiti (kaplumbağa ve tavşan)

1 · Problem. Bir bağlı listenin döngü içerip içermediğini, içeriyorsa döngünün başladığı düğümü (μ: baştan uzaklık) ve döngü uzunluğunu (λ) **sabit ek bellekle** bulmak. Döngülü bir listeyi dolaşan ya da yok eden olağan kod sonsuza dek döner (Hata Avcısı 8). **2 · Fikir.** İki işaretçi baştan çıkar: kaplumbağa bir, tavşan iki adım atar. Döngü yoksa tavşan NULL'a varır. Döngü varsa ikisi de döngüye girer; içeride tavşan kaplumbağaya her adımda bir düğüm yaklaşır ve onu yakalar (**evre 1**). Sonra kaplumbağa başa döner; ikisi birer adım ilerler ve **döngünün başlangıcında** buluşurlar (**evre 2**). Buluşma düğümünden çıkıp ona dönene kadar saymak λ'yı verir (**evre 3**).

3 · Elle izleme. Örnek 10'da n = 10 düğüm (0 … 9) ve son düğüm 4 konumuna bağlı: μ = 4, λ = 6 (Şekil 8.8). Evre 1'de konum çiftleri (1, 2), (2, 4), (3, 6), (4, 8), (5, 4), (6, 6): 6 adımda 6 düğümünde buluşma; evre 2'de (1, 7), (2, 8), (3, 9), (4, 4): 4 adımda döngü başlangıcı 4.

Şekil 8.8
Floyd döngü tespiti: ρ biçimli liste (kuyruk μ = 4, halka λ = 6); evre 1'de konum çiftleri ve 6 düğümünde buluşma, evre 2'de iki işaretçinin 4 düğümünde buluşması (Örnek 10)



bağıntı: buluşmada K 6 adım attı; 6 mod λ = 0 → buluşmadan μ adım ileri = döngü başı

evre 1 (kaplumbağa 1, tavşan 2):

```
adım 0: K = 0, T = 0
adım 1: K = 1, T = 2
adım 2: K = 2, T = 4
adım 3: K = 3, T = 6
adım 4: K = 4, T = 8
adım 5: K = 5, T = 4
adım 6: K = 6, T = 6
buluşma: 6 (6 adım)
```

evre 2 (K baştan, T buluşmadan; birer):

```
adım 1: K = 1, T = 7
adım 2: K = 2, T = 8
adım 3: K = 3, T = 9
adım 4: K = 4, T = 4
başlangıç: 4 (μ = 4 adım)
```

TAHMİN ET · Floyd nerede buluşur?

7 düğümlü (0 … 6) listede son düğüm 3 konumuna bağlıysa (μ = 3, λ = 4) evre 1'de kaplumbağa ile tavşan hangi düğümde buluşur, evre 2 hangi düğümü bulur? Önce tahmin edin, sonra izleyin. (Yanıt: buluşma, kaplumbağanın attığı adım sayısı i hem i ≥ μ hem de λ'nın katı olduğunda olur: i = 4, düğüm 4; evre 2 döngü başlangıcını, 3 düğümünü bulur. Kavram 4'ün tablosundaki satırla karşılaştırın.)

SÖZDE KOD · 4 · Sözde kod: Floyd döngü tespiti (üç evre)

```
algoritma FLOYD(bas)
    k ← bas;  t ← bas                          ▷ evre 1: buluşma
    iken t ≠ NULL ve t.sonraki ≠ NULL
        k ← k.sonraki;  t ← t.sonraki.sonraki
        eğer k = t ise dur
    eğer t = NULL veya t.sonraki = NULL ise döndür “döngü yok”
    k ← bas                                    ▷ evre 2: μ adım
    iken k ≠ t:  k ← k.sonraki;  t ← t.sonraki
    λ ← 1;  p ← t.sonraki                      ▷ evre 3: uzunluk
    iken p ≠ t:  p ← p.sonraki;  λ ← λ + 1
    döndür (k, λ)                              ▷ k: döngünün ilk düğümü
```

5 · Doğruluk. Evre 1 biter: tavşan döngüye en geç μ adımda girer; kaplumbağa da girdikten sonra aralarındaki uzaklık her adımda 1 azalır, dolayısıyla en çok μ + λ adımda buluşurlar. **Evre 2 doğrudur:** buluşma i. adımda olsun; kaplumbağa i, tavşan 2i adım atmıştır ve ikisi aynı düğümdedir, yani aradaki fark i döngü uzunluğunun katıdır: **i ≡ 0 (mod λ)**. Buluşma düğümü baştan i adım ötededir. Baştan μ adım ilerleyen işaretçi döngü başlangıcına varır; buluşmadan μ adım ilerleyen işaretçi baştan i + μ adım ötededir ve i, λ'nın katı olduğundan o da döngü başlangıcındadır. İkisi aynı anda varır. Kavram 4 (`kavram/H08_K04_floyd_bagintisi.c`) bu bağıntıyı 7 farklı (n, μ) için tablolar: her satırda i mod λ = 0 ve bulunan başlangıç μ'dür (ör. n = 10, μ = 4: λ = 6, i = 6).

6 · Analiz. Evre 1 en çok μ + λ, evre 2 μ, evre 3 λ adım: toplam Θ(n), ek bellek O(1). Ziyaret edilen düğümleri bir kümede tutmak da döngüyü bulur ama O(n) bellek ister. Örnek 10'un 1000 rastgele listesinde `döngülülerde evre 1 adımı / (mu + lambda) = 0.819`.

Tablo 8.5
Floyd döngü tespiti kimlik kartı

ALGORİTMA KİMLİK KARTI **Floyd döngü tespiti** *(Floyd's cycle detection (tortoise and hare))*	
Strateji	İki işaretçi: kaplumbağa bir, tavşan iki adım
Girdi önkoşulu	Her düğümün en çok bir sonraki bağlantısı var
Zaman (en iyi · ortalama · en kötü)	O(1) · Θ(n) · Θ(n)
Ek bellek	O(1)
Özellikler	Kaplumbağa ve tavşan: evre 1 buluşma (≤ μ + λ adım), evre 2 başlangıç (μ adım), evre 3 uzunluk λ; 1000 rastgele listede 1000/1000 doğru. Döngülü liste yok edilmeden önce kırılmalıdır
Değişmez / doğruluk	Evre 2: baştan ve buluşma düğümünden birer adım ilerleyen iki işaretçi, döngü başlangıcına aynı anda (μ adımda) varır, çünkü buluşmadaki adım sayısı λ'nın katıdır
Kütüphane karşılığı	C: yok (elle) · C++: yok · Python: yok

Ne zaman kullanılır	Bir bağlı yapının döngü içerip içermediğini sabit bellekle denetlemek (yok etmeden önce); bir yinelemeli dizinin (x ← f(x)) dönemini bulmak
Ne zaman kullanılmaz	Ziyaret bayrağı ya da karma kümesi kullanılabiliyorsa daha basittir, ama O(n) ek bellek ister

7 · C gerçekleştirimi. (Örnek 10, evre 1; `izle` bayrağı adımları yazdırır, buluşma işaretçilerin aynı düğümü göstermesidir):

C **c/H08_10_floyd_dongu.c · satır 80–96**

```c
static Dugum *bulusma_bul(Dugum *bas, bool izle)
{
    Dugum *kaplumbaga = bas, *tavsan = bas;
    while (tavsan != NULL && tavsan->sonraki != NULL) {
        kaplumbaga = kaplumbaga->sonraki;
        tavsan = tavsan->sonraki->sonraki;
        adim_sayisi++;
        if (izle) {
            printf("  adım %lld: kaplumbağa = %d, tavşan = %d\n",
                   adim_sayisi, kaplumbaga->veri, tavsan->veri);
        }
        if (kaplumbaga == tavsan) {
            return kaplumbaga;
        }
    }
    return NULL;
}
```

8 · Üç dil karşılaştırması. Düğüm kimliği C/C++'ta `==`, Python'da `is` ile karşılaştırılır (`==` değer eşitliği içindir). Evre 2 Python'da (C ve C++: `dongu_baslangici`, Ek A.10):

Python **H08_10_floyd_dongu.py · satır 81–93**

```python
def dongu_baslangici(bas, bulusma, izle):
    """Evre 2: kaplumbağa baştan, tavşan buluşmadan; ikisi de BİRER adım.
    μ adım sonra döngünün başlangıç düğümünde buluşurlar."""
    global adim_sayisi
    kaplumbaga, tavsan = bas, bulusma
    while kaplumbaga is not tavsan:
        kaplumbaga = kaplumbaga.sonraki
        tavsan = tavsan.sonraki
        adim_sayisi += 1
        if izle:
            print(f"  adım {adim_sayisi}: kaplumbağa = {kaplumbaga.veri}, "
                  f"tavşan = {tavsan.veri}")
    return kaplumbaga
```

9 · Test. Seçilmiş durumlar tablosu döngüsüz listeyi, tek düğümlü döngüyü, tam döngüyü ve uzun kuyruğu kapsar (Örnek 10, 2. bölüm); rastgele sınamada kurulan (μ, λ) bilinir: `doğru (mu, lambda): 1000/1000; evre 1 adım sınırı tutan: 1000/1000`. **10 · Deney.** Evre 1'in adım sayısı μ + λ sınırının ortalama %81,9'u kadardır. **11 · Varyantlar.** Döngüyü kırıp güvenle yok etme (`donguyu_kir`, Örnek 10; Hata Avcısı 8); bir dizinin x ← f(x) yinelemesindeki dönemi bulmak (aynı fikir, düğüm yerine

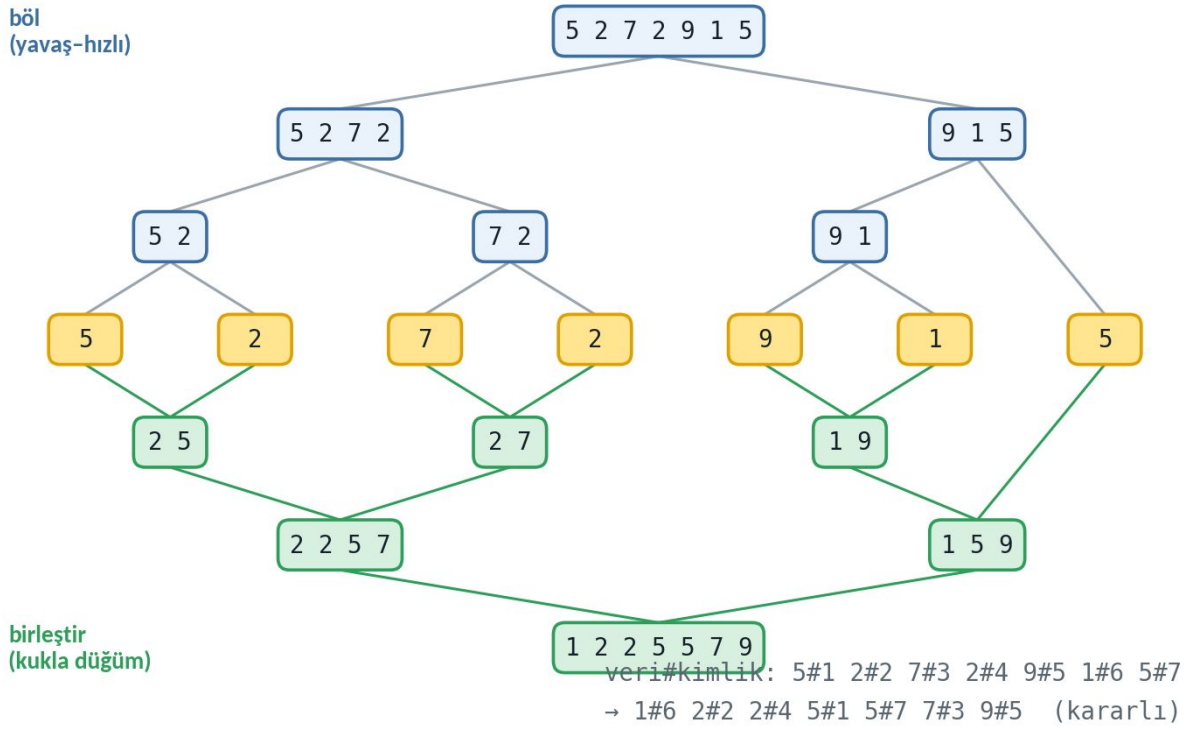
değer); Mini-OBS 2 v2.7 meydan okuması: `bekleme_gecerli_mi`'yi döngülü listede de sonlanır yapmak.

4.6 Listede birleştirmeli sıralama

1 · Problem. Bağlı listeyi öğe taşımadan, ek dizi kullanmadan ve kararlı biçimde sıralamak. **2 · Fikir.** H6'nın birleştirmeli sıralaması listeye doğal olarak uyar: listeyi ortadan ikiye **böl**, iki yarıyı özyinelemeyle sırala, Bölüm 4.4'ün `birlestir`'iyle **birleştir**. Bölme yavaş–hızlı işaretçiyle tek geçişte yapılır (hızlı bir adım önden başlar; ilk yarı ⌈n/2⌉ düğüm). Dizideki sürümün Θ(n) yardımcı dizisi gerekmez: birleştirme bağlantıları yeniden kurar.

3 · Elle izleme. Örnek 12, n = 7 için bölme ve birleştirme ağacını girintiyle yazar (girinti = derinlik; Şekil 8.9): `girdi : [5#1 2#2 7#3 2#4 9#5 1#6 5#7] (veri#kimlik)` → `çıktı : [1#6 2#2 2#4 5#1 5#7 7#3 9#5]`; `kararlı mı (2#2 2#4 ve 5#1 5#7 sırası korundu mu)? evet`.

Şekil 8.9
Listede birleştirmeli sıralamanın bölme (üstte) ve birleştirme (altta) ağacı, n = 7; düğümlerde veri#kimlik: eşit anahtarların sırası korunur (Örnek 12)



SÖZDE KOD · 4 · Sözde kod: bağlı listede birleştirmeli sıralama

```
algoritma LİSTE-BİRLEŞTİRMELİ(bas)
    eğer bas = NULL veya bas.sonraki = NULL ise döndür bas   ▷ 0 ya da 1 düğüm
    y ← bas;  h ← bas.sonraki                ▷ hızlı bir adım önde
    iken h ≠ NULL ve h.sonraki ≠ NULL:  y ← y.sonraki;  h ← h.sonraki.sonraki
    ikinci ← y.sonraki;  y.sonraki ← NULL      ▷ ilk yarıyı kes
    döndür BİRLEŞTİR(LİSTE-BİRLEŞTİRMELİ(bas), LİSTE-BİRLEŞTİRMELİ(ikinci))
```

5 · Doğruluk. Değişmez (Tablo 8.6): Her çağrı, aldığı düğümleri kararlı sıralı bir liste olarak döndürür (birleştirmenin değişmezine dayanır). Taban durumu (0 ya da 1 düğüm) sıralıdır; iki yarı sıralı dönerse birleştirme sıralı ve kararlı bir liste verir. İlk yarının ⌈n/2⌉ düğüm olması n ≥ 2'de iki yarıyı da boş olmayan

yapar; özyineleme sonlanır. **6 · Analiz.** Her düzeyde bölme Θ(n) yürür ve birleştirme Θ(n) karşılaştırma yapar; $\lceil \log_2 n \rceil$ düzey: Θ(n log n). Derinlik $\lceil \log_2 n \rceil + 1$, yani O(log n) yığın; ek dizi yok. n = 10^5'te 1 536 388 karşılaştırma (n $\log_2$ n'nin 0,925 katı), derinlik 18, çağrı 199 999 = 2n − 1.

Tablo 8.6
Listede birleştirmeli sıralama kimlik kartı

ALGORİTMA KİMLİK KARTI	**Listede birleştirmeli sıralama** *(merge sort on linked lists)*
Strateji	Böl ve fethet: yavaş–hızlıyla ortadan böl, sıralı listeleri birleştir (H6'nın listeye uyarlanması)
Girdi önkoşulu	Yok
Zaman (en iyi · ortalama · en kötü)	Θ(n log n) · Θ(n log n) · Θ(n log n)
Ek bellek	O(log n) yığın
Özellikler	Ek dizi gerektirmez (yeni düğüm 0); kararlı. n = 10^5'te 1 536 388 karşılaştırma, k/(n lg n) = 0,925, derinlik 18 = ⌈lg n⌉ + 1
Değişmez / doğruluk	Her çağrı, aldığı düğümleri kararlı sıralı bir liste olarak döndürür (birleştirmenin değişmezine dayanır)
Kütüphane karşılığı	C: yok (elle; `qsort` dizi içindir) · C++: `std::list::sort` (kararlı) · Python: `sorted`, `list.sort` (Timsort; dinamik dizide)
Ne zaman kullanılır	Veri zaten bağlı listedeyse ve kararlı sıralama gerekiyorsa; düğümler büyük ve taşınmamalıysa
Ne zaman kullanılmaz	Veri bir dizide durabiliyorsa: diziyi sıralamak (önbellek dostu) genellikle daha hızlıdır; dizide birleştirmeli sıralama O(n) ek dizi ister (H6)

7 · C gerçekleştirimi. (Örnek 12; `birlestir` Örnek 8'inkiyle aynıdır):

c/H08_12_listede_birlestirmeli.c · satır 113–141

```c
113 static Dugum *ortadan_bol(Dugum *bas)
114 {
115     Dugum *yavas = bas, *hizli = bas->sonraki;
116     while (hizli != NULL && hizli->sonraki != NULL) {
117         yavas = yavas->sonraki;
118         hizli = hizli->sonraki->sonraki;
119     }
120     Dugum *ikinci = yavas->sonraki;
121     yavas->sonraki = NULL;  /* ilk yarıyı kes */
122     baglanti_sayisi++;
123     return ikinci;
124 }
125
126 /* Sıralı listenin başını döndürür; düğümler yeniden bağlanır (ek dizi
127  * yok). Derinlik [log2 n] + 1: yığında O(log n) çerçeve. */
128 static Dugum *birlestirmeli_sirala(Dugum *bas, int derinlik)
129 {
130     cagri_sayisi++;
131     if (derinlik > en_derin) {
132         en_derin = derinlik;
133     }
134     if (bas == NULL || bas->sonraki == NULL) {
```

```
        return bas;                     /* 0 ya da 1 düğüm: sıralı */
    }
    Dugum *ikinci = ortadan_bol(bas);
    Dugum *sol = birlestirmeli_sirala(bas, derinlik + 1);
    Dugum *sag = birlestirmeli_sirala(ikinci, derinlik + 1);
    return birlestir(sol, sag);
}
```

8 · Üç dil karşılaştırması. Algoritma üç dilde aynıdır; derinlik $\lceil \log_2 n \rceil + 1$ olduğundan Python'un özyineleme sınırı sorun olmaz:

Python `H08_12_listede_birlestirmeli.py · satır 115–120`

```
    if bas is None or bas.sonraki is None:
        return bas                      # 0 ya da 1 düğüm: sıralı
    ikinci = ortadan_bol(bas)
    sol = birlestirmeli_sirala(bas, derinlik + 1)
    sag = birlestirmeli_sirala(ikinci, derinlik + 1)
    return birlestir(sol, sag)
```

9 · Test. Kâhin sonucu kütüphane sıralamasıyla, kararlılığı düğüm kimlikleriyle, derinliği sınırla karşılaştırır: `kâhinle aynı: 200/200, kararlı: 200/200, derinlik <= ⌈lg n⌉ + 1: 200/200`; sınır durumları n = 0, 1, 2. **10 · Deney.** k/(n lg n) oranı 0,873, 0,906, 0,925 ($n = 10^3, 10^4, 10^5$): 1'in altında, çünkü birleştirme bir liste bitince durur. **11 · Varyantlar.** Aşağıdan yukarıya (özyinelemesiz) birleştirmeli sıralama: önce 1'erli, sonra 2'şerli, 4'erli parçaları birleştirmek; C++ `std::list::sort`. Veri bir dizide durabiliyorsa diziyi sıralamak genellikle daha hızlıdır (önbellek; Bölüm 6).

5. Çözümlü Örnekler ve Bellek Sahipliği

Örnek merdiveninin amiral olmayan basamakları ve haftanın beşinci öğrenme çıktısı (bellek sahipliği ve araçlar) burada. Bütün örneklerin C, C++ ve Python sürümleri aynı çıktıyı verir; yalnızca Örnek 6'nın 5. bölümü dile özgüdür (Δ). C++ ve Python çekirdek fonksiyonları ve Örnek 6'nın üç dildeki 5. bölümü Ek A'dadır.

5.1 Sahiplik sözleşmesi ve listenin yok edilmesi (Örnek 6)

Sahiplik, bir bellek bloğunu kimin serbest bırakmakla yükümlü olduğudur. Her blok tam olarak **bir** sahibe aittir; aynı bloğu gösteren başka işaretçiler **takma addır**, bloğu bırakmaz ve sahibi bıraktıktan sonra kullanılmaz. Sahiplik C dilinde görünmez; bu yüzden **arayüzde yazılır**. Bu haftanın bütün programları dört kurala uyar: (1) ayıran fonksiyonun yorumu sahipliği söyler (`/* Sahiplik: dönen liste çağıranındır; liste_yok_et ile serbest bırakılır */`, Örnek 6); (2) yok etme yinelemelidir ve `sonraki`'yi `free`'den **önce** saklar; (3) yok eden fonksiyon çağıranın işaretçisini NULL yapar (`liste_yok_et(Dugum **bas)`), öteki takma adları geçersiz saymak çağıranın işidir; (4) **derin** kopyanın ayrı bir sahibi vardır, işaretçiyi kopyalamak (sığ kopya) ise sahip sayısını artırmaz: iki yok etme çift `free` olur.

C `c/H08_06_liste_yok_etme.c · satır 58–70`

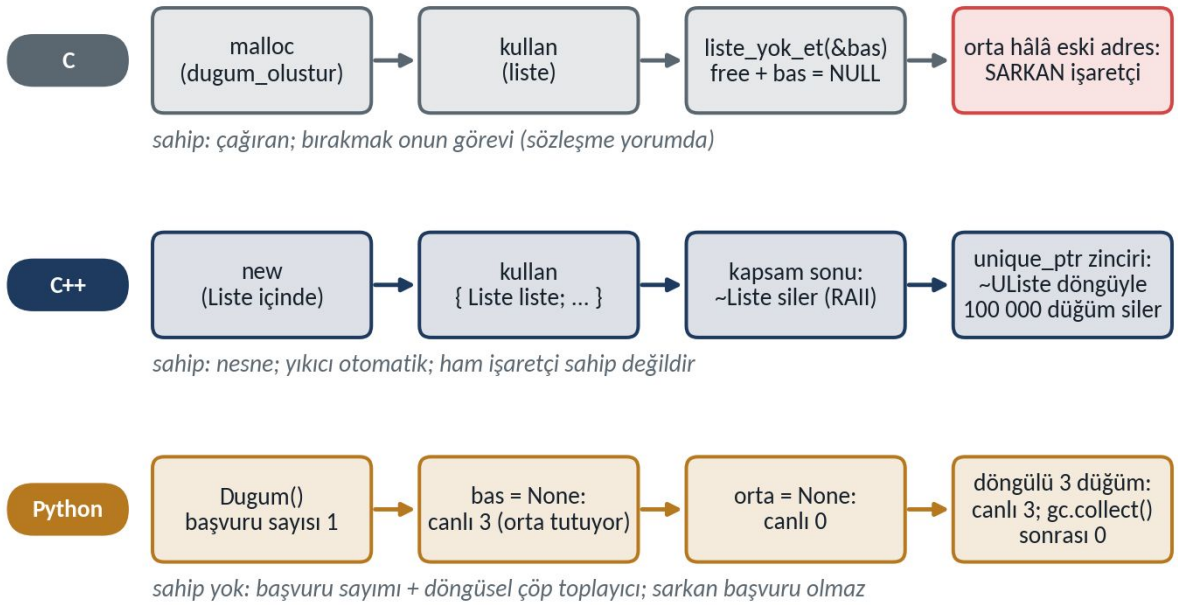
```
/* Yinelemeli yok etme: O(1) ek bellek. sonraki, free'den ÖNCE saklanır
 * (free'den sonra d->sonraki okunamaz). Sonunda çağıranın başı NULL olur:
 * sarkan baş işaretçisi kalmaz. */
static void liste_yok_et(Dugum **bas)
{
```

```
    Dugum *d = *bas;
    while (d != NULL) {
        Dugum *sonraki = d->sonraki;
        dugum_birak(d);
        d = sonraki;
    }
    *bas = NULL;
}
```

Örnek 6'nın 2. bölümü yok etmeyi adım adım yazar: `bırak 3 (sonraki = 1 önceden saklandı)`, … , `bırak 5 (sonraki = NULL)`, `bas == NULL`.

Örnek 6'nın 3. bölümü derin kopyanın iki sahibini, 4. bölümü 10^5 düğümlük listenin yinelemeli yok edilmesini gösterir; özyinelemeli yok etme küçük listede çalışır ama derinliği n + 1'dir (`özyinelemeli (n = 5): en büyük derinlik = 6 (n + 1); n = 100000 için 100001`): büyük listede yığını taşırır. Her bölümden sonra `ayrılan`, `bırakılan` ve `canlı` sayaçları basılır; program `canlı = 0` ile biter. 5. bölüm dile özgüdür (Şekil 8.10): C'de serbest bırakılmış düğümü gösteren `orta` işaretçisi **sarkar** (okumak tanımsız davranıştır, program okumaz); C++'ta `Liste`'nin yıkıcısı kapsam sonunda düğümleri siler (RAII), ham işaretçi yine sarkar; Python'da `free` yoktur: `orta.veri = 4 okunabilir: sarkan başvuru yoktur`, çünkü başvuru nesneyi canlı tutar, döngülü yapıları ise döngüsel çöp toplayıcı geri alır (`gc.collect(): canlı düğüm = 0 (döngüsel çöp toplayıcı)`).

Şekil 8.10
Bellek sahipliği ve yaşam döngüsü üç dilde: C'de free sonrası sarkan işaretçi, C++'ta kapsam sonu yıkıcı ve unique_ptr tek sahip, Python'da başvuru sayacı ve döngüsel çöp toplayıcı (Örnek 6, 5. bölüm)



5.2 Araçlarla doğrulama: ASan, UBSan, LSan ve Valgrind

Bellek hataları çoğu zaman **sessizdir**: sızıntı çıktıyı değiştirmez, serbest bırakılmış belleği okumak bu çalıştırmada doğru değeri bile verebilir. Bu yüzden iki tür araç kullanıyoruz. **Derleyici araçları** programı ek denetim koduyla derler: AddressSanitizer (ASan) sınır dışı erişimi, serbest bırakma sonrası kullanımı ve çift `free`'yi, LeakSanitizer (LSan; ASan'la birlikte açılır) program sonunda erişilemeyen blokları,

UndefinedBehaviorSanitizer (UBSan) işaretli taşma gibi tanımsız davranışları yakalar. **Valgrind Memcheck** ise sıradan bir ikiliyi yorumlayarak çalıştırır; derlemeyi değiştirmez, çok daha yavaştır, başlatılmamış değer kullanımını da bildirir. İkisi aynı ikilide birlikte kullanılmaz:

KOMUTLAR (KODLAR/BELLEK_DENETIMI.TXT İLE AYNI BAYRAKLAR)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -O1 -g \
      -fsanitize=address,undefined -fno-omit-frame-pointer \
      H08_06_liste_yok_etme.c -o liste_asan
$ ./liste_asan
$ gcc -std=c17 -Wall -Wextra -pedantic -O0 -g H08_06_liste_yok_etme.c -o liste_vg
$ valgrind --leak-check=full --show-leak-kinds=all --error-exitcode=3 ./liste_vg
$ python3 esdeger_test.py . --asan          # bütün merdiven, üç dil
```

`kodlar/BELLEK_DENETIMI.txt` bu haftanın bütün C ve C++ programlarının (merdiven, deney, kavram, alıştırma, quiz ve Hata Avcısı'nın düzeltilmiş dosyaları) iki araçla denetim sonucunu, araç sürümlerini ve komutları içerir: merdivenin `esdeger_test.py --asan` sınaması 13/13 örnek geçti; **ASan + UBSan temiz 73/73; Valgrind 'ERROR SUMMARY: 0 errors' ve 'definitely lost: 0 bytes' 73/73**. Kasıtlı hatalı dosyaların gerçek raporları `hata_avcisi/ciktilar/` altındadır. Bir ASan raporu üç soruyu yanıtlar: **ne oldu** (ilk satır: hata türü ve erişimin boyu), **nerede** (yığın izinin ilk kendi kodunuza ait çerçevesi: dosya ve satır), **blok kimindi** ("freed by … here" ve "previously allocated by … here" izleri). Hata Avcısı 3'ün raporu (bu derleyicide, bu çalıştırmada; adresler sadeleştirildi) `ERROR: AddressSanitizer: heap-use-after-free` ile başlar ve `SUMMARY: AddressSanitizer: heap-use-after-free H08_HA3_free_sonrasi.c:34 in liste_yok_et` ile biter.

Okuma (`hata_avcisi/ciktilar/H08_HA3_free_sonrasi_asan.txt`): 16 baytlık bir düğümün 8. baytından (`sonraki` alanı) 8 baytlık okuma, `liste_yok_et`'in 34. satırında; blok aynı fonksiyonun 35. satırında serbest bırakılmış, `dugum_olustur`'da ayrılmış. Sızıntı raporu ise programın **sonunda** gelir ve bloğun nerede ayrıldığını söyler, nerede kaybedildiğini değil: Hata Avcısı 2'de LSan `SUMMARY: AddressSanitizer: 16 byte(s) leaked in 1 allocation(s).`, Valgrind `definitely lost: 16 bytes in 1 blocks` yazar; kaybedildiği yer (`sil`), ayrıldığı yerden (`dugum_olustur`, `main`) geriye doğru düşünülerek bulunur.

5.3 Ortadaki düğüm ve sondan k'ıncı (Örnek 9)

Yavaş–hızlı işaretçi: hızlı iki, yavaş bir adım atar; hızlı sona varınca yavaş ortadadır. Sözleşme önemlidir: `ortadaki` 0 tabanlı $\lfloor n/2 \rfloor$ indisli düğümü (çift n'de **ikinci** orta), `ilk_orta` $\lfloor (n-1)/2 \rfloor$'yi (çift n'de birinci orta; birleştirmeli sıralamanın bölmesi bunu kullanır) döndürür. **Sondan k'ıncı:** ileri işaretçi k adım önden gider, sonra ikisi birlikte ilerler; ileri NULL olunca geri işaretçi hedeftedir (k = 1 son düğüm; k < 1 ya da k > n ise NULL).

c/H08_09_iki_isaretci.c · satır 67–76

```c
67  static Dugum *ortadaki(Dugum *bas)
68  {
69      Dugum *yavas = bas, *hizli = bas;
70      while (hizli != NULL && hizli->sonraki != NULL) {
71          yavas = yavas->sonraki;
72          hizli = hizli->sonraki->sonraki;
73          ilerletme_sayisi += 3;
74      }
75      return yavas;
76  }
```

Kâhin, uzunluğu bulup indisle erişen iki geçişli çözümdür: `sınama = 11792, uyuşmazlık = 0`. İki işaretçi işi **azaltmaz**: n = 10^5'te `ortadaki : yavaş–hızlı = 150000, iki geçiş = 150000; aynı düğüm? evet`. Kazanç, uzunluğu ayrıca hesaplamadan tek döngüde sonuca varmaktır.

5.4 Çift bağlı dairesel liste ve O(1) silme (Örnek 11)

Tek bağlı listede bir düğümü silmek için **öncülünü** bilmek gerekir; öncül yalnızca baştan yürüyerek bulunur. Çift bağlı listede her düğüm `onceki` bağlantısını da taşır. Örnek 11 bunu **kukla düğümlü dairesel** biçimde gerçekler: kukla düğüm listenin yapısının içindedir; boş listede kendini gösterir, ilk düğümün öncülü ve son düğümün ardılı kukladır. Böylece hiçbir düğümün komşusu NULL olmaz ve ekleme de silme de özel durum içermez: ekleme (`once_ekle`) dört, silme iki bağlantı yazar.

c/H08_11_cift_bagli_liste.c · satır 92–99

```c
static void dugumu_sil(CiftListe *liste, CDugum *d)
{
    d->onceki->sonraki = d->sonraki;
    d->sonraki->onceki = d->onceki;
    baglanti_sayisi += 2;
    liste->uzunluk--;
    bellek_birak(d);
}
```

Her işlemden sonra ileri ve geri dolaşma ve `d->sonraki->onceki == d` tutarlılığı denetlenir (`uzunluk = 5, bağlantı yazma = 20, tutarlı mı? evet`). Bin düğüm rastgele sırayla adresleriyle silindiğinde: `çift bağlı: ziyaret = 0, bağlantı yazma = 2000 (2n), tutarlı mı? evet`; `tek bağlı : ziyaret = 255893 (n²/4 ≈ 250000), bağlantı yazma = 1000`. Bedel düğüm başına ikinci işaretçidir: bu makinede `sizeof` 16 yerine 24 bayt (glibc'de ikisi de 32 baytlık bloğa düşer). C++ `std::list` çift bağlıdır, `std::forward_list` tek bağlı.

6. Haftanın Deneyi: Dizi mi Liste mi, Gerçekten?

Soru. Stroustrup deneyi: sıralı dinamik diziye ve bağlı listeye rastgele konumlarda ekleme ve silme, n = 10^3…10^5; ayrıca büyüme stratejisine göre toplam kopyalama sayıları ve sona ekleme süreleri. Programlar `kodlar/deney/` altındadır: `H08_deney_stroustrup` (C, C++, Python), `H08_deney_buyume` (C, C++, Python), `H08_deney_dolasma` (C, Python) ve bellek gözlemleri; sürücü `h08_olcum_calistir.py`, çözümleme `h08_cozumle.py`.

6.1 Ölçüm düzeni

Ortam (`deney/OLCUM_ORTAMI.txt`): Intel(R) Xeon(R) Processor @ 2.80GHz, 2 çekirdek (paylaşımlı sanal makine); önbellek L1d 64 KiB; L1i 64 KiB; L2 2 MiB; L3 33 MiB; gcc 13.3.0, Python 3.11.15; -O2; 1 ısınma + 5 tekrar, ortanca (uzun ölçümlerde 3 tekrar ya da † tek çalıştırma).

- **İş yükü (Bentley'nin alıştırması; Stroustrup, 2014):** n rastgele tamsayı (ortak LCG, tohum 42) sıralı yapıya **doğrusal aramayla** bulunan yerine eklenir; sonra n kez rastgele bir konumdan silinir. Silinen değerlerin toplamı bütün serilerde aynı olmalıdır (kâhin).
- **Seriler.** C: `dizi` (× 2 `realloc`, kaydırma döngüsü), `liste` (düğüm başına `malloc`/`free`), `liste_bitisik` (bütün düğümler tek blokta ve **son** sıralı düzende: ideal, gerçek bir ayırıcıyla elde edilemeyen yerleşim), `liste_karisik` (aynı blok, adresler karıştırılmış). C++: elle dizi ve liste, etiketli `std::vector` ve `std::list` (konum yine elle aranır). Python: elle dizi (`[None] * k`), elle liste, etiketli `list` (`insert`/`pop`).

- **Süre ölçütü işlemci zamanıdır.** Makine paylaşımlıydı; C ölçümlerinde duvar saati işlemci zamanının ortanca 1,29 katıydı (en çok 2,11). İşaretler: **†** ısınmasız tek çalıştırma (çok uzun ölçüm), **(k)** ölçülmedi, n^2 modeliyle **kestirim** (kestirimler grafiklerde içi boş işaretlidir).

6.2 Sonuç: C ve C++'ta ölçülen her n'de dizi daha hızlı

Sayaçlar (Örnek 13). Örnek 13, bu iş yükünü küçük n'de (250, 500, 1000) çalıştırır ve yalnızca makineden bağımsız **sayaçları** yazar. Dizide ekleme yeri doğrusal aramayla bulunur, sonra sağdaki öğeler kaydırılır; listede yer yürüyerek bulunur, kaydırma yoktur. İki yapının konum bulma işi **aynıdır**: n = 1000'de dizinin arama karşılaştırması ve listenin ekleme ziyareti 250 575. Fark, dizinin ayrıca kaydırma yapmasıdır (ekleme 249 916, silme 244 905): iki yapıda da adım sayısı $\Theta(n^2)$ ve dizi daha **çok** adım atar.

Sayaçlara bakan biri listeyi seçerdi. Oysa dizinin adımı ardışık bellekte bir öğe ilerlemek, listenin adımı dağınık bir düğümün adresini izlemektir. Süreler `--olc` kipindedir. `dizi adımı (karş. + kaydırma) oranı: 3.94, 3.99`, `liste ziyareti oranı : 3.96, 3.97`: n iki katına çıkınca adım ≈ 4 katı.

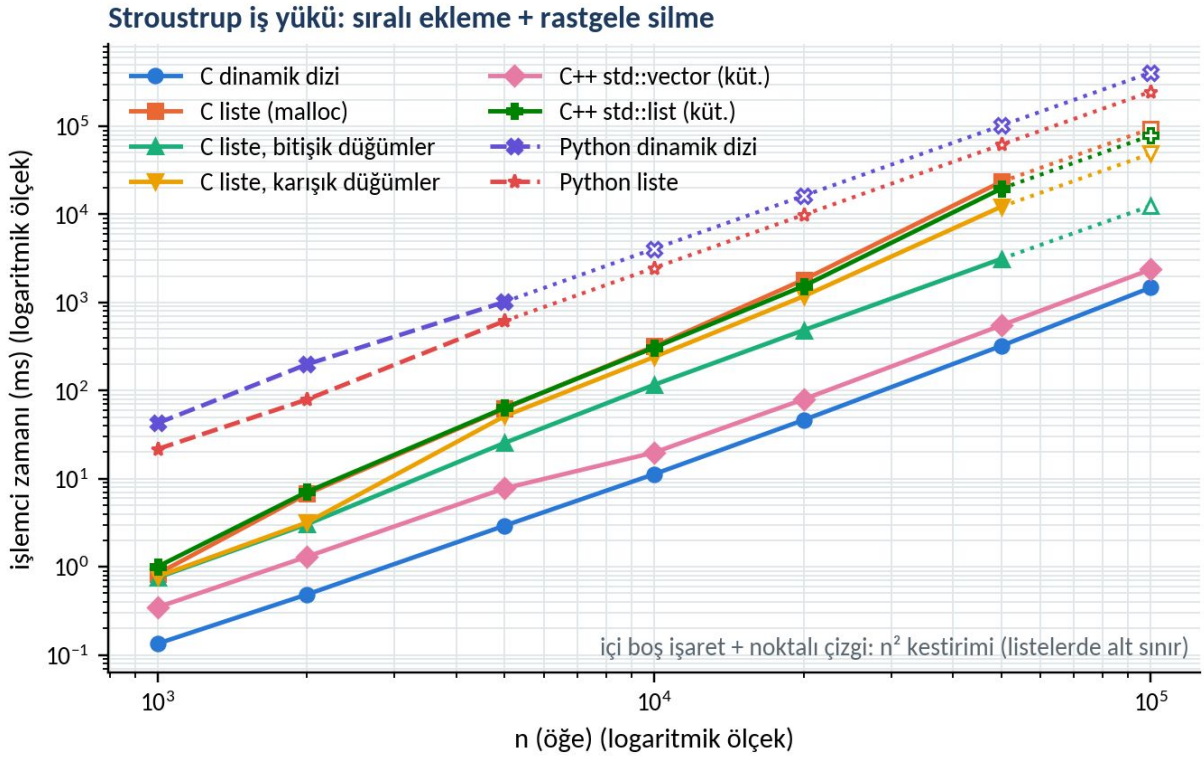
Tablo 8.7

Stroustrup iş yükü: işlemci zamanı ortancası, bu makinede (C ve C++, -O2; † tek çalıştırma; ≈ … (k): n^2 kestirimi, liste kestirimleri alt sınırdır)

n	C dizi	C liste	liste / dizi	liste bitişik	liste karışık	std::vector	std::list	list / vector
10^3	0,134 ms	0,834 ms	6,2×	0,754 ms	0,777 ms	0,348 ms	1,01 ms	2,9×
$2{\cdot}10^3$	0,486 ms	6,75 ms	13,9×	3,04 ms	3,19 ms	1,31 ms	7,10 ms	5,4×
$5{\cdot}10^3$	2,93 ms	62,5 ms	21,3×	25,5 ms	51,4 ms	7,79 ms	63,6 ms	8,2×
10^4	11,3 ms	318,5 ms	28,3×	116,4 ms	240,7 ms	19,8 ms	308,4 ms	15,5×
$2{\cdot}10^4$	46,4 ms	1,81 s	39,1×	485,2 ms	1,18 s	79,7 ms	1,54 s	19,3×
$5{\cdot}10^4$	321,7 ms	23,45 s †	72,9×	3,13 s †	12,22 s †	549,6 ms	19,61 s †	35,7×
10^5	1,48 s	≈ 1,6 dk (k)	≥ 63,6× (k)	≈ 12,51 s (k)	≈ 48,88 s (k)	2,37 s	≈ 1,3 dk (k)	≥ 33,1× (k)

C'de liste/dizi oranı n ile büyür: 6,2×, 13,9×, 21,3×, 28,3×, 39,1×, 72,9× ($n = 10^3$ … $5{\cdot}10^4$). C++'ta `std::list`/`std::vector` 2,9× ile 35,7× arasındadır. **Yerleşimin etkisi:** aynı liste kodu $5{\cdot}10^4$'te düğümler bitişik ve sıralıyken 3,13 s, karışıkken 12,22 s (3,9×), düğüm başına `malloc` ile 23,45 s (7,5×) sürdü; dizi 321,7 ms. İdeal yerleşimli liste bile diziden 5,6–10,5 kat yavaştır (Şekil 8.11).

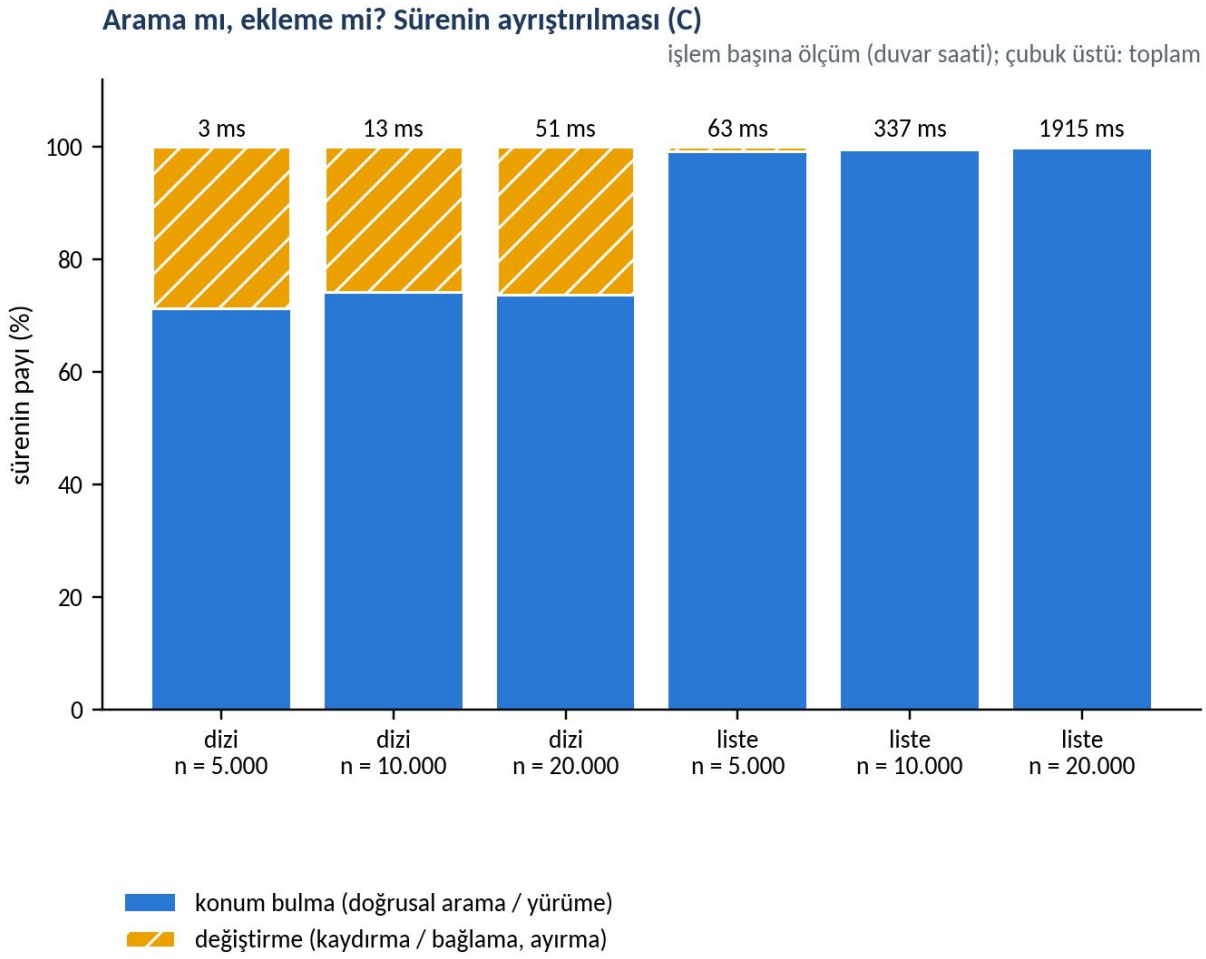
Şekil 8.11
Stroustrup iş yükü, C ve C++ (-O2) ile Python: işlemci zamanı, log–log; içi boş işaretler ve noktalı çizgiler kestirim (bu makinede)



6.3 Neden? Konum bulma ve önbellek

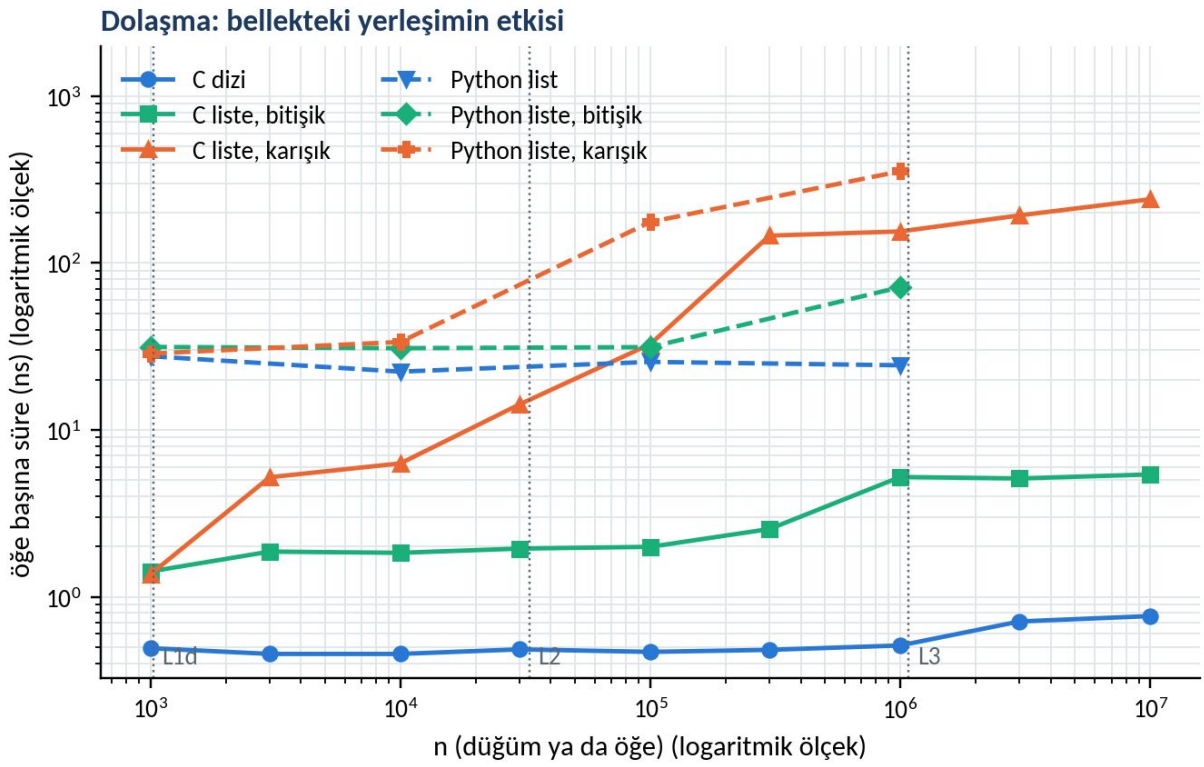
İş yükü iki işten oluşur: **konum bulma** (doğrusal arama) ve **değiştirme** (dizide kaydırma, listede bağlama + `malloc`/`free`). C programı her işlemi zaman damgasıyla ikiye ayırdı (Şekil 8.12): dizide konum bulmanın payı %71,2–74,2, listede %99,1–99,7. Kitapların "O(1) ekleme"si listenin süresinin %1'inden azdır; geri kalan her şey yürümektir. Üstelik listenin yürümesi dizinin aramasından aynı n'de 29,7–51,9 kat uzun sürdü: iki yapı **aynı sayıda** adım atar (Örnek 13), ama listenin her adımı bir önbellek kaçırması olabilir. **Doğrusal arama maliyeti ekleme maliyetini gölgeler.**

Şekil 8.12
Konum bulma ve değiştirme süreleri (C, işlem başına zaman damgası, duvar saati, 3 tekrar): listede sürenin %99'undan fazlası konum bulmadır (bu makinede)



Dolaşma deneyi önbellek etkisini tek başına gösterir: n düğüm bir kez art arda ayrılır, sonra ya ayrılma sırasıyla (bitişik) ya da karıştırılmış sırayla (karışık) bağlanır ve 10^7 ziyaretle dolaşılır (Şekil 8.13). Düğüm belleği L2'ye sığarken ($n = 10^4$: 312,5 KiB) karışık/bitişik 3,4×; $n = 3{\cdot}10^5$'te (9,2 MiB) 57,3×: düğüm başına 146 ns, dizide öğe başına 0,48 ns. L3'ü aşan 10^7 düğümde karışık düğüm başına 241 ns.

Şekil 8.13
Dolaşma: öğe/düğüm başına süre (C düz, Python kesikli çizgi); noktalı dikey çizgiler C düğümlerinin L1d, L2 ve L3 önbellek boyutunu doldurduğu n (bu makinede)



Kuramla karşılaştırma. Log–log eğimi büyüme sınıfını verir (Sedgewick & Wayne, 2011): C dizi 2,05 ($\Theta(n^2)$ iş yükü), liste 2,42, bitişik liste 2,21, karışık liste 2,55. Listeler n^2'den **hızlı** büyür, çünkü çalışma kümesi önbellekten taştıkça adım başına süre artar. Bu yüzden $2{\cdot}10^4$'ten n^2 ile yapılan kestirim $5{\cdot}10^4$'te dağınık listelerde düşük kaldı (liste %51,6, karışık %39,8, `std::list` %50,9 düşük), bitişik listede yalnızca %3,1. Tablodaki (Tablo 8.7) 10^5 liste değerleri bu yüzden **alt sınırdır**.

6.4 Büyüme stratejisi: kopya sayısı ve süre

Kopya sayıları kuramla birebir aynıdır (ilk kapasite 1): 10^6 eklemede +1 ile 499 999 500 000, +10 ile 49 999 600 000, × 1,5 ile 2 099 719, × 2 ile 1 048 575 kopya. Ekleme başına süreler iki şey söyler. (1) **Elle kopyalamada kuram tam görünür:** C'de +1 ile 10^5 ekleme 7,36 s (†), × 2 ile 10^6 ekleme ekleme başına 7,0 ns. (2) **realloc modelden sapar:** +1 ile bile 10^6 ekleme ekleme başına 16,4 ns sürdü ve ölçülen tekrarlarda blok 0 kez taşındı: glibc bloğu yerinde büyüttü, büyük bloklarda sayfaları kopyalamadan yeniden eşledi (sayaç ısınmadan sonraki tekrarı sayar; taşıma sayısı ayırıcının o anki durumuna bağlıdır). Araya küçük bir blok sokulunca (+1) ekleme başına 81,3 ns, 2 taşıma. Bu bir ayırıcı özelliğidir; parçalanmış bir öbekte ya da başka bir ayırıcıda geçerli olmayabilir. Kütüphaneler: C++ `std::vector::push_back` 7,9 ns (21 kapasite değişimi; × 2), Python `list.append` 55,8 ns (86 yeniden boyutlandırma), Python elle × 2 dizi 130,4 ns.

6.5 Python: ters yönlü bulgu

Python'da (3.11) elle yazılmış liste, elle yazılmış dinamik diziden **hızlıydı**: liste/dizi 0,51, 0,40, 0,61 ($n = 10^3$, $2{\cdot}10^3$, $5{\cdot}10^3$). Yorumlayıcıda her adımın sabit yükü (bayt kodu çözme, nesne erişimi, başvuru sayımı) önbellek farkını bastırır ve elle dizinin kaydırma döngüsü de bir Python döngüsüdür. C'de yazılmış `list.insert`/`pop` ise kaydırmayı tek bir bellek kopyasıyla yapar ve elle dizinin 0,25, 0,24, 0,33 katı sürede biter. Önbellek etkisi Python'da da vardır, yalnızca daha küçük görünür: dolaşmada

n = 10^5'te karışık/bitişik 5,6×. Ders: "hangi yapı?" sorusunun yanıtı dile ve gerçekleştirime bağlıdır; ölçmeden genelleme yapılmaz.

6.6 Yorum, sınırlılıklar ve Büyük Soru'nun yanıtı

- **Büyük Soru'nun yanıtı: hayır.** Ortaya eklemenin "O(1)" olması yalnızca yeri bilinen düğüm içindir. Yeri aramak gerekiyorsa iki yapı da Θ(n) adım atar ve dizinin adımı çok daha ucuzdur; bu makinede C ve C++'ta ölçülen her n'de dizi kazandı (Python'un ters yönlü sonucu için Bölüm 6.5). Stroustrup'un üç dersi bu ölçümün özetidir: gereksiz veri saklamayın (düğüm başına işaretçi), veriyi sıkışık tutun, belleğe öngörülebilir biçimde erişin (Stroustrup, 2014).
- **Liste ne zaman kazanır?** Yer zaten biliniyorsa (adresi tutulan düğüm, baştan çıkarma, iki listenin birleştirilmesi), öğeler büyükse ya da taşınmamaları gerekiyorsa. Mini-OBS 2 v2.7'nin bekleme listesi (baştan çıkarma) bunun örneğidir (Bölüm 10).
- **Kütüphane notu:** C++'ta `std::vector` bu makinede elle yazılmış diziden 1,55–3,01 kat yavaş ölçüldü (aynı arama döngüsü; fark büyük olasılıkla `insert`/`erase` yolundadır, ancak bu deneyde ayrıştırılmadı); "kütüphane her zaman hızlıdır" ya da tersi söylenemez.
- **Sınırlılıklar:** paylaşımlı, 2 çekirdekli sanal makine; tek tohum (42) ve tek öğe türü (int); listeler 10^5'te ölçülmedi (kestirim, alt sınır); Python Stroustrup ölçümü n ≤ 5·10^3; `liste_bitisik` gerçek bir ayırıcının veremeyeceği ideal yerleşimdir. Mutlak süreler başka makinede farklı olur; oranlar, eğimler ve sayaçlar yorumlanmalıdır.

Kendi makinenizde. `cd kodlar/deney && python3 h08_olcum_calistir.py && python3 h08_cozumle.py` bütün ızgarayı ölçer ve özet CSV'leri üretir (bu makinede toplam 4,7 dk; `--hizli` küçük bir alt küme çalıştırır). Süreleriniz farklı olacaktır; büyüme sınıfları, sayaçlar ve karşılaştırmaların yönü aynı kalmalıdır.

7. Hata Avcısı

Her satırın hatalı ve düzeltilmiş programı `kodlar/hata_avcisi/` altındadır (kasıtlı dosyaların başlığında "BU DOSYA KASITLI HATA İÇERİR" yazar). Belirtiler bu makinede gözlenen çıktılardır (`hata_avcisi/ciktilar/`; olağan çalıştırma, ASan ve Valgrind raporları). 1, 3 ve 8 tanımsız davranıştır: belirti yalnızca **bu derleyicide ve bu çalıştırmada** gözlenendir. 2, 4, 5 ve 6 olağan çalıştırmada **sessizdir**; hatayı yalnızca araçlar ya da sayaçlar gösterir.

Tablo 8.8
Hata Avcısı: hata, belirti, neden ve çözüm

#	Hata	Belirti (bu makinede)	Neden	Çözüm
1	Boş ve tek elemanlı liste sınır durumunu atlamak (`p->sonraki->sonraki`)	C: iki silmeden sonra SIGSEGV; ASan: `SEGV H08_HA1_sinir_durumu.c:48 in son_dugumu_sil`. Python: `AttributeError`	Döngü en az iki düğüm varsayar; tek düğümde `p->sonraki` NULL'dır	Önce boş ve tek düğüm durumu; her işlevi n = 0, 1, 2 ile sına
2	Silinen düğümü `free` etmeden bağlantıyı koparmak	Çıktı doğru görünür, sayaç ele verir: `ayrılan = 5, bırakılan = 4 -> 1 düğüm sızdı`; LSan: 16 bayt, 1 blok	Düğüme giden tek işaretçi bağlantıydı; blok erişilemez kalır	Sakla, bağlantıdan çıkar, sonra `free`
3	`free` sonrası `sonraki`'ye erişmek (`for (…; p = p->sonraki) free(p);`)	gcc 13 uyarır (`-Wuse-after-free`); çalıştırma: SIGSEGV; ASan: heap-use-after-free (Bölüm 5.2)	Artırım `free(p)`'den sonra çalışır: serbest bırakılmış blok okunur	`sonraki = p->sonraki; free(p); p = sonraki;`
4	Baş işaretçisini değerle aktarıp değişmesini beklemek (AP-1 H8 takasının yeni yüzü)	C ve Python: `3 kez basa_ekle sonrası liste: [NULL]`; C'de LSan: `48 byte(s) leaked in 3 allocation(s).`	`bas = d` fonksiyonun yerel kopyasını değiştirir (AP-1 H8'deki takasın aynısı)	`Dugum **bas` ya da yeni başı döndür: `bas = basa_ekle(bas, x)`

#	Hata	Belirti (bu makinede)	Neden	Çözüm
5	`p = realloc(p, …)`	Başarısızlık benzetimi: `ekle 9: bellek yetersiz (benzetim); d.veri = NULL, boyut = 8`; LSan: `32 byte(s) leaked in 1 allocation(s).`	Başarısız `realloc` NULL döndürür, eski bloğa dokunmaz; tek adres ezilir	Sonucu geçici işaretçiye al (Örnek 1)
6	Ters çevirmede `sonraki`'yi kaydetmeden bağlantıyı değiştirmek	`ters : [1 -> NULL]`; LSan: `64 byte(s) leaked in 4 allocation(s).`	`gecerli->sonraki = onceki` kalana giden tek bağlantıyı ezer	Kaydet, çevir, ilerle (Örnek 7)
7	Python `list`'ini bağlı liste sanmak (`insert(0, x)`)	n = 10 000: 21,9 ms; 20 000: 73,1 ms; 40 000: 287,4 ms; n iki katına çıkınca süre ≈ 3,3 ve 3,9 katı (bu çalıştırmada)	`list` dinamik dizidir: her `insert(0, x)` bütün öğeleri kaydırır	`collections.deque.appendleft`, kendi listen ya da `append` + bir kez `reverse`
8	Döngülü listeyi döngüyü kırmadan yok etmek	C: `free(): double free detected in tcache 2` ve SIGABRT; ASan: heap-use-after-free	Son düğümden sonra döngünün başına (serbest bırakılmış düğüme) dönülür	Floyd ile başlangıcı bul, döngüyü kır, sonra yok et (Örnek 10)
9	C++ `std::unique_ptr` zincirini varsayılan yıkıcıyla yok etmek	n = 10^6'da SIGSEGV, n = $3{\cdot}10^5$'te `liste yok edildi`; ASan: stack-overflow	Her düğümün yıkıcısı sonrakininkini çağırır: n derinlikte özyineleme	Yıkıcıda döngü: `while (bas) bas = std::move(bas->sonraki);`

8. Dil Penceresi: C · C++ · Python

Tablo 8.9
Dil karşılıkları: bellek yönetimi, bağlantılar ve kütüphane yapıları (bellek değerleri bu makinede)

Öğe	C	C++	Python
Bellek yönetimi	`malloc`/`free` elle; sahiplik yorumda yazılır	RAII (yıkıcı), `std::unique_ptr` (tek sahip); `new`/`delete`	başvuru sayımı + döngüsel çöp toplayıcı (`gc`); `free` yok
Boş bağlantı	`NULL`	`nullptr`	`None` (`is None` ile)
Düğüm kimliği	`p == q` (adres)	`p == q` (adres)	`p is q` (`==` değer eşitliği)
Başı değiştirmek	`Dugum **bas` ya da yeni başı döndür	`Dugum*&` gelir; yürümede yine `Dugum**`	yeni başı döndür ya da kukla düğüm
Dinamik dizi	elle, `realloc`	`std::vector` (`push_back`, `reserve`, `capacity`)	`list` (başvuru dizisi)
Bağlı liste	elle	`std::list` (çift), `std::forward_list` (tek)	yerleşik yok; `collections.deque` (bloklu iki uçlu)
Büyüme politikası	seçim sizde (Örnek 2)	gerçekleştirime bağlı; libstdc++'ta × 2 (bu makinede 21 değişim, 10^6)	fazladan ayırma: (n + n/8 + 6) & ~3 (CPython)
Düğüm belleği	`sizeof(Dugum)` = 16 (öbekte 32)	`std::list<int>` 24, `std::forward_list<int>` 16 bayt istek	`__slots__`'lu düğüm ≈ 80 bayt (3.11, int dahil)
Ayırma hatası	NULL döner	`std::bad_alloc` fırlatılır	`MemoryError`

C++: RAII ve unique_ptr. C++'ta sahiplik türün parçasıdır: `Liste` yapısının yıkıcısı kapsam bitince düğümleri siler, kopyalaması silinmiştir (`Liste(const Liste&) = delete`) ki iki nesne aynı

düğümlere sahip olmasın. `std::unique_ptr<Dugum> sonraki` her düğümü sonrakinin **tek sahibi** yapar. Tuzak: zincirin varsayılan yıkıcısı **özyinelemelidir**; baş silinirken sonraki silinir, o da sonrakini siler… n = 10^6 düğümde çağrı yığını taştı (Hata Avcısı 9). Örnek 6'nın `UListe`'si yıkıcıda döngüyle koparır:

C++ `cpp/H08_06_liste_yok_etme.cpp · satır 152–162`

```cpp
struct UListe {
    std::unique_ptr<UDugum> bas;
    // Varsayılan yıkıcı bas'ı siler, o da sonraki'yi siler, ...: n
    // derinlikte özyineleme (büyük n'de yığın taşar). Döngüyle koparırız:
    ~UListe()
    {
        while (bas != nullptr) {
            bas = std::move(bas->sonraki);   // eski baş burada silinir
        }
    }
};
```

C++ kütüphane yapıları (Kavram 6, `kavram/H08_K06_vector_list.cpp`): `vector kapasiteleri (bu derleyicide): 1 2 4 8 16 32 64 128`; standart yalnızca `push_back`'in amortize sabit zamanlı olmasını ister, çarpanı belirtmez (ISO/IEC, 2017). `std::list` ve `std::forward_list` düğümleri taşımadan `reverse`, `merge` ve `sort` yapar: `list.merge: 1 2 3 3 5 6`, `merge sonrası b boş mu? evet` (düğümler a'ya taşındı), `forward_list.sort: 1 3 4 5 8`.

Python: list bir dinamik dizidir (Kavram 7). CPython `list.append`'de fazladan yer ayırır; `sys.getsizeof` liste nesnesinin ve başvuru dizisinin boyunu verir (öğeleri saymaz) ve yuva sayısı değiştikçe büyür. CPython kaynak kodunda (`Objects/listobject.c`, `list_resize`) yeni yuva sayısı (n + (n >> 3) + 6) & ~3'tür, büyük boyda ≈ × 1,125 (Python Software Foundation, t.y.-b); bu makinede gözlenen değerler 3.10–3.13'te formülle birebir aynıydı. `insert(0, x)` ve `pop(0)` bütün öğeleri kaydırır; iki uçta O(1) işlem için `collections.deque` vardır (Python Software Foundation, t.y.-a):

Kavram 7 (`kavram/H08_K07_list_deque.py`, CPython 3.11): `getsizeof([]) = 56 bayt`; `append` ile yuva sayısının değiştiği uzunluklar (uzunluk → yuva): 1 → 4, 5 → 8, 9 → 16, 17 → 24, 25 → 32, 33 → 40. `deque (appendleft 3, 1, 4; append 9): [4, 1, 3, 9]`.

Python düğümünde `__slots__` ve None (Kavram 8, `kavram/H08_K08_slots_none.py`): `__slots__` nesne başına `__dict__` sözlüğünü kaldırır (`__dict__ var mı? slots: False · sözlüklü: True`); düğüme sonradan alan eklenemez ama bellek azalır: CPython 3.11'de 10^5 düğümde `DugumSlots : düğüm başına ≈ 80 bayt`, `DugumSozluk : düğüm başına ≈ 128 bayt` (tracemalloc; değerler sürüme bağlıdır). Boş bağlantı `is None` ile, düğüm kimliği `is` ile denetlenir: `a.veri == b.veri: True · a is b: False`.

9. Mühendis Gözüyle: Önce Ölç, Sahipliği Yaz

- **Önce ölç.** Karmaşıklık tablosu "liste ortaya O(1)'de ekler" der; ölçüm, konum bulmanın maliyeti eklemenin önünde olduğu için dizinin bu makinede 73 kata kadar hızlı olduğunu gösterdi. Kuramsal üstünlük önbellek gerçekliğiyle çelişebilir; karar, iş yükünü temsil eden bir ölçümle verilir (Bölüm 6).
- **Bellek sahipliği arayüz belgesinde yazılı olmalıdır.** Bir fonksiyonun döndürdüğü belleği kimin bırakacağı imzadan anlaşılmaz. Her ayıran fonksiyonun yorumuna sahiplik sözleşmesini yazın; ekip arkadaşınız (ve altı ay sonraki siz) ancak böyle doğru kullanır. Mini-OBS 2 v2.7 bunu bir tabloyla ister.

- **Araçları sürekli çalıştırın.** Sızıntı ve sarkan işaretçi çoğu zaman sessizdir. Testleri ASan ve Valgrind ile çalıştırmak bu haftanın her programında yapıldı (73/73 temiz); bir projede bu, her değişiklikte çalışan bir `make asan` hedefidir.
- **Sınır durumları birer test vakasıdır.** Boş liste, tek düğüm, baş ve son düğüm her işlev için ayrı testtir; kâhinli rastgele testler (Örnek 5, 8, 9, 12) elle düşünülmemiş durumları da bulur.
- **Modelin varsayımını sorgulayın.** Kopya sayacı `realloc`'un her büyütmede taşıdığını varsayar; bu makinedeki ayırıcı ölçülen tekrarlarda bloğu taşımadı (Bölüm 6.4). Model ile ölçüm ayrıldığında hata ölçümde değil, modelin varsayımındadır; raporda ikisini birlikte ve açıklamasıyla verin.

10. Mini-OBS 2 v2.7: Bekleme Listesi ve Bağlı Listede Birleştirme

Bu hafta: Mini-OBS'ye ilk bağlı yapıyı ekliyoruz: kontenjanı dolu bir dersin **bekleme listesi** (tek bağlı liste + kuyruk işaretçisi) ve kontenjan açılınca **otomatik aktarma**; iki şubenin sıralı kayıt listelerini **bağlı listede, kukla düğümle, düğüm taşımadan** birleştiriyoruz; kayıt dizisinin **büyüme politikasını** belgeleyip **kopyalama sayacı** ekliyoruz. Bütün yapıların **bellek sahipliğini** bir tabloyla yazıyor, **ASan** ve **Valgrind** ile sızıntısız olduğunu gösteriyoruz. Ölçümde bekleme listesi işlemlerini dizi ve bağlı listeyle karşılaştırıyoruz (Stroustrup deneyi).

> MİNİ-OBS 2 PROJESİ · **Ara proje teslimi bu hafta**
>
> Ara proje **H8'in ilk dersinin başı** teslim edilir; kesin saat ve teslim ortamı derste ve ders sayfasında duyurulur. Kapsam: Mini-OBS 2 v2.3–v2.6 (H4–H7: arama ve sıralama). Teslim edilecekler (yönerge § 6):
>
> - **Kod:** `v2.6/` — kendi başına derlenir; `make temiz` yapılmış hâliyle.
> - **Testler:** `test_obs.c`, `test_kahin.c` (kasıtlı hata gösterimleriyle) ve `testler/` girdileri.
> - **Ölçüm verisi:** `olcum/*.csv` ve `olcum/OLCUM_ORTAMI.txt` — raporda kullanılan çalıştırmanın kendisi.
> - **Rapor:** `rapor/ara_proje_raporu.pdf`, Markdown kaynağı, şekiller ve tabloları/grafikleri üreten betik.
> - `v2.6/RAPOR.md`: v2.3–v2.6 kabul ölçütleri ✓/✗ (her sürüm belgesinin listesi) ve bilinen eksikler.
> - **Beyan:** raporun Ek C'si — yararlanılan kaynaklar, referans sürümden alınan dosyalar, kullanılan araçlar ve amaçları.
>
> Teslimden önce yönergenin § 11'indeki 8 maddelik denetim listesini temiz bir kopyada uygulayın; derleme ürünleri ve büyük veri dosyaları teslime konmaz (yönerge § 6). v2.7 ara projenin parçası değildir; v2.6'nın üzerine kurulur.

10.1 Gereksinimler ve ipuçları

- `dinamik.h/.c`: `NoDizisi` — `no_dizisi_ekle` (× 2; `realloc` sonucu geçici işaretçiye), `no_dizisi_sil_say` (kaydırma = taşıma), `no_dizisi_bul_say`, `no_dizisi_birak`; **kopyalama sayacı** (büyütmede eski n) ve büyütme sayacı; isteğe bağlı + 1, + 10, × 1,5 politikaları.
- `bekleme.h/.c`: `BeklemeDugumu { long long no; BeklemeDugumu *sonraki; }`, `BeklemeListesi { bas, son, n }`; `bekleme_sona_ekle` (O(1)), `bekleme_bastan_cikar` (O(1)), `bekleme_sil` (`BeklemeDugumu **` ile; son düğümde `son` güncellenir), `bekleme_sira` (1'den başlayan konum, yoksa 0), `bekleme_yok_et` (yinelemeli), `bekleme_gecerli_mi` (değişmez).
- `kontenjan.h/.c`: `ders_kaydol` (KABUL / BEKLEME / ZATEN_*), `ders_birak` (kayıtlıdan çıkınca bekleme listesinin başını **otomatik aktar**; bekleyenden çıkınca numarayla sil), `ders_kontenjan_ayarla`, `ders_yok_et`, `ders_gecerli_mi`.

- `kayit_listesi.h/.c`: `kayit_listesi_kur` (kayıt kopyaları), `kayit_listeleri_birlestir` (**kukla düğüm**, çakışmada A'nın düğümü kalır, B'ninki free), `kayit_listesi_yok_et`; `birlestir --yontem liste --denetle`.
- `kayit` programı: `kaydol NO`, `birak NO`, `sira NO`, `kontenjan K`, `durum` komutları; hatalı satır numarasıyla bildirilir.
- **Bellek sahipliği tablosu** (yapı | kim ayırır | kim serbest bırakır | yaşam süresi) RAPOR.md'de; her ayıran fonksiyonun yorumunda sahiplik sözleşmesi.
- `make asan` ve `make valgrind` bütün testlerde temiz ("ERROR SUMMARY: 0 errors", "definitely lost: 0 bytes").
- Ölçüm: bekleme listesi işlemlerinde dizi ↔ bağlı liste, n = 10^3…10^5 (FIFO, numarayla silme, sıra sorgusu); en az bir seride düğümlerin dağınık yerleşimi; büyüme politikalarının kopyalama sayıları; dürüst yorum.

> **MİNİ-OBS 2 PROJESİ · İpuçları (v2.7 belgesinden seçme)**
>
> - Boş liste, tek düğüm, baş düğüm ve son düğüm: her işlemi bu dört durumda kâğıtta kutu–okla çizin. Son düğümü silince `son`'u, tek düğümü çıkarınca **hem** `bas`'ı hem `son`'u güncelleyin.
> - `BeklemeDugumu **bag = &b->bas;` ile başlayın; ilerlerken `bag = &(*bag)->sonraki;`. Silmede `*bag = silinen->sonraki;` baş düğümde de çalışır.
> - Yok ederken `sonraki`'yi `free`'den **önce** okuyun: `BeklemeDugumu *s = d->sonraki; free(d); d = s;`.
> - `realloc(p, …)` sonucunu doğrudan `p`'ye atamayın: `long long *g = realloc(p, …); if (g == NULL) { … } p = g;`.
> - Kukla düğümü yığında açın (`KayitDugumu kukla; kukla.sonraki = NULL;`); sonuç `kukla.sonraki`'dir, kukla free edilmez.
> - Öteki 4 ipucu (aktarma sırası, Valgrind için ayrı ikili, kâhin, ölçüm düzeni) v2.7 belgesinin § 10'undadır.

Merdivendeki karşılıklar: bekleme listesi = Örnek 4 + Örnek 5; şube listelerinin birleştirilmesi = Örnek 8; numara dizisi = Örnek 1–2 (× 1,5 tanımı farklıdır, Bölüm 4.1).

10.2 Kabul ölçütleri ve test senaryoları

1. `make` ve `make test` sıfır uyarıyla derlenir; v1.0'ın 99 doğrulaması ve önceki sürümlerin testleri dahil bütün birim testleri ve `test_kahin` (300 deneme) geçer.
2. × 2 politikasında n sona eklemenin kopyalama sayacı P − 1'dir (P: n'den büyük ya da eşit en küçük 2'nin kuvveti; 1 000 eklemede 1 023 kopya, 11 büyütme); + 1 politikasında n(n − 1)/2 (1 000 eklemede 499 500).
3. Bekleme listesinde 10, 20, 30 eklendikten sonra 30 (son düğüm) silinir ve 40 eklenirse liste 10, 20, 40 olur ve `son` 40'ı gösterir; 10 (baş düğüm) silinirse `bas` 20'yi gösterir; tek düğüm çıkarılınca `bas` = `son` = `NULL`.
4. `./kayit ornek_veri/ogrenciler.csv --kontenjan 3 --komutlar ornek_veri/kayit_komutlari.txt` şu özet satırını basar ve 0 ile çıkar: `Özet: 17 komut · kabul 3 · bekleme 4 · aktarma 3 · bırakma 2 · ret 1 · hata 0 · son durum: kayıtlı 5/5, bekleyen 0`; 2301010048 bırakınca "AKTARMA: 2301010046 …" satırı görünür.
5. `./kayit ornek_veri/ogrenciler.csv --komutlar testler/kayit_hatali.txt`: beş hatalı satır satır numarasıyla bildirilir, sonraki geçerli komut yine çalışır; çıkış 1.
6. `./birlestir ornek_veri/ogrenciler.csv ornek_veri/sube_b.csv --yontem liste --denetle`: 14 kayıt, 10 karşılaştırma, **0 taşıma**, 11 bağlantı; "Kâhin (v2.5 dizi

birleştirmesi): AYNI"; çıkış 0. `testler/sube_cakisma.csv` ile: "ÇAKIŞMA: 1 numara … 2301010047", çıkış 1.

7. `./test_kahin --hata-goster bas` uyuşmazlığı ve en kısa karşı örneği (`sona_ekle(1); sil(1)`) basar, 1 ile çıkar; `--hata-goster cakisma` 1 ile çıkar.
8. `make asan` "bellek hatası, tanımsız davranış ve sızıntı yok" ile biter; `make valgrind`'in her satırında "ERROR SUMMARY: 0 errors" ve "definitely lost: 0 bytes" (ya da "All heap blocks were freed") vardır.
9. RAPOR.md'de bellek sahipliği tablosu en az bekleme düğümünü, kayıtlı diziyi, kayıt dizisini ve birleştirmedeki kayıt düğümlerini kapsar.
10. Kendi makinenizde 10^5 işlemlik FIFO'da bağlı liste dizi sürümünden en az 10 kat hızlıdır (bu makinede 247 kat); numarayla silmede ise dizi bitişik listeden hızlıdır (bu makinede n = 32 000'de 4,5 kat). İki yapının karşılaştırma sayıları aynıdır.

v2.7 belgesinin § 10'unda ayrıca 12 test senaryosu (boş listede baştan çıkar, sil, sıra; sona ekle 10, 20, 30; sıra(30); sil(10) — baş düğüm; sil(30) — son düğüm; sona ekle 40; …) vardır; kendi testleriniz en azından bunları kapsamalıdır. **Verilir:** kendi v2.6'nız (ya da referans v2.6), `dinamik.h`, `bekleme.h`, `kontenjan.h`, `kayit_listesi.h` (bildirimler, sözleşmeler, sahiplik yorumları), `valgrind_denetle.sh`, `ornek_veri/kayit_komutlari.txt`, `olcer_yapi`'nın ölçüm düzeneği (kurma adımı, düğüm yerleşimi, bütçe). **Yazılır:** `dinamik.c`, `bekleme.c`, `kontenjan.c`, `kayit_listesi.c`, `kayit.c`; `birlestir.c`'ye `--yontem liste`; testler (birim + kâhin); sahiplik tablosu; ölçüm ve yorum.

11. Alıştırmalar ve Atölyeler

Alıştırmalar yedi türdedir: A) izleme ve çıktı tahmini, B) karmaşıklık, C) değişmez yazma, D) hata bulma, E) Parsons bulmacası (Parsons & Haden, 2006), F) kod yazma, G) diller arası okuma. Kodu verilen her alıştırmanın dosyası `kodlar/alistirmalar/` altındadır; önce **çalıştırmadan** kutu–ok çizerek tahmin edin, sonra çalıştırıp karşılaştırın. Liste kodlarınızı her zaman ASan ile derleyin (Bölüm 5.2). Çözümler öğretim üyesindedir.

11.1 İzleme ve çıktı tahmini (A)

Alıştırma 8.1 ★☆☆ *Çıktı tahmini*

`dugum(x, s)` x'i taşıyan yeni bir düğüm ayırır ve onu s'nin önüne bağlar (16–25. satırlar). Dört `yaz` çağrısının çıktısını kutu–ok çizerek tahmin edin. (3) ve (4) adımlarında hangi düğüm serbest bırakılır, `q` hangi bağlantıyı gösterir?

alistirmalar/H08_A01_cikti_tahmini.c · satır 38–56

```
38      Dugum *bas = NULL;
39      for (int i = 1; i <= 4; i++) {
40          bas = dugum(i * 10, bas);
41      }
42      yaz(bas);                                   /* (1) */
43      Dugum *p = bas->sonraki;
44      p->sonraki = dugum(25, p->sonraki);
45      yaz(bas);                                   /* (2) */
46      Dugum *s = bas;
47      bas = bas->sonraki;
48      free(s);
49      yaz(bas);                                   /* (3) */
50      Dugum **q = &bas;
51      while ((*q)->sonraki != NULL) {
```

```
52          q = &(*q)->sonraki;
53      }
54      free(*q);
55      *q = NULL;
56      yaz(bas);                                   /* (4) */
```

Alıştırma 8.2 ★★☆ *İzleme*

`f`'yi [1 2 3 4 5 6] ve [1 2 3 4 5] listelerinde izleyin: her turda `sira`, `p`, `son_a` ve `son_b` hangi düğümdedir? Sonuç listesini yazın ve fonksiyonun ne yaptığını bir cümleyle söyleyin. `f` yeni düğüm ayırır mı; kaç bağlantı yazar? 30. satır olmasaydı ne olurdu?

alistirmalar/H08_A02_izleme.c · satır 16–33

```
16  static Dugum *f(Dugum *bas)
17  {
18      Dugum kukla_a = {0, NULL}, kukla_b = {0, NULL};
19      Dugum *son_a = &kukla_a, *son_b = &kukla_b;
20      int sira = 0;
21      for (Dugum *p = bas; p != NULL; p = p->sonraki, sira++) {
22          if (sira % 2 == 0) {
23              son_a->sonraki = p;
24              son_a = p;
25          } else {
26              son_b->sonraki = p;
27              son_b = p;
28          }
29      }
30      son_b->sonraki = NULL;
31      son_a->sonraki = kukla_b.sonraki;
32      return kukla_a.sonraki;
33  }
```

11.2 Karmaşıklık (B)

Alıştırma 8.3 ★★☆ *Karmaşıklık*

Şu işlemlerin en kötü durum maliyetini dört gerçekleştirim için bir tabloda verin: (i) yalnız baş işaretçili tek bağlı liste, (ii) baş ve kuyruk işaretçili tek bağlı liste, (iii) kukla düğümlü dairesel çift bağlı liste, (iv) × 2 büyüyen dinamik dizi. İşlemler: başa ekleme, sona ekleme, baştan silme, **sondan** silme, i. öğeyi okuma, adresi bilinen düğümü silme, uzunluk (sayaç tutulmuyorsa). Kuyruk işaretçisi varken sondan silme neden hâlâ Θ(n)'dir? Hangi işlemler (iii)'te O(1)'e iner ve bunun bedeli nedir?

Alıştırma 8.4 ★★☆ *Karmaşıklık*

Bir dinamik dizi boyut kapasitenin **yarısına** inince kapasiteyi yarıya indiriyor. Kapasite 8 ve boyut 8 iken “ekle, sil, ekle, sil, …” dizisinin her işleminde kaç öğe kopyalanır? Bu politika neden amortize O(1) değildir? Kapasite **dörtte bire** inince yarıya küçültme kuralıyla ekleme ve silmenin amortize O(1) olduğunu banka yöntemiyle (Bölüm 4.1) gösterin: her işleme kaç jeton yeter?

11.3 Değişmez yazma (C)

Alıştırma 8.5 ★★☆ *Değişmez yaz*

Örnek 5'in `tumunu_sil` döngüsü (`c/H08_05_degere_gore_silme.c`, 127–142. satırlar) için p cinsinden bir değişmez yazın (p neyi gösterir, p'den önceki düğümler hakkında ne bilinir?). Silme dalında p neden ilerlemez; ilerleseydi [1 1 2] listesinde ne olurdu? Döngünün sonlandığını gösteren azalan niceliği yazın.

Alıştırma 8.6 ★★☆ *Değişmez yaz*

Örnek 9'un `ortadaki` fonksiyonunda (`c/H08_09_iki_isaretci.c`, 67–76. satırlar; Bölüm 5.3) k. turun sonunda `yavas` ve `hizli`'nin 0 tabanlı indisleri nedir? Bu bağıntıyı değişmez olarak yazın ve döngü bittiğinde yavaşın $\lfloor n/2 \rfloor$ indisinde olduğunu çift ve tek n için ayrı ayrı gösterin. `ilk_orta`'da hızlı bir adım önden başlayınca değişmez nasıl değişir?

11.4 Hata bulma (D)

Alıştırma 8.7 ★★☆ *Hata bul*

`sondan_k` sözleşmesi Örnek 9'unkiyle aynıdır (k = 1 son düğüm; k < 1 ya da k > n ise NULL), ama program [10 20 30 40 50] listesinde bazı k'lar için yanlış düğüm döndürüyor. Çalıştırmadan önce k = 1 için ne döndüğünü izleyin; hatalı satırı bulup düzeltin. Hangi k değerlerinde hata görünmez ve bu, test seçimi hakkında ne söyler?

alistirmalar/H08_A07_hata_bul_sondan_k.c · satır 16–34

```
16  static Dugum *sondan_k(Dugum *bas, int k)
17  {
18      if (k < 1) {
19          return NULL;
20      }
21      Dugum *ileri = bas;
22      for (int i = 0; i <= k; i++) {
23          if (ileri == NULL) {
24              return NULL;
25          }
26          ileri = ileri->sonraki;
27      }
28      Dugum *geri = bas;
29      while (ileri != NULL) {
30          ileri = ileri->sonraki;
31          geri = geri->sonraki;
32      }
33      return geri;
34  }
```

Alıştırma 8.8 ★★☆ *Hata bul*

Bu `birlestir` bazı girdilerde düğüm kaybediyor (sonuç kısa çıkıyor). `main` üç girdi dener: {1, 4, 6} + {2, 3, 9}, {5, 8} + {1, 2, 3} ve {1, 2} + {7, 8, 9}. Hangi girdilerde düğüm kaybolur, hangisinde hata görünmez ve neden? Hatalı satırı Örnek 8 ile karşılaştırarak düzeltin. Düğümler `malloc` ile ayrılmış olsaydı bu hata hangi araçla, hangi raporla görünürdü?

alistirmalar/H08_A08_hata_bul_birlestir.c · satır 17–33

```
17  static Dugum *birlestir(Dugum *a, Dugum *b)
18  {
19      Dugum kukla = {0, NULL};
20      Dugum *son = &kukla;
21      while (a != NULL && b != NULL) {
22          if (a->veri <= b->veri) {
23              son->sonraki = a;
24              a = a->sonraki;
25          } else {
26              son->sonraki = b;
27              b = b->sonraki;
28          }
29          son = son->sonraki;
30      }
31      son->sonraki = a;
32      return kukla.sonraki;
33  }
```

11.5 Parsons (E)

Alıştırma 8.9 ★★☆ *Parsons*

Aşağıdaki satırlar kukla düğümlü dairesel çift bağlı listede yeni düğümü `konum`'un **önüne** ekleyen fonksiyonun karıştırılmış satırlarıdır; iki satır fazladır (dosya bu hâliyle derlenmez). Doğru sırayı ve girintiyi yazın; fazla iki satır neden yanlıştır? Dört bağlantı atamasından hangisi `konum->onceki = d;`'den **sonra** yapılırsa liste bozulur? Sonuç Örnek 11'in `once_ekle`'siyle aynı bağlantıları kurmalıdır.

`alistirmalar/H08_A09_parsons_once_ekle.c · satır 7–21`

```
    konum->onceki = d;
    d->veri = x;
    return d;
CDugum *once_ekle(CDugum *konum, int x)
    konum->onceki->sonraki = d;
    konum->sonraki = d;
}
    d->sonraki = konum;
    CDugum *d = malloc(sizeof *d);
    d->sonraki = konum->sonraki;
{
    d->onceki = konum->onceki;
    if (d == NULL) {
        return NULL;
    }
```

11.6 Kod yazma (F)

Alıştırma 8.10 ★☆☆ *Kod yaz*

Tek bağlı liste için iki fonksiyon yazın: `int say(const Dugum *bas, int x)` x'in kaç kez geçtiğini, `Dugum *indisteki(Dugum *bas, int i)` 0 tabanlı i. düğümü (yoksa NULL) döndürsün. Boş liste, i < 0 ve i ≥ n sınır durumlarını sınayın; ortak rastgele üreteçle (tohum 42) kurulan listelerde sonuçları aynı değerleri tutan bir diziyle (kâhin) karşılaştırın. C++ ve Python sürümlerini de yazın.

Alıştırma 8.11 ★★☆ *Kod yaz*

Sıralı bir listeden yinelenen düğümleri silen `int yinelenenleri_sil(Dugum *bas)` fonksiyonunu Θ(n) zaman ve O(1) ek bellekle yazın; silinen düğümleri serbest bırakın ve silinen sayısını döndürün ([1 1 2 3 3 3 4 5 5] → [1 2 3 4 5]). Kâhinle rastgele sınayın ve program sonunda ayrılan ve bırakılan düğüm sayılarının eşit olduğunu gösterin (ASan temiz). Liste sıralı olmasaydı aynı işi hangi maliyetle yapabilirdiniz?

Alıştırma 8.12 ★★★ *Kod yaz*

Tek bağlı listenin **palindrom** olup olmadığını O(1) ek bellekle bulan bir fonksiyon yazın: ilk ortayı yavaş–hızlıyla bulun (Örnek 9), ikinci yarıyı ters çevirin (Örnek 7), iki yarıyı karşılaştırın ve ikinci yarıyı **geri çevirin** (liste bozulmadan kalmalı). Tek ve çift uzunlukta, boş ve tek düğümlü listelerde sınayın; kâhin olarak listeyi diziye çevirip karşılaştırın ve fonksiyondan sonra listenin değişmediğini de denetleyin.

11.7 Diller arası okuma (G)

Alıştırma 8.13 ★★☆ *Diller arası okuma*

C++ programı ve Python eşi (`alistirmalar/H08_A13_okuma.py`: aynı düğüm zinciri, `__slots__`'lu sınıfla) ne yazdırır? Düğümleri kim, ne zaman serbest bırakır? Bağımsız değişkenle `./H08_A13_okuma 1000000` çalıştırılırsa C++'ta ne olabilir, neden? Python sürümünde aynı soru neden doğmaz? C++'ta sorunu düzeltmek için ne yazardınız (Bölüm 8)? Dosyalar: `alistirmalar/H08_A13_okuma.cpp` (düğüm: `std::unique_ptr<Dugum> sonraki`; liste n'den 1'e başa ekleyerek kurulur) ve `alistirmalar/H08_A13_okuma.py`.

Alıştırma 8.14 ★★☆ *Diller arası okuma*

Örnek 6'nın 5. bölümü üç dilde farklıdır (Ek A.6). Üç çıktıyı yan yana okuyun ve şu soruları yanıtlayın: (a) C'de `orta` neden sarkar ve program onu neden okumaz? (b) C++'ta `Liste`'nin yıkıcısı kapsam sonunda çalıştıktan sonra ham `orta` işaretçisi ne durumdadır; `std::unique_ptr` ile tutulsaydı ne değişirdi? (c) Python'da `bas = None` yapıldıktan sonra neden 3 düğüm canlı kalır; döngülü üç düğüm neden `gc.collect()`'e kadar yaşar? (d) Üç dilde `ayrılan` sayaçları neden farklıdır?

11.8 Atölye 1 · Kutu–ok izleme: altı liste işlemi (sınıf, ≈ 15 dk)

ALGORİTMA ATÖLYESİ · **Kâğıtta altı işlem**

Boş bir listede (baş ve kuyruk işaretçili, Kavram 2'nin sözleşmesiyle) şu işlemleri sırayla uygulayın ve **her** işlemden sonra listeyi kutu–okla çizin: `basa_ekle(2)`, `sona_ekle(7)`, `sona_ekle(9)`, `konuma_ekle(1, 5)`, `bastan_sil`, `sil(9)`, `sona_ekle(8)`, `ters_cevir`. Altı işlem türünün (başa, sona, araya ekleme; baştan ve değere göre silme; ters çevirme) her biri en az bir kez geçer.

- Her çizimde değişen bağlantıyı kırmızıyla işaretleyin; `bas` ve `son` etiketlerini kutuların dışına yazın. Yürünen düğüm sayısını (ziyaret) ve yazılan `sonraki` alanı sayısını (bağlantı) not edin.
- `sil(9)` son düğümü siler: `son` hangi düğüme çekilir? `ters_cevir`'den sonra `bas` ve `son` ne olur?
- Bitince `alistirmalar/H08_A11_kutu_ok_atolye.c`'yi çalıştırıp çizimlerinizi ve sayılarınızı karşılaştırın; web sayfasındaki kutu–ok oynatıcısıyla aynı işlemleri adım adım yineleyin. Sınır durumları bölümünü (boş listede silme, liste kısayken araya ekleme) de önce kâğıtta tahmin edin.

11.9 Atölye 2 · Liste kütüphanesi ve birim testleri (eşli, ≈ 30 dk)

`alistirmalar/H08_A15_liste_kutuphanesi/` klasöründe opak bir liste kütüphanesinin **arayüzü** (`liste.h`) ve 6 test fonksiyonu içeren birim testleri (`liste_test.c`, `DENETLE` makrosu başarısız koşulu satır numarasıyla yazar) verilir. Gerçekleştirimi (`liste.c`) siz yazacaksınız.

Arayüzdeki işlevler: `liste_olustur`, `liste_yok_et`, `liste_uzunluk`, `liste_basa_ekle`, `liste_sona_ekle`, `liste_bastan_cikar`, `liste_sil`, `liste_icerir`, `liste_ters_cevir`, `liste_diziye`. `typedef struct Liste Liste;` tipi opaktır (Bölüm 3.2); her işlevin karmaşıklık ve sahiplik sözleşmesi yorumunda yazılıdır (ör. `liste_sil`: "Son düğüm silinirse kuyruk işaretçisi güncellenir").

> ALGORİTMA ATÖLYESİ · **Eşli çalışma: gerçekleştiren ve sınayan**
>
> - **İlk 15 dk:** A, `liste.c`'yi yazar (yapı: `bas`, `son`, `uzunluk`); B, `liste_test.c`'ye en az üç yeni `test_` fonksiyonu ekler: boş listede her işlem, son düğüm silindikten sonra sona ekleme, tek düğümde baştan çıkarma, ters çevirmeden sonra kuyruk işaretçisi. Testler arayüze göre yazılır; B, A'nın kodunu görmeden yazar.
> - **Sonraki 15 dk:** roller değişir. Testler ASan ile derlenir (komut `liste.h`'nin başında); her başarısız `DENETLE` satırı ve her ASan/LSan raporu birlikte okunur. Hedef: bütün testler geçer, rapor temiz.
> - **Tartışma:** Hangi test hangi hatayı yakaladı? Hangi sözleşme maddesi (ör. "son düğüm silinirse kuyruk işaretçisi güncellenir") test edilmeden kaldı? Arayüze bir işlem ekleyecek olsanız (`liste_ekle_sirali`), sözleşmesini ve sahipliğini nasıl yazardınız?

11.10 Atölye 3 · Sızıntı avı (bireysel ya da eşli, ≈ 20 dk)

> ALGORİTMA ATÖLYESİ · **Bilinçli olarak hatalı liste kodunu ASan ile düzeltin**
>
> `alistirmalar/H08_A14_sizinti_avi.c` doğru görünen çıktılar verir, ama belleği eksik bırakır (dosyanın başında "BU DOSYA KASITLI HATA İÇERİR" yazar). Programda birden fazla sızıntı vardır ve hepsi farklı türdendir.
>
> - Dosya başındaki komutla ASan + LSan açık derleyin ve çalıştırın. Raporda kaç blok ve kaç bayt sızdığını, her "Direct leak" ve "Indirect leak" bloğunun yığın izinde **kendi kodunuzdaki** ilk satırı bulun.
> - LSan bloğun nerede **ayrıldığını** söyler, nerede **kaybedildiğini** değil. Her blok için "bu düğüme son işaret eden işaretçi ne zaman kayboldu?" sorusunu sorun ve hatalı satırı bulun: yok etme, silme, kopyalama ve `main`'in kendisi.
> - Bir hatayı düzeltin, yeniden derleyin ve raporun nasıl küçüldüğünü izleyin; rapor tamamen temizlenene kadar yineleyin. Sonra sanitizer'sız `-O0 -g` derleyip Valgrind ile `definitely lost: 0 bytes` ve `ERROR SUMMARY: 0 errors` görün.
> - Son olarak her ayıran fonksiyonun sahiplik yorumunu okuyun: hangi hata yorumla çelişen bir kullanımdı, hangisi yorumun kendisindeki eksiklikti?

12. Kendini Değerlendirme

Aşağıdakilerin her birini kitaba bakmadan yapabiliyorsanız işaretleyin; yapamadıklarınız için parantez içindeki bölüme dönün.

- ☐ Dinamik dizide boyut ve kapasiteyi ayırt ediyor, `realloc`'u geçici işaretçiyle güvenli kullanabiliyorum. (3.4, 4.1, HA5)

- ☐ Sona eklemenin amortize O(1) olduğunu toplam yöntemiyle gösterebiliyor, banka sezgisini anlatabiliyor ve sabit miktarda büyütmenin neden $\Theta(n^2)$ toplam iş verdiğini açıklayabiliyorum. (4.1)
- ☐ Tek bağlı listede başa, sona ve araya ekleme ile silmeyi boş liste, tek düğüm, baş ve son düğüm durumlarında doğru kodlayabiliyorum. (4.2, HA1)
- ☐ `Dugum **` ile baş işaretçisini özel durumsuz güncelleyebiliyor; arayüz (.h) ile gerçekleştirimi (.c) ayırıp sahiplik sözleşmesi yazabiliyorum. (3.2, 4.2, 5.1)
- ☐ Listeyi ters çevirebiliyor, iki sıralı listeyi kukla düğümle kararlı birleştirebiliyor ve listede birleştirmeli sıralamayı yazabiliyorum. (4.3, 4.4, 4.6)
- ☐ Yavaş–hızlı işaretçiyle ortadaki ve sondan k'ıncı düğümü bulabiliyor; Floyd'un iki evresinin neden doğru olduğunu açıklayabiliyorum. (4.5, 5.3)
- ☐ Dizi ile listeyi bir iş yükünde ölçüp sonucu konum bulma maliyeti ve önbellekle açıklayabiliyorum. (6, Örnek 13)
- ☐ Sızıntı ve sarkan işaretçi hatalarını ASan ve Valgrind raporlarından bulabiliyorum; C, C++ ve Python'un bellek yönetimini karşılaştırabiliyorum. (5.2, 7, 8)

13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · Bu haftanın konusu nerede derinleşiyor?

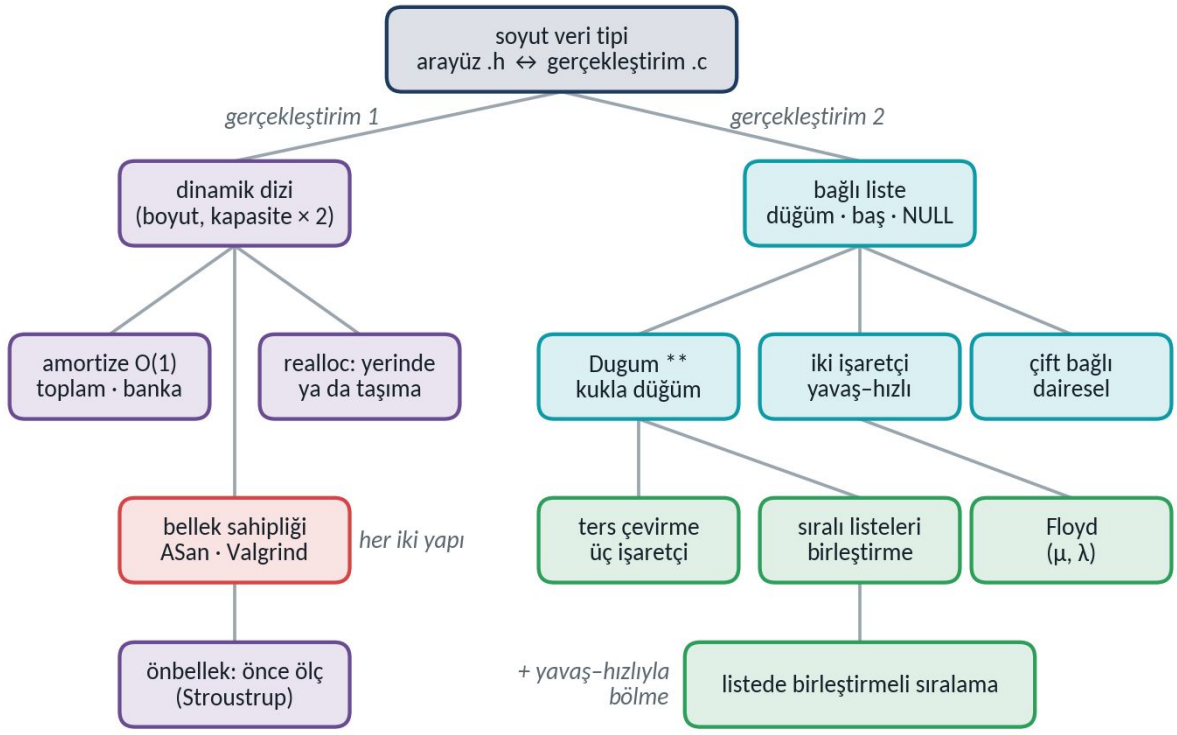
- H9 (yığın ve kuyruk): iki soyut veri tipinin bağlı liste gerçekleştirimi bu haftanın başa ekleme, baştan çıkarma ve kuyruk işaretçisidir; dizi gerçekleştirimi ise dinamik dizi ve dairesel tampondur. Hangisinin seçileceği Bölüm 6'nın sorusudur.
- H10 (karma tabloları): zincirlemeli karma tablosunda her kova bir bağlı listedir; tablo doldukça yeniden boyutlandırılır ve bu işlemin amortize maliyeti Bölüm 4.1'in analiziyle aynıdır.
- Bağlı yapılar; Veri Yapıları dersinde ağaçlar (BST, AVL), atlamalı listeler ve graf temsilleri için temel oluşturur.
- Bellek ayırıcıların (malloc) iç yapısı İşletim Sistemleri dersinde ele alınır.

MERAK KUTUSU · Düğümleri tek bir blokta ayırsak? Havuz ayırıcı ve açılmış listeler

Haftanın Deneyi'nde aynı liste kodu düğüm başına `malloc` ile 23,45 s, bütün düğümler tek bir blokta ve sıralı yerleşimle 3,13 s sürdü ($n = 5 \cdot 10^4$; 7,5×). Gerçek programlarda düğümler sık sık bir **havuzdan** (arena) ayrılır: büyük bir blok bir kez ayrılır, düğümler sırayla ondan dağıtılır, iş bitince blok tek seferde bırakılır. Bu, `malloc`'un düğüm başına yükünü kaldırır ve düğümleri bitişik tutar; ama tek tek düğüm bırakmak artık kolay değildir. Öteki yol düğüm başına birkaç öğe saklamaktır (**açılmış bağlı liste**, unrolled linked list). Soru: düğüm başına k öğeli bir listede Stroustrup iş yükünün konum bulma adımı kaç önbellek satırına dokunur; dizinin 321,7 ms süresine ne kadar yaklaşılabilir? Kendi programınızla ölçün.

14. Özet, Kavram Haritası ve Terim Sözlüğü

Şekil 8.14
Hafta 8 kavram haritası



- Dinamik dizi bitişik bir blokta boyut ve kapasite tutar; dolunca × 2 büyür. n eklemenin toplam kopyası $2^m - 1 < 2n$: sona ekleme amortize O(1). Sabit miktarda büyütme $\Theta(n^2)$ toplam iş verir.
- Soyut veri tipi işlemleri ve sözleşmeleri tanımlar; C'de arayüz .h'de, gerçekleştirim .c'dedir. Opak tip istemciyi gerçekleştirimden ayırır; maliyet de sözleşmenin parçası olmalıdır.
- Tek bağlı listede başa ekleme O(1), kuyruk işaretçisiyle sona ekleme O(1), arama ve konuma erişim Θ(n). Her işlem boş, tek düğümlü, baş ve son durumlarında sınanır.
- `Dugum **` bir bağlantıyı gösterir; baş da bir bağlantı olduğu için özel durum kalkar. Kukla düğüm aynı etkiyi verir (Python'da tek yol).
- Ters çevirme (kaydet–çevir–ilerle), kukla düğümlü kararlı birleştirme, yavaş–hızlı işaretçi, Floyd döngü tespiti (buluşma $i \equiv 0 \bmod \lambda$) ve listede birleştirmeli sıralama (O(log n) yığın, ek dizi yok) iki işaretçi tekniğinin uygulamalarıdır.
- Çift bağlı dairesel liste adresi bilinen düğümü O(1) siler; bedeli düğüm başına ikinci işaretçidir.
- Ortaya ekleme için liste "her zaman daha hızlı" değildir: konum bulma Θ(n) ve listede her adım dağınık bir düğüme atlamadır. Bu makinede $n = 5 \cdot 10^4$'te liste dizinin 72,9 katı sürdü; Python'da sonuç tersine döndü. Önce ölçün.
- Her bloğun bir sahibi vardır ve sahiplik arayüzde yazılır. Yok etmede `sonraki free`'den önce saklanır; döngülü liste önce kırılır. ASan, LSan ve Valgrind sessiz hataları görünür kılar.

Tablo 8.10
Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Not
dinamik dizi; boyut; kapasite	dynamic array; size; capacity	büyüme çarpanı (growth factor)
amortize analiz; toplam yöntemi; banka (muhasebe) yöntemi	amortized analysis; aggregate method; accounting (banker's) method	Cormen vd. (2022)
soyut veri tipi; arayüz; gerçekleştirim; opak tip	abstract data type (ADT); interface; implementation; opaque type	.h / .c
düğüm; bağlantı; baş; kuyruk işaretçisi	node; link (next pointer); head; tail pointer	"kuyruk" H9'da veri yapısıdır
kukla düğüm	dummy (sentinel) node	veri taşımaz
işaretçiye işaretçi	pointer to pointer (`Node **`)	bir bağlantının adresi
Floyd döngü tespiti (kaplumbağa ve tavşan)	Floyd's cycle detection (tortoise and hare)	μ: başlangıç, λ: uzunluk
sahiplik; yaşam döngüsü; takma ad	ownership; lifetime; alias	RAII (C++)
sarkan işaretçi; sızıntı; serbest bırakma sonrası kullanım	dangling pointer; memory leak; use-after-free	ASan, LSan, Valgrind
öbek (bellek bölgesi); yığın (çağrı yığını)	heap; (call) stack	öbek veri yapısıyla karıştırmayın (H7)
önbellek satırı; önbellek kaçırması; ön-getirme	cache line; cache miss; prefetching	Kaputun Altında

15. Kaynaklar

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

ISO/IEC. (2017). *Programming languages — C++* (ISO/IEC Standard No. 14882:2017). International Organization for Standardization.

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Knuth, D. E. (1997). *The art of computer programming: Vol. 1. Fundamental algorithms* (3rd ed.). Addison-Wesley.

Parsons, D., & Haden, P. (2006). Parson's programming puzzles: A fun and effective learning tool for first programming courses. In *Proceedings of the 8th Australasian Conference on Computing Education* (Vol. 52, pp. 157–163). Australian Computer Society.

Python Software Foundation. (t.y.-a). *The Python standard library*. Erişim tarihi 12 Ekim 2026, https://docs.python.org/3/library/

Python Software Foundation. (t.y.-b). *CPython: Objects/listobject.c* (v3.11.15) [Kaynak kod]. GitHub. Erişim tarihi 13 Ekim 2026, https://github.com/python/cpython/blob/v3.11.15/Objects/listobject.c

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Stroustrup, B. (2014). *Are lists evil?* Standard C++ Foundation. https://isocpp.org/blog/2014/06/stroustrup-lists

Torvalds, L. (2016, February). *The mind behind Linux* [Video]. TED Conferences. https://www.ted.com/talks/linus_torvalds_the_mind_behind_linux

Ek A. Dil Karşılıkları

Bu ekte örnek merdiveninin on üç basamağının C++ ve Python sürümlerinden çekirdek fonksiyonlar, dosyadaki satır numaralarıyla verilir; tam dosyalar `kodlar/cpp/` ve `kodlar/python/` altındadır. Hardal satırlar C sürümünden **anlam olarak** farklılaşan satırlardır (`kodlar/ornekler.json` → `farklar`); fonksiyon dışındaki farklar not kutusunda açıklanır. Fonksiyon ve değişken adları üç dilde aynıdır. Bölüm 4'te Python'u gösterilen amiral fonksiyonların burada C++ sürümü, öteki örneklerin her iki dilden birer çekirdek fonksiyonu verilir. Örnek 6 dışındaki bütün örneklerin çıktısı üç dilde aynıdır (=); Örnek 6'nın dile özgü 5. bölümü A.6'dadır (Δ).

Tablo 8.11
Derleme ve çalıştırma komutları (X: örneğin kök adı, ör. H08_05_degere_gore_silme)

Dil	Derleme / çalıştırma	Not
C	`gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c ortak_rastgele.c -o X · ./X`	ortak_rastgele.c yalnızca onu kullanan örneklerde; Örnek 12'de sona -lm, Örnek 13'te zamanlayici.c de eklenir
C++	`g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X · ./X`	ortak başlıklar (.hpp) yalnızca #include
Python	`python3 X.py (Windows: py X.py)`	Python 3.10 ve üstü; yalnızca standart kütüphane
ASan	`gcc … -O1 -g -fsanitize=address,undefined -fno-omit-frame-pointer X.c -o X`	Bölüm 5.2; `python3 esdeger_test.py . --asan`
Valgrind	`gcc … -O0 -g X.c -o X · valgrind --leak-check=full ./X`	sanitizer'sız ikili
Argümanlar	`./H08_06_liste_yok_etme [n] · ./H08_07_ters_cevirme [n] · ./H08_13_dizi_mi_liste_mi --olc [n] [tekrar]`	isteğe bağlı

A.1 Örnek 1 · Dinamik dizi ADT'si

C++ **cpp/H08_01_dinamik_dizi.cpp · satır 34–50**

```
34 static void dd_ekle(DinamikDizi* d, int x)
35 {
36     if (d->boyut == d->kapasite) {
37         std::size_t yeni_kapasite = 2 * d->kapasite;
38         int* yeni = new int[yeni_kapasite];
39         for (std::size_t i = 0; i < d->boyut; i++) {
40             yeni[i] = d->veri[i];
41         }
42         kopya_sayisi += static_cast<long long>(d->boyut);
43         buyutme_sayisi++;
44         delete[] d->veri;
45         d->veri = yeni;
46         d->kapasite = yeni_kapasite;
47     }
48     d->veri[d->boyut] = x;
49     d->boyut++;
50 }
```

Python **python/H08_01_dinamik_dizi.py · satır 45–49**

```python
def dd_al(d, i):
    """Ön koşul yerine denetim: i < boyut ise öğeyi, değilse None döner"""
    if i >= d.boyut:
        return None
    return d.veri[i]
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de büyütme `realloc` ile yapılır: blok yerinde büyüyebilir ya da taşınabilir; sayaç en kötü durumu (taşınmayı, yani `boyut` kadar kopyayı) sayar, gerçekte taşınıp taşınmadığı yalnızca stderr'e yazılır. C++'ta `new[]` ile ayrılmış blok `realloc` edilemez: yeni blok + kopya döngüsü + `delete[]`; ayırma hatası dönüş değeriyle değil `std::bad_alloc` istisnasıyla bildirilir. Python'da yuvalar `[None] * kapasite` ile ayrılır, `dd_al` sınır dışında `None` döner. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 27, 53; Python 22, 34, 58.)

A.2 Örnek 2 · Büyüme stratejisi

C++ **cpp/H08_02_buyume_stratejisi.cpp · satır 24–36**

```cpp
static long long yeni_kapasite(int strateji, long long k)
{
    switch (strateji) {
    case 0:
        return k + 1;
    case 1:
        return k + 10;
    case 2:
        return k + (k + 1) / 2;
    default:
        return 2 * k;
    }
}
```

Python **python/H08_02_buyume_stratejisi.py · satır 15–24**

```python
def yeni_kapasite(strateji, k):
    """Kapasite k doluyken yeni kapasite. × 1,5'te tamsayı yuvarlaması üç
    dilde aynı tanımlıdır: k + (k + 1) // 2, yani ⌊1,5 k⌋ (k = 1 → 2)."""
    if strateji == 0:
        return k + 1
    if strateji == 1:
        return k + 10
    if strateji == 2:
        return k + (k + 1) // 2
    return 2 * k
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Strateji seçimi C ve C++'ta `switch`, Python'da `if` zinciridir. × 1,5'in tamsayı yuvarlaması üç dilde aynı tanımlıdır: `k + (k + 1) / 2` (C/C++) ile `k + (k + 1) // 2` (Python) pozitif sayılarda aynı sonucu verir. Strateji adları önceden doldurulmuş dizgilerdir, çünkü `printf`/`std::setw` genişliği BAYT sayar ve UTF-8'deki "×" 2 bayttır; Python'un f-dizgisi ise karakter sayar. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 19, 86, 121, 145; Python 102, 138.)

A.3 Örnek 3 · Tek bağlı liste: başa ekleme

C++ **cpp/H08_03_tek_bagli_liste.cpp · satır 38–44**

```cpp
38 static Dugum* basa_ekle(Dugum* bas, int x)
39 {
40     Dugum* d = dugum_olustur(x);
41     d->sonraki = bas;
42     baglanti_sayisi++;
43     return d;
44 }
```

Python **python/H08_03_tek_bagli_liste.py · satır 38–53**

```python
38 def basa_ekle(bas, x):
39     """Yeni düğüm eski başın önüne bağlanır; YENİ BAŞI döndürür: O(1)"""
40     global baglanti_sayisi
41     d = dugum_olustur(x)
42     d.sonraki = bas
43     baglanti_sayisi += 1
44     return d
45
46
47 def liste_yaz(bas):
48     parcalar = []
49     p = bas
50     while p is not None:
51         parcalar.append(f"{p.veri} -> ")
52         p = p.sonraki
53     print("[" + "".join(parcalar) + "NULL]")
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Boş bağlantı C'de `NULL`, C++'ta `nullptr`, Python'da `None`'dır; çıktıda üç dil de `NULL` yazar. C'de düğüm `malloc`, C++'ta `new` ile ayrılır; C++'taki `Liste` yapısı RAII ile sahiptir: kapsam bitince yıkıcısı düğümleri siler. Python düğüm sınıfında `__slots__` nesne başına sözlüğü kaldırır (bellek tasarrufu); `free` yoktur, `dugum_birak` yalnızca C'deki adımı sayar. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 28, 83, 110; Python 12, 16, 35.)

A.4 Örnek 4 · Sona ekleme ve arama

C++ **cpp/H08_04_sona_ekleme_arama.cpp · satır 69–79**

```cpp
69 static void sona_ekle(Liste& liste, int x)
```

```cpp
{
    Dugum* d = dugum_olustur(x);
    if (liste.son == nullptr) {  // boş liste
        liste.bas = d;
    } else {
        liste.son->sonraki = d;
        baglanti_sayisi++;
    }
    liste.son = d;
}
```

Python `python/H08_04_sona_ekleme_arama.py · satır 91–102`

```python
def ara(bas, x):
    """İlk x'i arar; (düğüm ya da None, 0 tabanlı konum ya da -1) döner"""
    global ziyaret_sayisi
    i = 0
    p = bas
    while p is not None:
        ziyaret_sayisi += 1
        if p.veri == x:
            return p, i
        i += 1
        p = p.sonraki
    return None, -1
```

DİL PENCERESİ · Farklılaşan satırlar

Arama düğümü ve konumu döndürür: C'de konum `int *` çıkış parametresiyle, C++'ta `int&` başvurusuyla, Python'da demetle (`return p, i`). Liste başlığı (`bas`, `son`) C'de `struct`, C++'ta yıkıcılı `struct`, Python'da `__slots__`'lu sınıftır; C++ `Liste` nesnesini işlevlere başvuruyla (`Liste&`) verir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 56, 101; Python 108.)

A.5 Örnek 5 · Değere göre silme

C++ `cpp/H08_05_degere_gore_silme.cpp · satır 114–127`

```cpp
static bool sil_v2(Dugum*& bas, int x)
{
    Dugum** p = &bas;
    while (*p != nullptr && !aranan_mi(*p, x)) {
        p = &(*p)->sonraki;
    }
    if (*p == nullptr) {
        return false;
    }
    Dugum* silinen = *p;
    *p = silinen->sonraki;  // baş da aynı satır
    dugum_birak(silinen);
    return true;
}
```

Python **python/H08_05_degere_gore_silme.py · satır 108–122**

```python
def tumunu_sil(bas, x):
    """Tüm x'leri siler (kukla düğümle); (yeni_bas, silinen sayısı) döner"""
    silinen_sayisi = 0
    kukla = Dugum(0)
    kukla.sonraki = bas
    onceki = kukla
    while onceki.sonraki is not None:
        if aranan_mi(onceki.sonraki, x):
            silinen = onceki.sonraki
            onceki.sonraki = silinen.sonraki  # onceki ilerlemez
            dugum_birak(silinen)
            silinen_sayisi += 1
        else:
            onceki = onceki.sonraki
    return kukla.sonraki, silinen_sayisi
```

DİL PENCERESİ · Farklılaşan satırlar

C'de çağıranın baş işaretçisini değiştirmek için adresi verilir (`Dugum **bas`); `p = &(*p)->sonraki` ile p hep "sıradaki düğümü gösteren bağlantının" adresidir, baş da bir bağlantı olduğundan özel durum kalmaz (Torvalds'ın TED 2016'da gösterdiği örnek). C++'ta baş başvuruyla (`Dugum*&`) gelir ama yürüyüşte yine `Dugum**` gerekir (başvuru yeniden bağlanamaz). Python'da işaretçiye işaretçi yoktur: başın önüne geçici kukla düğüm konur ve yeni baş döndürülür. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 130; Python 95, 105, 159.)

A.6 Örnek 6 · Listenin yok edilmesi ve sahiplik (Δ)

C++ **cpp/H08_06_liste_yok_etme.cpp · satır 59–68**

```cpp
static void liste_yok_et(Dugum*& bas)
{
    Dugum* d = bas;
    while (d != nullptr) {
        Dugum* sonraki = d->sonraki;
        dugum_birak(d);
        d = sonraki;
    }
    bas = nullptr;
}
```

Python **python/H08_06_liste_yok_etme.py · satır 50–58**

```python
def liste_yok_et(bas):
    """C'deki yinelemeli yok etmenin karşılığı. Çağıranın değişkeni
    değiştirilemez: çağıran ayrıca bas = None yazar (dönüş değeriyle)."""
    d = bas
    while d is not None:
        sonraki = d.sonraki
        dugum_birak(d)
        d = sonraki
    return None
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Bölüm 1–4 üç dilde aynıdır; 5. bölüm dile özgüdür. C'de `free` edilmiş düğümü gösteren işaretçi sarkar (okumak tanımsız davranış). C++'ta `Liste`'nin yıkıcısı kapsam sonunda siler (RAII); `std::unique_ptr` zincirinin varsayılan yıkıcısı özyinelemelidir, bu yüzden `~UListe` döngüyle `bas = std::move(bas->sonraki)` yapar. Python'da `free` yoktur: CPython nesneyi son başvurusu kalkınca geri alır (`orta` tutuldukça düğümler yaşar, sarkan başvuru olmaz); döngülü yapıları ise döngüsel çöp toplayıcı (`gc`) toplar. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 147, 159, 239; Python 121, 168, 199, 216.)

ÇIKTI · C

```
$ ./H08_06_liste_yok_etme   (C, 5. bölüm)
5) C: serbest bırakılmış düğümü gösteren işaretçi SARKANDIR
  orta = bas->sonraki->sonraki; orta->veri = 4
  liste_yok_et(&bas): bas = NULL yapıldı; orta ise eski adresi tutuyor
  orta->veri okunursa tanımsız davranış (ASan: heap-use-after-free;
  bkz. Hata Avcısı 3). Çözüm: sahibi yok edince tüm takma adları
  geçersiz say (orta = NULL).
  ayrılan = 100025, bırakılan = 100025, canlı = 0
```

ÇIKTI · C++

```
$ ./H08_06_liste_yok_etme   (C++, 5. bölüm)
5) C++: RAII (yıkıcı kapsam sonunda siler) ve std::unique_ptr (tek sahip)
  { Liste liste; ... } içinde orta->veri = 4
  kapsam kapandı: yıkıcı 5 düğümü sildi; ham işaretçi orta sahip değildir,
  artık SARKANDIR (C'deki gibi; okunursa tanımsız davranış)
  unique_ptr zinciri (n = 100000): ~UListe 100000 düğümü döngüyle sildi
  (varsayılan yıkıcı özyinelemeli olurdu: derinlik n)
  ayrılan = 200025, bırakılan = 200025, canlı = 0
```

ÇIKTI · PYTHON

```
$ python3 H08_06_liste_yok_etme.py   (5. bölüm)
5) Python: free yok; nesne son başvurusu kalkınca geri alınır (CPython)
  orta = bas.sonraki.sonraki; canlı düğüm = 5
  bas = None: canlı düğüm = 3 (orta ve ardılları hâlâ erişilebilir)
  orta.veri = 4 okunabilir: sarkan başvuru yoktur
  orta = None: canlı düğüm = 0
  döngülü 3 düğüm, bas = None: canlı düğüm = 3 (başvurular birbirini tutar)
  gc.collect(): canlı düğüm = 0 (döngüsel çöp toplayıcı)
  ayrılan = 100020, bırakılan = 100020, canlı = 0
```

A.7 Örnek 7 · Ters çevirme

C++ cpp/H08_07_ters_cevirme.cpp · satır 96–108

```
96  static Dugum* ters_cevir(Dugum* bas)
97  {
98      Dugum* onceki = nullptr;
99      Dugum* gecerli = bas;
100     while (gecerli != nullptr) {
101         Dugum* sonraki = gecerli->sonraki; // kaydet
```

```
102         gecerli->sonraki = onceki;         // çevir
103         baglanti_sayisi++;
104         onceki = gecerli;                  // ilerle
105         gecerli = sonraki;
106     }
107     return onceki;
108 }
```

Python **python/H08_07_ters_cevirme.py · satır 110–122**

```
110 def ters_cevir_ozyinelemeli(bas, derinlik):
111     """Önce kuyruğu ters çevir, sonra bas'ı onun sonuna tak. Dönüşte
112     bas.sonraki, ters kuyruğun SON düğümüdür."""
113     global baglanti_sayisi, en_derin
114     if derinlik > en_derin:
115         en_derin = derinlik
116     if bas is None or bas.sonraki is None:
117         return bas                          # 0 ya da 1 düğüm
118     yeni_bas = ters_cevir_ozyinelemeli(bas.sonraki, derinlik + 1)
119     bas.sonraki.sonraki = bas
120     bas.sonraki = None
121     baglanti_sayisi += 2
122     return yeni_bas
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Yinelemeli sürüm üç dilde satır satır aynıdır; Python'da da önce `sonraki` saklanmalıdır (yoksa bağlantı kaybolur). Özyinelemeli sürüm n derinlik ister: Python varsayılan özyineleme sınırında (1000) `RecursionError` verir; C ve C++ bu makinede (-O2, 8 MB yığın) n = 5·10⁵'i taşır, 10⁶'da yığın taşar (isteğe bağlı argümanla: `./H08_07_ters_cevirme 1000000`). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 201; Python 80, 160, 161.)

A.8 Örnek 8 · Sıralı listeleri birleştirme

C++ **cpp/H08_08_sirali_birlestirme.cpp · satır 90–109**

```
90  static Dugum* birlestir(Dugum* a, Dugum* b)
91  {
92      Dugum kukla{0, 0, nullptr};   // yığında: ayrılmaz, bırakılmaz
93      Dugum* son = &kukla;          // sonucun son düğümü
94      while (a != nullptr && b != nullptr) {
95          karsilastirma_sayisi++;
96          if (a->veri <= b->veri) {
97              son->sonraki = a;
98              a = a->sonraki;
99          } else {
100             son->sonraki = b;
101             b = b->sonraki;
102         }
103         baglanti_sayisi++;
104         son = son->sonraki;
105     }
106     son->sonraki = (a != nullptr) ? a : b;   // kalan kuyruk tek bağlantıyla
```

```
107      baglanti_sayisi++;
108      return kukla.sonraki;
109  }
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Kukla düğüm C ve C++'ta yığında yerel değişkendir (`Dugum kukla`; ayrılmaz, serbest bırakılmaz), Python'da geçici bir nesnedir. Birleştirmeden sonra a ve b'nin düğümleri sonucun olur: C'de `a = b = NULL`, C++'ta `a.bas = b.bas = nullptr` (aksi hâlde yıkıcılar aynı düğümleri iki kez silerdi). Kâhin C'de `qsort`, C++'ta `std::sort`, Python'da `sorted`'dır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 168, 219; Python 74, 86, 173.)

A.9 Örnek 9 · Ortadaki ve sondan k'ıncı düğüm

C++ **cpp/H08_09_iki_isaretci.cpp · satır 70–80**

```
70  static Dugum* ortadaki(Dugum* bas)
71  {
72      Dugum* yavas = bas;
73      Dugum* hizli = bas;
74      while (hizli != nullptr && hizli->sonraki != nullptr) {
75          yavas = yavas->sonraki;
76          hizli = hizli->sonraki->sonraki;
77          ilerletme_sayisi += 3;
78      }
79      return yavas;
80  }
```

Python **python/H08_09_iki_isaretci.py · satır 81–98**

```
81  def sondan_k(bas, k):
82      """Sondan k'ıncı (k = 1: son düğüm). Sözleşme: k < 1 ya da k > n ise
83      None. ileri, k adım önden gider; ileri None olunca geri hedeftedir."""
84      global ilerletme_sayisi
85      if k < 1:
86          return None
87      ileri = bas
88      for _ in range(k):
89          if ileri is None:
90              return None                  # liste k'dan kısa
91          ileri = ileri.sonraki
92          ilerletme_sayisi += 1
93      geri = bas
94      while ileri is not None:
95          ileri = ileri.sonraki
96          geri = geri.sonraki
97          ilerletme_sayisi += 2
98      return geri
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Düğüm kimliği C ve C++'ta adres karşılaştırmasıyla (`p == d`), Python'da `is` ile denetlenir (`==` değer eşitliği içindir). Python'un zincirleme karşılaştırması `1 <= k <= n` C'de `k >= 1 && k <= n` olarak yazılır. Hücre metni C'de `snprintf` ile tampona, C++'ta `std::to_string` ile, Python'da f-dizgisiyle kurulur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 145, 193; Python 131, 160, 178.)

A.10 Örnek 10 · Floyd döngü tespiti

C++ **cpp/H08_10_floyd_dongu.cpp · satır 105–120**

```cpp
static Dugum* dongu_baslangici(Dugum* bas, Dugum* bulusma, bool izle)
{
    Dugum* kaplumbaga = bas;
    Dugum* tavsan = bulusma;
    while (kaplumbaga != tavsan) {
        kaplumbaga = kaplumbaga->sonraki;
        tavsan = tavsan->sonraki;
        adim_sayisi++;
        if (izle) {
            std::cout << "  adım " << adim_sayisi << ": kaplumbağa = "
                      << kaplumbaga->veri << ", tavşan = " << tavsan->veri
                      << "\n";
        }
    }
    return kaplumbaga;
}
```

Python **python/H08_10_floyd_dongu.py · satır 64–78**

```python
def bulusma_bul(bas, izle):
    """Evre 1: kaplumbağa bir, tavşan iki adım atar. Döngü varsa döngünün
    içinde buluşurlar (buluşma düğümü döner); yoksa tavşan None'a varır."""
    global adim_sayisi
    kaplumbaga = tavsan = bas
    while tavsan is not None and tavsan.sonraki is not None:
        kaplumbaga = kaplumbaga.sonraki
        tavsan = tavsan.sonraki.sonraki
        adim_sayisi += 1
        if izle:
            print(f"  adım {adim_sayisi}: kaplumbağa = {kaplumbaga.veri}, "
                  f"tavşan = {tavsan.veri}")
        if kaplumbaga is tavsan:          # aynı NESNE mi? (is)
            return kaplumbaga
    return None
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Buluşma, işaretçilerin aynı düğümü göstermesidir: C/C++'ta `==`, Python'da `is`. `lambda` Python'da ayrılmış sözcük olduğu için değişken `lambda_` adını alır. Döngülü liste yok edilmeden önce kırılmalıdır: C'de ve C++'ın yıkıcısında aksi hâlde sonsuz döngü (ve serbest bırakılmış düğüme erişim) olur; Python'da da döngüdeki başvurular birbirini canlı tutar ve yalnız döngüsel çöp toplayıcı geri alabilir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 46, 170; Python 86, 98.)

A.11 Örnek 11 · Çift bağlı dairesel liste

C++ cpp/H08_11_cift_bagli_liste.cpp · satır 70–78

```cpp
static void dugumu_sil(CiftListe& liste, CDugum* d)
{
    d->onceki->sonraki = d->sonraki;
    d->sonraki->onceki = d->onceki;
    baglanti_sayisi += 2;
    liste.uzunluk--;
    delete d;
    birakma_sayisi++;
}
```

Python python/H08_11_cift_bagli_liste.py · satır 70–78

```python
def dugumu_sil(liste, d):
    """Adresi bilinen düğümü O(1) siler: iki bağlantı, yürüme yok, baş/son
    için özel durum yok (komşular her zaman vardır: en kötü kukla)."""
    global baglanti_sayisi, birakma_sayisi
    d.onceki.sonraki = d.sonraki
    d.sonraki.onceki = d.onceki
    baglanti_sayisi += 2
    liste.uzunluk -= 1
    birakma_sayisi += 1     # free yok; yalnız sayaç
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Kukla düğüm C'de ve C++'ta liste yapısının İÇİNDE bir alandır (ayrılmaz); C++'ta kopyalama yasaklanır, çünkü kopyada kuklanın kendini gösteren bağlantıları eski nesneyi gösterirdi. Python'da kukla ayrı bir nesnedir; yapı döngüsel olduğundan `cl_yok_et` bağlantıları açıkça koparır ki başvuru sayımı (çöp toplayıcıyı beklemeden) geri alabilsin. Tek bağlı karşılaştırmada C/C++ `Dugum **`, Python kukla düğüm kullanır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 36, 37, 138; Python 35, 88, 140.)

A.12 Örnek 12 · Listede birleştirmeli sıralama

C++ cpp/H08_12_listede_birlestirmeli.cpp · satır 132–145

```cpp
static Dugum* birlestirmeli_sirala(Dugum* bas, int derinlik)
{
    cagri_sayisi++;
    if (derinlik > en_derin) {
        en_derin = derinlik;
    }
    if (bas == nullptr || bas->sonraki == nullptr) {
        return bas;                       // 0 ya da 1 düğüm: sıralı
    }
    Dugum* ikinci = ortadan_bol(bas);
    Dugum* sol = birlestirmeli_sirala(bas, derinlik + 1);
    Dugum* sag = birlestirmeli_sirala(ikinci, derinlik + 1);
    return birlestir(sol, sag);
}
```

Python **python/H08_12_listede_birlestirmeli.py · satır 93–105**

```python
def ortadan_bol(bas):
    """En az iki düğümlü listeyi ikiye böler: ilk yarı [n/2] düğüm (yavaş–
    hızlı, hızlı bir adım önden: çift n'de BİRİNCİ orta). İkinci yarının
    başını döndürür."""
    global baglanti_sayisi
    yavas, hizli = bas, bas.sonraki
    while hizli is not None and hizli.sonraki is not None:
        yavas = yavas.sonraki
        hizli = hizli.sonraki.sonraki
    ikinci = yavas.sonraki
    yavas.sonraki = None  # ilk yarıyı kes
    baglanti_sayisi += 1
    return ikinci
```

DİL PENCERESİ · Farklılaşan satırlar

Algoritma üç dilde aynıdır; girinti C'de `printf("%*s", 2 * derinlik, "")`, C++'ta `std::string(2 * derinlik, ' ')`, Python'da `" " * (2 * derinlik)` ile yazılır. Kâhin C'de `qsort` (kararlı olduğu garanti edilmez; yalnız değerler karşılaştırılır), C++'ta `std::sort`, Python'da `sorted`'dır; kararlılık her dilde düğüm kimlikleriyle ayrıca denetlenir. Derinlik $\lceil \lg n \rceil + 1$ olduğundan Python'un özyineleme sınırı sorun olmaz ($n = 10^5$'te 18). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 153, 244; Python 127, 158, 194.)

A.13 Örnek 13 · Dizi mi liste mi?

C++ **cpp/H08_13_dizi_mi_liste_mi.cpp · satır 82–94**

```cpp
static void liste_sirali_ekle(Dugum*& bas, int x)
{
    Dugum** p = &bas;
    while (*p != nullptr) {
        liste_ziyaret_ekle++;
        if ((*p)->veri > x) {
            break;
        }
        p = &(*p)->sonraki;
    }
    *p = new Dugum{x, *p};
    ayirma_sayisi++;
}
```

Python **python/H08_13_dizi_mi_liste_mi.py · satır 42–62**

```python
def dizi_sirali_ekle(d, x):
    """Sıralı diziye x: konum DOĞRUSAL aranır (ilk > x), sağdakiler kayar"""
    global dizi_karsilastirma, dizi_kaydirma_ekle
    konum = 0
    while konum < d.boyut:
        dizi_karsilastirma += 1
        if d.veri[konum] > x:
            break
```

```
50          konum += 1
51      if d.boyut == d.kapasite:                    # × 2 büyütme
52          yeni = 1 if d.kapasite == 0 else 2 * d.kapasite
53          blok = [None] * yeni
54          for i in range(d.boyut):
55              blok[i] = d.veri[i]
56          d.veri = blok
57          d.kapasite = yeni
58      for j in range(d.boyut, konum, -1):          # sağa kaydır
59          d.veri[j] = d.veri[j - 1]
60      dizi_kaydirma_ekle += d.boyut - konum
61      d.veri[konum] = x
62      d.boyut += 1
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Sayaçlar üç dilde aynıdır; süreler yalnızca `--olc` kipinde basılır. C'de dizi `realloc` ile, C++ ve Python'da yeni blok + kopya ile büyür. C++ `--olc` kipinde aynı iş yükünü etiketli kütüphane serisiyle de ölçer: konum yine elle (doğrusal) aranır, kaydırmayı `std::vector::insert/erase`, bağlamayı `std::list::insert/erase` yapar. Python'da liste ekleme/silme işlevleri başı değiştirebildiği için yeni başı döndürür (`Dugum **` yok). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 160, 169, 180, 189; Python 88, 107, 174.)